PROGRESS MEASURES FOR GROKKING VIA MECHANISTIC INTERPRETABILITY

Neel Nanda[*,†] **Lawrence Chan**[‡] **Tom Lieberum**[†] **Jess Smith**[†] **Jacob Steinhardt**[‡]

ABSTRACT

Neural networks often exhibit emergent behavior, where qualitatively new capabilities arise from scaling up the amount of parameters, training data, or training steps. One approach to understanding emergence is to find continuous *progress measures* that underlie the seemingly discontinuous qualitative changes. We argue that progress measures can be found via mechanistic interpretability: reverse-engineering learned behaviors into their individual components. As a case study, we investigate the recently-discovered phenomenon of "grokking" exhibited by small transformers trained on modular addition tasks. We fully reverse engineer the algorithm learned by these networks, which uses discrete Fourier transforms and trigonometric identities to convert addition to rotation about a circle. We confirm the algorithm by analyzing the activations and weights and by performing ablations in Fourier space. Based on this understanding, we define progress measures that allow us to study the dynamics of training and split training into three continuous phases: memorization, circuit formation, and cleanup. Our results show that grokking, rather than being a sudden shift, arises from the gradual amplification of structured mechanisms encoded in the weights, followed by the later removal of memorizing components.

1 INTRODUCTION

Neural networks often exhibit emergent behavior, in which qualitatively new capabilities arise from scaling up the model size, training data, or number of training steps (Steinhardt, 2022; Wei et al., 2022a). This has led to a number of breakthroughs, via capabilities such as in-context learning (Radford et al., 2019; Brown et al., 2020) and chain-of-thought prompting (Wei et al., 2022b). However, it also poses risks: Pan et al. (2022) show that scaling up the parameter count of models by as little as 30% can lead to emergent reward hacking.

Emergence is most surprising when it is abrupt, as in the case of reward hacking, chain-of-thought reasoning, or other phase transitions (Ganguli et al., 2022; Wei et al., 2022a). We could better understand and predict these phase transitions by finding *hidden progress measures* (Barak et al., 2022): metrics that precede and are causally linked to the phase transition, and which vary more smoothly. For example, Wei et al. (2022a) show that while large language models show abrupt jumps in their performance on many benchmarks, their cross-entropy loss decreases smoothly with model scale. However, cross-entropy does not explain *why* the phase changes happen.

In this work, we introduce a different approach to uncovering hidden progress measures: via *mechanistic explanations*.[1] A mechanistic explanation aims to reverse engineer the mechanisms of the network, generally by identifying the circuits (Cammarata et al., 2020; Elhage et al., 2021) within a model that implement a behavior. Using such explanations, we study *grokking*, where models abruptly transition to a generalizing solution after a large number of training steps, despite initially overfitting (Power et al., 2022). Specifically, we study modular addition, where a model takes inputs $a, b \in \{0, \ldots, P-1\}$ for some prime P and predicts their sum $c \bmod P$. Small transformers trained with weight decay on this task consistently exhibit grokking (Figure 2, Appendix C.2).

*Corresponding author, please direct correspondence to: neelnanda27@gmail.com
†Independent researcher.
‡University of California, Berkeley.
[1]Interactive versions of figures, as well as the code to reproduce our results, are available at https://neelnanda.io/grokking-paper.

arXiv:2301.05217v3 [cs.LG] 19 Oct 2023

通过机制可解释性衡量理解能力提升的进展度量指标

尼爾·南达[*、†] **劳伦斯·陈**[‡] **汤姆·利伯鲁姆**[†] **杰西·史密斯**[†] **雅各布·斯坦哈特**[‡]

摘要

神经网络常常会展现出涌现行为，即通过增加参数数量、训练数据规模或训练步数，能够产生质上全新的能力。理解这种涌现现象的一种方法，是寻找那些潜藏在看似不连续的质变背后的连续<span style id='1'>进展度量指标。我们认为，这些进展度量指标可以通过机制可解释性来获得——也就是将模型学到的行为逆向拆解为各个组成要素。作为案例研究，我们探讨了近期被发现的一种现象：经过模块加法任务训练的小型Transformer模型会展现出“理解能力提升”的特性。我们对这些模型所学的算法进行了全面逆向分析，发现该算法利用离散傅里叶变换与三角恒等式，将加法运算转化为围绕圆周的旋转操作。此外，我们还通过分析激活值和权重，以及在傅里叶空间开展消融实验，进一步验证了这一算法的正确性。基于上述理解，我们定义了相应的进展度量指标，借此能够研究训练过程的动态变化，并将训练过程划分为记忆阶段、电路形成阶段和清理阶段三个连续的阶段。研究结果表明，“理解能力提升”并非突然出现，而是源于权重中编码的结构化机制逐渐被强化，随后那些用于记忆的组件又被逐步移除。

1 引言

神经网络常常会展现出涌现行为，即通过增大模型规模、训练数据量或训练步数，模型便能获得本质上全新的能力（Steinhardt，2022年；Wei等人，2022a）。这类现象催生了诸多突破性成果，例如基于上下文学习的能力（Radford等人，2019年；Brown等人，2020年）以及思维链提示技术（Wei等人，2022b）。然而，这种现象同时也存在风险：Pan等人（2022年）的研究表明，即便仅将模型的参数数量增加30%，也有可能引发奖励操纵这种涌现行为。

当某种涌现行为是突然发生的，比如奖励操纵、思维链推理或其他相变现象时，其震撼程度最为显著（Ganguli等人，2022年；Wei等人，2022a）。如果我们能够找到隐性的进展度量指标（Barak等人，2022年），也就是那些在相变发生之前就会出现且与其存在因果关联、变化更为平缓的指标，就能更好地理解并预测这些相变现象。例如，Wei等人（2022a）发现，尽管大型语言模型在许多测试基准上的性能会出现突跃，但它们的交叉熵损失值却会随着模型规模的扩大而平稳下降。不过，交叉熵损失无法解释为何会出现这些相变现象。

在本研究中，我们提出了一种不同的方法来揭示隐藏的进展度量指标：即通过机制性解释。[1] 机制性解释旨在逆向分析网络的运作机制，通常是通过识别实现某种行为的模型中的电路结构（Cammarata等人、2020年；Elhage等人、2021年）。借助这类解释，我们研究了理解能力现象，即模型在最初出现过拟合的情况下，经过大量训练步骤后会突然转变为泛化解（Power等人、2022年）。具体而言，我们研究了模加法这一场景，即模型接收输入值a、$b \in \{0, \ldots, P-1\}$，这些输入对应某个素数 P，然后预测它们在 P模 P 意义下的求和结果 c。针对这项任务并采用权重衰减策略进行训练的小型变换器始终能够展现出理解能力提升的现象（见图2，附录C.2）。

*如需联系通讯作者，请发送邮件至：neelnanda27@gmail.com
†独立研究人员。
‡加利福尼亚大学伯克利分校。[1]图表的交互版本以及用于复现我们研究结果的代码均可在以下地址获取：https://neelnanda.io/grokking-paper。

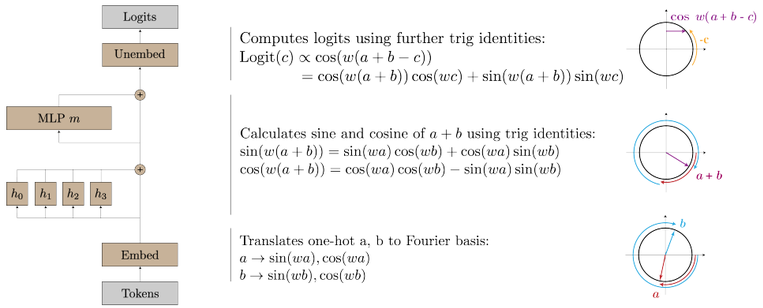


Figure 1: The algorithm implemented by the one-layer transformer for modular addition. Given two numbers a and b, the model projects each point onto a corresponding rotation using its embedding matrix. Using its attention and MLP layers, it then composes the rotations to get a representation of $a+b \mod P$. Finally, it "reads off" the logits for each $c \in \{0, 1, ..., P-1\}$, by rotating by $-c$ to get $\cos(w(a+b-c))$, which is maximized when $a+b \equiv c \mod P$ (since w is a multiple of $\frac{2\pi}{P}$).

We reverse engineer the weights of these transformers and find that they perform this task by mapping the inputs onto a circle and performing addition on the circle. Specifically, we show that the embedding matrix maps the inputs a, b to sines and cosines at a sparse set of *key frequencies* w_k. The attention and MLP layers then combine these using trigonometric identities to compute the sine and cosine of $w_k(a+b)$, and the output matrices shift and combine these frequencies.

We confirm this understanding with four lines of evidence (Section 4): (1) the network weights and activations exhibit a consistent periodic structure; (2) the neuron-logit map W_L is well approximated by a sum of sinusoidal functions of the key frequencies, and projecting the MLP activations onto these sinusoidal functions lets us "read off" trigonometric identities from the neurons; (3) the attention heads and MLP neuron are well approximated by degree-2 polynomials of trigonometric functions of a single frequency; and (4) ablating key frequencies used by the model reduces performance to chance, while ablating the other 95% of frequencies slightly *improves* performance.

Using our understanding of the learned algorithm, we construct two progress measures for the modular addition task—*restricted loss*, where we ablate every non-key frequency, and *excluded loss*, where we instead ablate all key frequencies. Both metrics improve continuously prior to when grokking occurs. We use these metrics to understand the training dynamics underlying grokking and find that training can be split into three phases: **memorization** of the training data; **circuit formation**, where the network learns a mechanism that generalizes; and **cleanup**, where weight decay removes the memorization components. Surprisingly, the sudden transition to perfect test accuracy in grokking occurs during cleanup, *after* the generalizing mechanism is learned. These results show that grokking, rather than being a sudden shift, arises from the gradual amplification of structured mechanisms encoded in the weights, followed by the later removal of memorizing components.

2 Related Work

Phase Changes. Recent papers have observed that neural networks quickly develop novel qualitative behaviors as they are scaled up or trained longer (Ganguli et al., 2022; Wei et al., 2022a). McGrath et al. (2021) find that AlphaZero quickly learns many human chess concepts between 10k and 30k training steps and reinvents human opening theory between 25k and 60k training steps.

Grokking. Grokking was first reported in Power et al. (2022), which trained two-layer transformers on several algorithmic tasks and found that test accuracy often increased sharply long after achieving perfect train accuracy. Millidge (2022) suggests that this may be due to SGD being a random walk on the optimal manifold. Our results echo Barak et al. (2022) in showing that the network instead makes continuous progress toward the generalizing algorithm. Liu et al. (2022) construct small examples of grokking, which they use to compute phase diagrams with four separate "phases" of learning. Thilak et al. (2022) argue that grokking can arise without explicit regularization, from an optimization anomaly they dub the slingshot mechanism, which may act as an implicit regularizer.

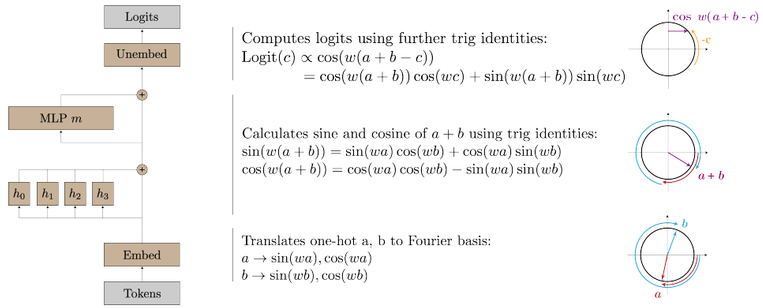


图1：单层Transformer模型用于实现模加法时的算法流程。给定两个数 a 和 b后，该模型会利用其嵌入矩阵将每个点映射到对应的旋转角度上；随后通过其注意力头与MLP层组合这些旋转角度，从而得到$a+b$模 P的结果表示。最后，模型通过将每个 $c \in \{0, 1, ..., P-1\}$旋转 $-c$度来获取对应的逻辑值$\cos(w(a+b-c))$，而当 $a+b \equiv c$模 P时这一值将达到最大值（因为 w 是$\frac{2\pi}{P}$的倍数）。

我们通过对这些Transformer模型的权重进行逆向工程，发现它们是通过将输入值映射到圆周上并在该圆周上进行加法运算来完成这项任务的。具体而言，我们证明了嵌入矩阵会将输入 a、b 映射到一组稀疏的关键频率 w_k处的正弦函数与余弦函数值。随后，注意力机制层和MLP层会利用三角恒等式将这些数值组合起来，进而计算出 $w_k(a+b)$的正弦值与余弦值，而输出矩阵则会对这些频率值进行移位与整合。

我们通过四项证据来验证这一理解（见第4节）：(1) 网络的权重与激活值呈现出一致的周期性结构；(2) 神经元逻辑映射 W_L 可以很好地用关键频率的正弦函数之和来近似，将MLP层的激活值投影到这些正弦函数上后，我们便能从神经元中识别出三角恒等式；(3) 注意力头与MLP神经元则可以很好地用单一频率的三角函数的二次多项式来近似；(4) 若去除模型所依赖的关键频率，其性能会降至随机水平，而仅略微去除其余95%的频率却能够提升性能。

基于对所学习算法的理解，我们为模加任务构建了两项进展度量指标——受限损失，即排除所有非关键频率后的损失值；以及排除损失，即排除所有关键频率后的损失值。在理解能力提升发生之前，这两项指标的值都会持续上升。我们利用这些指标来探究推动理解能力提升的训练机制，发现训练过程可划分为三个阶段：首先是对训练数据进行**记忆阶段**；接着是**电路结构形成阶段**，此时神经网络会学习具备泛化能力的机制；最后是**清理阶段**，通过权重衰减去除那些用于记忆的成分。令人惊讶的是，在清理阶段，也就是在网络学会具有泛化能力的机制之后，测试准确率会突然达到完美水平。这些结果表明，理解能力提升并非突如其来的变化，而是源于权重中编码的结构性机制逐渐被强化，随后再通过移除记忆相关成分而实现的。

2 相关工作

阶段变化。近期的一些研究指出，当神经网络的规模扩大或训练时间延长时，它们会迅速展现出新的定性行为（Ganguli等人，2022年；Wei等人，2022a）。McGrath等人（2021年）发现，AlphaZero在经过10千到3万步的训练后就能快速掌握许多人类国际象棋的策略概念，而在25千到6万步的训练后则重新创造了人类国际象棋的开局理论。

理解能力提升。关于理解能力提升的首次研究出自Power等人（2022年），该研究让双层变换器在多种算法任务上进行训练，结果发现即便在训练准确率早已达到完美水平后，测试准确率仍常常会出现大幅上升。Millidge（2022年）认为，这种现象可能是由于随机梯度下降实际上是在最优流形上进行的随机游走所致。我们的研究结果与Barak等人（2022年）的发现不谋而合，他们都表明网络实际上是在朝着泛化算法持续进步。Liu等人（2022年）构建了理解能力提升的小规模示例，并利用这些示例绘制出了包含四个不同学习“阶段”的相图。Thilak等人（2022年）则提出，即便没有显式的正则化处理，理解能力提升也可能通过他们称之为“弹弓机制”的优化异常现象出现，而这种机制本身或许就起到了隐式正则化的作用。

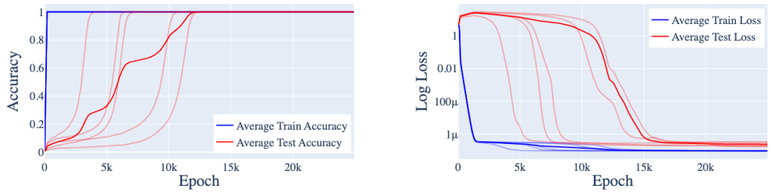


Figure 2: The train and test accuracy (left) and train and test loss (right) of one-layer transformers on the modular addition task described in Section 3, over 5 random seeds. These models consistently exhibit grokking: they quickly overfit early on in training, but then later learn to generalize.

Circuits-style mechanistic interpretability. The style of post-hoc mechanistic interpretability in Section 4 is heavily inspired by the Circuits approach of Cammarata et al. (2020), Elhage et al. (2021), and Olsson et al. (2022).

Progress measures. Barak et al. (2022) introduce the notion of *progress measures*—metrics that improve smoothly and that precede emergent behavior. They prove theoretically that training would amplify a certain mechanism and heuristically define a progress measure. In contrast, we use mechanistic intepretability to discover progress measures empirically.

3 Setup and Background

We train transformers to perform addition mod P. The input to the model is of the form "$a\ b\ =$", where a and b are encoded as P-dimensional one-hot vectors, and $=$ is a special token above which we read the output c. In our mainline experiment, we take $P = 113$ and use a one-layer ReLU transformer, token embeddings with $d = 128$, learned positional embeddings, 4 attention heads of dimension $d/4 = 32$, and $n = 512$ hidden units in the MLP. In other experiments, we vary the depth and dimension of the model. We did not use LayerNorm or tie our embed/unembed matrices.

Our mainline dataset consists of 30% of the entire set of possible inputs (that is, 30% of the $113 \cdot 113$ pairs of numbers mod P). We use full batch gradient descent using the AdamW optimizer (Loshchilov & Hutter, 2017) with learning rate $\gamma = 0.001$ and weight decay parameter $\lambda = 1$. We perform $40,000$ epochs of training. As there are only $113 \cdot 113$ possible pairs, we evaluate test loss and accuracy on all pairs of inputs not used for training.

Networks trained on this task consistently exhibit grokking. As Figure 2 shows, our networks first overfit the training set: train accuracy quickly converges to 100% and the train loss quickly declines, while the test accuracy remains low and the test loss remains high. After around $10,000$ epochs, the network generalizes and test accuracy increases to near 100%. In robustness experiments, we confirm that grokking consistently occurs for other architectures and prime moduli (Appendix C.2). In Section 5.3 we find that grokking does not occur without regularization.

To describe transformer components, we follow the conventions and notations laid out in Elhage et al. (2021). We focus on the $d \times p$ embedding matrix W_E, the $d \times n$ output matrix of the MLP layer W_{out}, and the $P \times d$ unembedding matrix W_U.[2] Let $\text{Logits}(a, b)$ denote the logit vector on inputs a, b, and $MLP(a, b)$ denote the MLP activations. Empirically, our networks do not significantly use the skip connection around the MLP (Appendix A.1), so $\text{Logits}(a, b) \approx W_U W_{out} \text{MLP}(a, b)$. We therefore also study the $P \times n$ neuron-logit map $W_L = W_U W_{out}$.

3.1 The Fourier multiplication algorithm

We claim that the learned networks use the following algorithm (Figure 1):

- Given two one-hot encoded tokens a, b map these to $\sin(w_k a)$, $\cos(w_k a)$, $\sin(w_k b)$, and $\cos(w_k b)$ using the embedding matrix, for various frequencies $w_k = \frac{2k\pi}{P}, k \in \mathbb{N}$.

[2] We ignore the embedding and unembedding of the '=' token for simplicity.

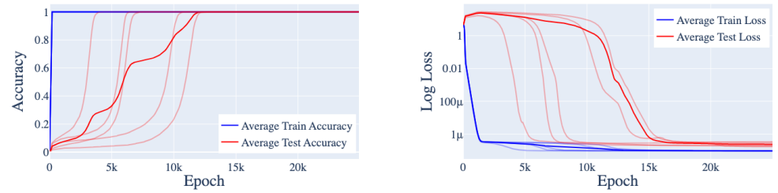


图2：在5种不同随机种子下，单层Transformer在模块加法任务上的训练与测试准确率（左侧）以及训练与测试损失值（右侧）。这些模型始终表现出理解能力提升的现象：它们在训练初期就会迅速出现过拟合情况，但随后又能学会进行泛化。3，2022年）提出了进展度量指标——即那些能够持续平滑提升，并且会在涌现行为出现之前出现的度量标准。他们从理论上证明了训练过程会强化某种特定机制，并据此启发式地定义了进展度量指标。与之不同，我们则借助机械可解释性方法来实证地发现这类进展度量指标。

基于电路结构的机制可解释性。第4 节中介绍的事后机制可解释性方法，其设计思路在很大程度上借鉴了Cammarata等人（2020年）、Elhage等人（2021年）以及Olsson等人（2022年）所提出的电路结构方法。

进展度量指标。Barak等人（2022年）首次提出了进展度量指标这一概念——指那些能够实现平稳提升，且在涌现行为出现之前便已显现的度量标准。他们从理论上论证了训练过程会强化某种特定机制，进而启发式地定义了进展度量指标。而我们则是通过机械可解释性方法，以实证的方式寻找这类进展度量指标。

3 设置与背景

我们训练Transformer模型以执行加法运算。该模型的输入形式为"$a\ b\ =$"，其中 a 和 b 会被编码为 P维的独热向量，而 $=$ 则是一个特殊的标记，我们在其上方读取输出 c。在我们的主要实验中，我们采用 $P = 113$作为输入，使用单层ReLU结构的Transformer模型，搭配 $d = 128$定义的标记嵌入、学习得到的位置嵌入、 $d/4 = 32$维的4个注意力头，以及MLP结构中的 $n = 512$ 个隐藏单元。在其他实验中，我们会改变模型的深度和维度。此外，我们没有使用LayerNorm机制，也没有将嵌入/解嵌入矩阵绑定在一起。

我们的主线数据集包含了所有可能输入值中30%的样本（即 $113 \cdot 113$ 组模运算后的数字对 P）。我们采用全批量梯度下降法，并搭配AdamW优化器（Loshchilov与Hutter，2017年），同时设置学习率 $\gamma = 0.001$ 以及权重衰减参数 $\lambda = 1$。整个训练过程共进行40,000个训练轮数。由于可选的数字对数量有限 $113 \cdot 113$，我们会针对所有未用于训练的输入对来评估测试损失值和测试准确率。

在该任务上训练得到的神经网络始终能够展现出理解能力提升的效果。如图2所示，这类网络最初会出现过拟合现象：训练准确率会迅速升至100%，训练损失值也会快速下降，而测试准确率则维持在较低水平，测试损失值居高不下。大约经过10,000个训练轮数后，网络开始具备泛化能力，测试准确率便会上升到接近100%。在鲁棒性实验中，我们证实其他架构以及不同的素数模数条件下同样会出现理解能力提升的现象（详见附录C.2）。在5.3节中，我们进一步发现若不采用正则化处理，则不会出现理解能力提升的效果。

为描述Transformer模型的各组成要素，我们采用了Elhage等人（2021年）所制定的规范与符号表示方法。重点研究的对象包括 $d \times p$ 嵌入矩阵 W_E、MLP层的 $d \times n$ 输出矩阵W_{out}，以及 $P \times d$ 解嵌入矩阵 W_U。[2]此处用$\text{Logits}(a, b)$表示输入a, b对应的逻辑值向量，而 $MLP(a, b)$则表示多层感知机激活值。从实际运行情况来看，我们的网络并未大量使用MLP周围的跳接连接结构（详见附录A.1），因此$\text{Logits}(a, b) \approx W_U W_{out} \text{MLP}(a, b)$。基于此，我们还对 $P \times n$ 神经元逻辑映射 $W_L = W_U W_{out}$进行了研究。

3.1 傅里叶乘法算法

我们认为这些经过训练的网络会采用以下算法（见图 1）：

- 给定两个经过独热编码的标记 a， b 可通过嵌入矩阵将它们分别映射为$\sin(w_k a)$、$\cos(w_k a)$、$\sin(w_k b)$和$\cos(w_k b)$，且该映射适用于不同的频率$w_k = \frac{2k\pi}{P}$， $k \in \mathbb{N}$。 ——

[2] 为简化分析，我们在此忽略 '=' 标记的嵌入与解嵌入过程。

- Compute $\cos(w_k(a+b))$ and $\sin(w_k(a+b))$ using the trigonometric identities:
$$\cos(w_k(a+b)) = \cos(w_k a)\cos(w_k a) - \sin(w_k a)\sin(w_k b)$$
$$\sin(w_k(a+b)) = \sin(w_k a)\cos(w_k b) + \cos(w_k a)\sin(w_k b)$$
In our networks, this is computed in the attention and MLP layers.
- For each output logit c, compute $\cos(w_k(a+b-c))$ using the trigonometric identity:
$$\cos(w_k(a+b-c)) = \cos(w_k(a+b))\cos(w_k c) + \sin(w_k(a+b))\sin(w_k c). \quad (1)$$
This is a linear function of the already-computed values $\cos(w_k(a+b))$, $\sin(w_k(a+b))$ and is implemented in the product of the output and unembedding matrices W_L.
- The unembedding matrix also adds together $\cos(w_k(a+b-c))$ for the various ks. This causes the cosine waves to constructively interfere at $c^* = a+b \mod p$ (giving c^* a large logit), and destructively interfere everywhere else (thus giving small logits to other cs).

We refer to this algorithm as Fourier multiplication, and will justify our claim in detail in Section 4.

4 REVERSE ENGINEERING A ONE-LAYER TRANSFORMER

In this section, we describe four lines of evidence that our transformers are using the Fourier multiplication algorithm described in Section 3.1. Here we apply our analysis to the mainline model from Section 3; the results are broadly consistent for other models, including across different number of layers, different fractions of the training data, and different prime moduli (see Appendix C.2, especially Table 5).

Our first line of evidence involves examining the network weights and activations and observing consistent **periodic structure** that is unlikely to occur by chance (Section 4.1). Moreover, when we take Fourier transforms, many components are either sparse or nearly sparse in the Fourier domain, supported on a handful of *key frequencies*.

We next look into the actual **mechanisms** implemented in the model weights (Section 4.2). We show that the unembedding matrix W_L is (approximately) rank 10, where each direction corresponds to the cosine or sine of one of 5 key frequencies. Projecting the MLP activations onto the components of W_L approximately produces multiples of the functions $\cos(w_k(a+b))$ and $\sin(w_k(a+b))$, showing that the MLP layer does compute these sums.

To better understand the mechanism, we **zoom in** to individual neurons (Section 4.3). We find that the attention heads and most neurons are well-approximated by degree-2 polynomials of sines and cosines at a *single* frequency. Moreover, the corresponding direction in W_L also contains only that frequency. This suggests that the model's computations are (1) localized across frequencies and (2) mostly aligned with the neuron basis.

Finally, we use **ablations** to confirm that our interpretation is faithful (Section 4.4). We replace various components of the model by the components of the Fourier multiplication algorithm and find that doing so consistently does not harm and sometimes even improves model performance.

4.1 SUGGESTIVE EVIDENCE: SURPRISING PERIODICITY

The first line of evidence that the network is using the algorithm described in Section 3.1 is the surprising periodicity in the activations of the transformer. That is, the output of every part of the network is periodic as a function of the input tokens.

Periodicity in the embeddings. We start by examining the embeddings. We apply a Fourier transform along the input dimension of the embedding matrix W_E then compute the ℓ_2-norm along the other dimension; results are shown in Figure 3. We plot only the components for the first 56 frequencies, as the norm of the components for frequencies k and $P-k$ are symmetric. The embedding matrix W_E is sparse in the Fourier basis–it only has significant nonnegligible norm at 6 frequencies. Of these frequencies, only 5 appear to be used significantly in later parts of the model (corresponding to $k \in \{14, 35, 41, 42, 52\}$). We dub these the *key frequencies* of the model.

Periodicity in attention heads and MLP neuron activations. This periodic structure recurs throughout the network. As an example, we plot the attention weight at position 0 for every combination of two inputs for head 0 in Figure 4. The attention exhibits a periodic structure with frequency

- 利用三角恒等式计算$\cos(w_k(a+b))$和$\sin(w_k(a+b))$，具体表达式为：$\cos(w_k(a+b)) = \cos(w_k a)\cos(w_k a) - \sin(w_k a)\sin(w_k b)$ $\sin(w_k(a+b)) = \sin(w_k a)\cos(w_k b) + \cos(w_k a)\sin(w_k b)$。在我们的模型中，这些计算会在注意力层与MLP层中完成。

- 对于每一个输出logit c，均可利用三角恒等式计算$\cos(w_k(a+b-c))$的值：该结果实际上是已计算出的$\cos(w_k(a+b))$和$\sin(w_k(a+b))$这些值的线性函数，其实现方式即为将输出logit与解嵌入矩阵相乘 W_L。

- 解嵌入矩阵还会把各个 k对应的$\cos(w_k(a+b-c))$值相加。这样一来，余弦波会在 $c^* = a+b \mod p$ 的条件下发生建设性干涉（从而使得对应的logit值很大），而在其他所有位置则会发生破坏性干涉（进而使其他 c的logit值保持较小）。

我们将这种算法称为傅里叶乘法，并将在第4节中对这一说法进行详细论证。

4 逆向工程单层Transformer

在本节中，我们将列举四项证据，用以证明我们的Transformer模型确实采用了第3.1节中介绍的傅里叶乘法算法。此处，我们的分析是针对第3节中的主线模型进行的；对于其他模型而言，分析结果也基本一致，这些模型包括具有不同层数、使用不同比例训练数据以及采用不同素数模数的模型（详见附录C.2，尤其是表5）。

我们的第一组证据来自对网络权重及激活值的分析，我们发现其中存在一致的**周期性结构**，而这种结构极不可能纯属偶然（见第4.1节）。此外，当我们对数据应用傅里叶变换后，会在傅里叶域中发现许多分量要么极为稀疏，要么几乎稀疏，且仅集中在少数关键频率上。

接下来，我们研究模型权重中实际**实现的机制**（见第4.2节）。研究表明，解嵌入矩阵 W_L 的秩大约为10，其每个方向分别对应5个关键频率中的某个频率的余弦函数或正弦函数。将多层感知机的激活值投影到 W_L 这些分量上后，大致可以得到$\cos(w_k(a+b))$ 和$\sin(w_k(a+b))$这些函数的倍数，这表明多层感知机层确实会计算这些数值的求和。

为更深入地理解这一机制，我们**进一步放大视角**，观察单个神经元的情况（见第4.3节）。我们发现，注意力头以及大多数神经元都可以通过二次阶的正弦函数与余弦函数多项式来很好地近似，且这些多项式仅对应单一频率。同时，在 W_L 该频率对应的方向上也仅包含这一频率。这些现象表明，模型的计算过程首先在频率分布上是局部化的，其次在很大程度上与神经元的特性相匹配。

最后，我们通过**消融实验**来验证我们的解释是准确的（见第4.4节）。我们将模型中的各种组件替换为傅里叶乘法算法中的对应组件，结果发现这样做不仅不会损害模型性能，有时甚至还能提升其性能。

4.1 提示性证据：惊人的周期性

能够证明该网络正在使用第3.1节中描述的算法的第一条证据，就是Transformer模型中激活值的惊人周期性。也就是说，网络各部分的输出会随着输入标记的变化而呈现周期性特征。

嵌入向量中的周期性。我们首先对嵌入向量进行分析。首先沿着嵌入矩阵的输入维度应用傅里叶变换 W_E，然后再沿着另一个维度计算 ℓ_2范数；相关结果展示在图3中。由于频率 k 和 $P-k$ 对应的向量分量具有对称性，因此我们仅展示了前56个频率的分量。该嵌入矩阵在傅里叶基下是稀疏的——仅有6个频率的分量具有非可忽略的显著范数。在这些频率中，只有5个在模型的后续部分被明显使用（对应于 $k \in \{14, 35, 41, 42, 52\}$）。我们将这些频率称为模型的关键频率。

注意力头与MLP神经元激活值中存在的周期性现象。这种周期性结构在网络各处都会出现。例如，我们在图4中展示了head0在每两种输入组合下位置0处的注意力权重。可以看出，该注意力机制呈现出具有特定频率的非周期性结构

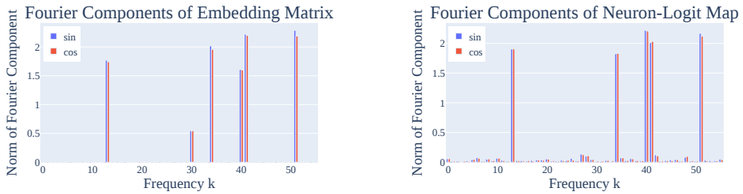


Figure 3: *(Left)* The norms of the Fourier components in the embedding matrix W_E. As discussed in Section 4.1, the sparsity of W_E in the Fourier basis is evidence that the network is operating in this basis. Of the six non-zero frequencies, five "key frequencies" appear in later parts of the network, corresponding to $k \in \{14, 35, 41, 42, 52\}$. *(Right)* Norm of Fourier components of the neuron-logit map W_L. A Fourier transform is taken over the logit axis, and then the norm is taken over the neuron axis. As discussed in Section 4.2, W_L is well-approximated by the 5 key frequencies w_k.

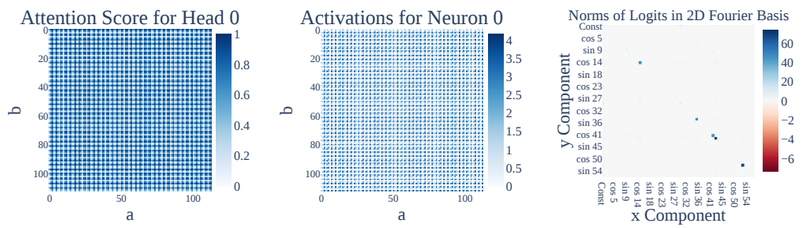


Figure 4: *(Left)* The attention score for head 0 from the token '=' to 'a', as a function of inputs a, b. *(Center)* The activations of MLP neuron 0 given inputs a, b. Both the attention scores and the neuron activations are periodic (Section 4.1). *(Right)* The norm of the Fourier components of the logits (2D Fourier transform is taken over the inputs a, b, and then norm is taken over the logit axis). There are 20 significant components corresponding to the 5 key frequencies (Section 4.1).

$k = 35$. In Figure 4, we also plot the activations of MLP neuron 0 for every combination of inputs. The activations are periodic with frequency $k = 42$. We see similar patterns for other attention heads and MLP neurons (Appendix C.1).

Periodicity in logits. Finally, the logits are also periodic. In Figure 4, we represent the logits in the 2D Fourier basis over the inputs, then take the ℓ_2-norm over the output dimension. There are only twenty components with significant norm, corresponding to the products of sines and cosines for the five key frequencies w_k. These show up as five 2×2 blocks in Figure 4.

4.2 Mechanistic Evidence: Composing Model Weights

We now demonstrate that the model implements the trigonometric identity (1) as follows: the functions $\cos(w_k(a+b))$, $\sin(w_k(a+b))$ are linearly represented in the MLP activations, and the unembed matrix reads these linear directions and multiplies them by $\cos(w_k c)$, $\sin(w_k c)$ respectively.

We will do this in two steps. First, we show that W_L (the matrix mapping MLP activations to logits) is (approximately) rank 10 and can be well approximated as:

$$W_L = \sum\nolimits_{k\in\{14,35,41,42,52\}} \cos(w_k)\, u_k^T + \sin(w_k)\, v_k^T \tag{2}$$

for some $u_k, v_k \in \mathbb{R}^{512}$, where $\cos(w_k), \sin(w_k) \in \mathbb{R}^{113}$ are vectors whose cth entry is $\cos(w_k c)$ and $\sin(w_k c)$. Second, note that our model implements the logits for a, b as:

$$\text{Logits}(a,b) = W_L \text{MLP}(a,b) \approx \sum\nolimits_k \cos(w_k)\, u_k^T \text{MLP}(a,b) + \sin(w_k)\, v_k^T \text{MLP}(a,b) \tag{3}$$

We check empirically that the terms $u_k^T \text{MLP}(a,b)$ and $v_k^T \text{MLP}(a,b)$ are approximate multiples of $\cos(w_k(a+b))$ and $\sin(w_k(a+b))$ ($> 90\%$ of variance explained). Thus the network computes trigonometric functions in the MLP and reads them off as claimed. As a sanity check, we confirm

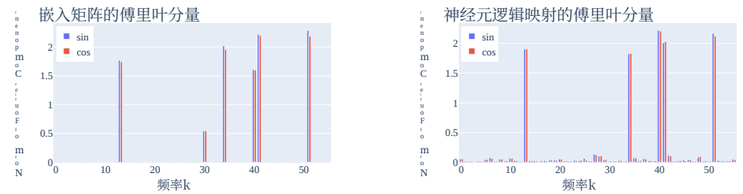


图3：*(左图)*嵌入矩阵中傅里叶分量的范数 W_E。如第4.1节所述， W_E 傅里叶基中的稀疏性是网络正在该基上运行的证据。在六个非零频率中，有五个"关键频率"出现在网络的后续部分，它们对应于 $k \in \{14, 35, 41, 42, 52\}$。 *(右图)*神经元逻辑映射的傅里叶分量范数 W_L。首先对逻辑值轴进行二维傅里叶变换，然后再对神经元轴计算范数。如第4.2节所讨论的， W_L 该数值可以很好地由这5个关键频率近似表示 w_k。

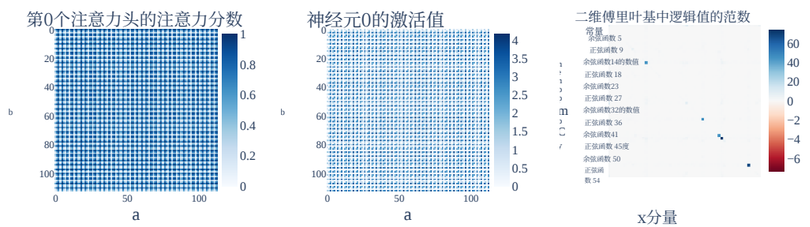


图4：*(左图)*从标记'='到'a'时，第0个注意力头对应的注意力分数随输入值变化的函数关系a， b。*(中图)*给定特定输入时，MLP 神经元 0 的激活值 a， b。无论是注意力分数还是神经元激活值都呈现出周期性（见第4.1节）。 *(右图)*逻辑值的傅里叶分量范数（首先对输入值进行二维傅里叶变换 a, b，之后再对逻辑值轴计算范数）。共有20个与5个关键频率相对应的显著分量（参见第4.1节）。

$k = 35$。在图4中，我们还绘制了每种输入组合下神经元0的激活值。这些激活值也呈现出了频率为 $k = 42$的周期性特征。其他注意力头及MLP神经元也展现出类似的规律（详见附录C.1）。

逻辑值的周期性。最后，这些逻辑值同样也具有周期性。在图4中，我们用输入值的2D傅里叶基来表示逻辑值，随后对输出维度计算 ℓ_2范数。其中仅有二十个分量的范数较大，它们对应于五个关键频率的正弦函数与余弦函数的乘积 w_k。这五个分量在图4中表现为五个 2×2 块。

4.2 机械论证据：模型权重组合方法

现在我们将证明该模型是通过以下方式实现三角恒等式（1）的：函数cos（w_k（$a+b$）、sin（w_k（$a+b$）在多层感知机的激活值中是线性表示的，而解嵌入矩阵会读取这些线性方向，并分别将它们乘以cos（$w_k c$）和sin（$w_k c$），以此得到相应的结果。

我们将分两步来完成这一任务。首先，我们会证明 W_L（即将多层感知机的激活值映射为逻辑值的矩阵）的秩大约为10，并且可以很好地用如下形式进行近似表示：

$$W_L = \sum\nolimits_{k\in\{14,35,41,42,52\}} \cos(w_k)\, u_k^T + \sin(w_k)\, v_k^T \tag{2}$$

对于某些 u_k， $v_k \in \mathbb{R}^{512}$，其中余弦函数(w_k)，正弦函数(w_k) $\in \mathbb{R}^{113}$均为向量，其第 c个分量分别为余弦函数($w_k c$)与正弦函数($w_k c$)。其次需要说明的是，我们的模型会将 a， b的逻辑值实现为如下形式：

$$\text{Logits}(a,b) = W_L \text{MLP}(a,b) \approx \sum\nolimits_k \cos(w_k)\, u_k^T \text{MLP}(a,b) + \sin(w_k)\, v_k^T \text{MLP}(a,b) \tag{3}$$

我们通过实证检验发现， $u_k^T \text{MLP}(a,\ b)$与 $v_k^T \text{MLP}(a,\ b)$这两个表达式实际上近似等于余弦函数cos(w_k($a+b$))与正弦函数sin(w_k($a+b$))的倍数（可解释方差占比达$> 90\%$）。由此可见，该网络确实在多层感知机内部计算出三角函数值，并如预期那样将其读取出来。作为进一步验证，我们还确认了

W_L Component	Fourier components of u_k^TMLP(a,b) or v_k^TMLP(a,b)	FVE
$\cos(w_{14}c)$	$44.6\cos(w_{14}a)\cos(w_{14}b) - 43.6\sin(w_{14}a)\sin(w_{14}b) \approx 44.1\cos(w_{14}(a+b))$	93.2%
$\sin(w_{14}c)$	$44.1\sin(w_{14}a)\cos(w_{14}b) + 44.1\cos(w_{14}a)\sin(w_{14}b) \approx 44.1\sin(w_{14}(a+b))$	93.5%
$\cos(w_{35}c)$	$40.7\cos(w_{35}a)\cos(w_{35}b) - 43.6\sin(w_{35}a)\sin(w_{35}b) \approx 42.2\cos(w_{35}(a+b))$	96.8%
$\sin(w_{35}c)$	$41.8\sin(w_{35}a)\cos(w_{35}b) + 41.8\cos(w_{35}a)\sin(w_{35}b) \approx 41.8\sin(w_{35}(a+b))$	96.5%
$\cos(w_{41}c)$	$44.8\cos(w_{41}a)\cos(w_{41}b) - 44.8\sin(w_{41}a)\sin(w_{41}b) \approx 44.8\cos(w_{41}(a+b))$	97.0%
$\sin(w_{41}c)$	$44.5\sin(w_{41}a)\cos(w_{41}b) + 44.5\cos(w_{41}a)\sin(w_{41}b) \approx 44.5\sin(w_{41}(a+b))$	97.0%
$\cos(w_{42}c)$	$64.6\cos(w_{42}a)\cos(w_{42}b) - 68.5\sin(w_{42}a)\sin(w_{42}b) \approx 66.6\cos(w_{42}(a+b))$	96.4%
$\sin(w_{42}c)$	$67.8\sin(w_{42}a)\cos(w_{42}b) + 67.8\cos(w_{42}a)\sin(w_{42}b) \approx 67.8\sin(w_{42}(a+b))$	96.4%
$\cos(w_{52}c)$	$60.5\cos(w_{52}a)\cos(w_{52}b) - 65.5\sin(w_{52}a)\sin(w_{52}b) \approx 63.0\cos(w_{52}(a+b))$	97.4%
$\sin(w_{52}c)$	$64.5\sin(w_{52}a)\cos(w_{52}b) + 64.5\cos(w_{52}a)\sin(w_{52}b) \approx 64.5\sin(w_{52}(a+b))$	98.2%

Table 1: For each of the directions u_k or v_k (corresponding to the $\cos(w_k)$ and $\sin(w_k)$ components respectively) in the unembedding matrix, we take the dot product of the MLP activations with that direction, then perform a Fourier transform (middle column; only two largest coefficients shown). We then compute the fraction of variance explained (FVE) if we replace the projection with a single term proportional to $\cos(w_k(a+b))$ or $\sin(w_k(a+b))$, and find that it is consistently close to 1.

that the logits are indeed well-approximated by terms of the form $\cos(w_k(a+b-c))$ (95% of variance explained).

W_L is well approximated by $\cos(w_kc)$ and $\sin(w_kc)$. We perform a discrete Fourier transform (DFT) on the logit axis of W_L and look at the 10 directions u_k, v_k corresponding to $\sin(w_k)$ and $\cos(w_k)$. When we approximate W_L with $\sum_{k\in\{14,35,41,42,52\}} \cos(w_k)\, u_k^T + \sin(w_k)\, v_k^T$, the residual has Frobenius norm that is under 0.55% of the norm of W_L. This shows that W_L is well approximated by the 10 directions corresponding to $\cos(w_k)$ and $\sin(w_k)$ for each of the five key frequencies. We also plot the norms of each direction in Figure 3, and find that no Fourier component outside the 5 key frequencies has significant norm.

The unembedding matrix "reads off" terms of the form $\cos(w_k(a+b))$ and $\sin(w_k(a+b))$ from the MLP neurons. Next, we take the dot product of the MLP activations with each of the directions u_k, v_k for $k \in \{14, 35, 41, 42, 52\}$. Table 1 displays the results: the dot products u_k^TMLP(a,b) and v_k^TMLP(a,b) are well approximated by a multiple of terms of the form

$$\cos(w_k(a+b)) = \cos(w_ka)\cos(w_kb) - \sin(w_ka)\sin(w_kb)\,, \text{ and}$$
$$\sin(w_k(a+b)) = \sin(w_ka)\cos(w_kb) + \cos(w_ka)\sin(w_kb)\,.$$

That is, for each key frequency k, u_k and v_k are linear directions in the space of MLP neuron activations that represent $\cos(w_k(a+b))$ and $\sin(w_k(a+b))$.

Logits are well approximated by a weighted sum of $\cos(w_k(a+b-c))$. We approximate the output logits as the sum $\sum_k \alpha_k \cos(w_k(a+b-c))$ for $k \in \{14, 35, 41, 42, 52\}$ and fit the coefficients α_k via ordinary least squares. This approximation explains 95% of the variance in the original logits. This is surprising—the output logits are a $113 \cdot 113 \cdot 113$ dimensional vector, but are well-approximated with just the 5 directions predicted by our interpretation. If we evaluate test loss using this logit approximation, we actually see an *improvement* in loss, from $2.4 \cdot 10^{-7}$ to $4.7 \cdot 10^{-8}$.

Taken together, these results confirm that the model computes sums of terms of the form $\cos(w_k(a+b-c)) = \cos(w_k(a+b))\cos(w_kc) + \sin(w_k(a+b))\sin(w_kc)$.

4.3 Zooming In: Approximating Neurons with Sines and Cosines

In the previous section, we showed how the model computes its final logits by using W_L to "read off" trigonometric identities represented in the MLP neurons. We now examine the attention heads and MLP neurons to understand how the identities come to be represented at the MLP layer. In Appendix C.1.2, we show that two of the attention heads approximately compute degree-2 polynomials of sines and cosines of a particular frequency (and the other two are used to increase the magnitude of the input embeddings in the residual stream). Here, we show that most neurons are also well-approximated by degree-2 polynomials, and the map from neurons to logits is localized by frequency.

Most MLP neurons approximately compute a degree-2 polynomial of a single frequency. We next try to approximate the activations of each MLP neuron by a degree-2 polynomial of one of the 5 key frequencies. As shown in Figure 5, out of 512 total neurons, 433 (84.6%) have over 95% of their variance explained with a single frequency.

W_L 组件	u_k^T多层感知机$(a,\ b)$或 v_k^T多层感知机$(a,\ b)$的傅里叶分量。	FVE
余弦函数$(w_{14}c)$	44。6 余弦函数$(w_{14}a)$ 余弦函数$(w_{14}b)$ − 43.6正弦函数$(w_{14}a)$ 正弦函数$(w_{14}b)$ ≈44.1余弦函数$(w_{14}(a+b))$	93.2%
$\sin(w_{14}c)$	44。1 × 正弦函数$(w_{14}a)$ × 余弦函数$(w_{14}b)$ + + 44.1×余弦函数$(w_{14}a)$×正弦函数$(w_{14}b)$ + ≈ 44.1×正弦函数$(w_{14}(a+b))$	93.5%
$\cos(w_{35}c)$	40。7 × 余弦函数$(w_{35}a)$ × 余弦函数$(w_{35}b)$ + − 43.6×正弦函数$(w_{35}a)$×正弦函数$(w_{35}b)$ + ≈ 42.2×余弦函数$(w_{35}(a+b))$	96.8%
$\sin(w_{35}c)$	41<span style='id':'1'>.8 $\sin(w_{35}a)$ 余弦函数$(w_{35}b)$ + 41.8余弦函数$(w_{35}a)$ $\sin(w_{35}b)$ ≈ 41.8$\sin(w_{35}(a+b))$	96.5%
$\cos(w_{41}c)$	44<span style='id':'1'>.8 余弦函数$(w_{41}a)$ 余弦函数$(w_{41}b)$ − 44.8正弦函数$(w_{41}a)$ $\sin(w_{41}b)$ ≈ 44.8余弦函数$(w_{41}(a+b))$	97.0%
$\sin(w_{41}c)$	44.5 正弦函数$(w_{41}a)$ 余弦函数$(w_{41}b)$ + 44.5余弦函数$(w_{41}a)$ 正弦函数$(w_{41}b)$ ≈ 44.5正弦函数$(w_{41}(a+b))$	97.0%
$\cos(w_{42}c)$	64.6 余弦函数$(w_{42}a)$ 余弦函数$(w_{42}b)$ − 68.5正弦函数$(w_{42}a)$ 正弦函数$(w_{42}b)$ ≈ 66.6余弦函数$(w_{42}(a+b))$	96.4%
$\sin(w_{42}c)$	67<span style='id':'1'>.8 $\sin(w_{42}a)$ 余弦函数$(w_{42}b)$ + 67.8余弦函数$(w_{42}a)$ $\sin(w_{42}b)$ ≈ 67.8$\sin(w_{42}(a+b))$	96.4%
$\cos(w_{52}c)$	60<span style='id':'1'>.5 余弦函数$(w_{52}a)$ 余弦函数$(w_{52}b)$ − 65.5正弦函数$(w_{52}a)$ $\sin(w_{52}b)$ ≈ 63.0余弦函数$(w_{52}(a+b))$	97.4%
$\sin(w_{52}c)$	64.5 正弦函数$(w_{52}a)$ 余弦函数$(w_{52}b)$ + 64.5余弦函数$(w_{52}a)$ 正弦函数$(w_{52}b)$ ≈ 64.5正弦函数$(w_{52}(a+b))$	98.2%

图1：对于解嵌入矩阵中的每一个方向 u_k 或 v_k （分别对应余弦函数w_k和正弦函数w_k的分量），我们首先计算多层感知机激活值与该方向的点积，随后对其进行傅里叶变换（见中间列，仅展示了两个最大的系数）。接着，如果我们用与$\cos(w_k(a+b))$或$\sin(w_k(a+b))$成比例的单一项来替代该投影，就会计算出方差解释率（FVE），结果发现这一数值始终接近1。

逻辑值确实能够被形如$\cos(w_k(a+b-c))$的表达式很好地近似表示（对应的方差解释率为95%）。

W_L 可以通过余弦函数w_kc和正弦函数w_kc进行很好地近似。我们对 W_L 的逻辑值轴执行离散傅里叶变换（DFT），并考察与正弦函数w_k和余弦函数w_k对应的10个方向 u_k、 v_k。当用$\sum_{k\in\{14,35,41,42,52\}} \cos(w_k)\, u_k^T + \sin(w_k)\, v_k^T$来近似 W_L 时，其残差的弗罗贝尼乌斯范数小于 W_L范数的0.55%。这表明对于五个关键频率而言， W_L 都可以通过对应于余弦函数w_k和正弦函数w_k的这10个方向进行很好的近似。我们还在图3中绘制了各个方向的范数，发现除了这5个关键频率之外的任何傅里叶分量都没有显著的范数值。

解嵌入矩阵能够“读取”出自多层感知机神经元中的形如$\cos(w_k(a+b))$以及$\sin(w_k(a+b))$的项。接下来，我们会计算多层感知机激活值与各个方向 u_k、 v_k 的点积，这些方向对应于 $k \in \{14, 35, 41, 42, 52\}$。表1展示了计算结果：$u_k^TMLP(a,\ b)$ 以及 v_k^TMLP$(a,\ b)$ 的点积可以很好地用这类项的倍数来近似表示。

$$\cos(w_k(a+b)) = \cos(w_ka)\cos(w_kb) - \sin(w_ka)\sin(w_kb)\,, \text{ and}$$
$$\sin(w_k(a+b)) = \sin(w_ka)\cos(w_kb) + \cos(w_ka)\sin(w_kb)\,.$$

也就是说，对于每个关键频率而言， k、 u_k 和 v_k 都是多层感知机神经元激活值空间中的线性方向，它们分别代表$\cos(w_k(a+b))$和$\sin(w_k(a+b))$。

逻辑值可以通过$\cos(w_k(a+b-c))$s的加权求和来很好近似。我们把输出逻辑值近似为对应的cos(())之和，并通过普通最小二乘法拟合系数。这种近似方法能够解释原始逻辑值中方差的95%。这一结果相当令人惊讶——尽管输出逻辑值是一个维向量，但仅用我们的解释方法预测出的5个方向就足以对其实现很好的近似。如果我们使用这种逻辑值近似来评估测试损失，实际上会发现损失从降到了，即出现了改善。

综合来看，这些结果证实该模型会计算如下形式各项的求和：$\cos(w_k(a+b-c))$ $=\cos(w_k(a+b))$ $\cos(w_kc)$ $+\sin(w_k(a+b))$ $\sin(w_kc)$。

4.3 深入探讨：利用正弦函数与余弦函数近似神经元

在上一节中，我们展示了模型是如何通过 W_L 读取多层感知机神经元中所表示的三角恒等式，从而计算出最终的逻辑值。现在我们将研究注意力头与多层感知机神经元，以了解这些恒等式究竟是如何在多层感知机层中被表示出来的。在附录C.1.2中，我们证明了其中的两个注意力头能够近似表示特定频率的正弦函数与余弦函数的度数为-2 的多项式（另外两个注意力头则用于提升残差流中输入嵌入向量的幅度）。此处我们进一步发现，大多数神经元也能被度数为-2 的多项式很好地近似，而且神经元到逻辑值的映射关系会受到频率的限制。

大多数多层感知机神经元大致会计算某个单一频率的二次多项式值。接下来，我们尝试用5个关键频率中的某一频率的二次多项式来近似表示每个多层感知机神经元的激活值。如图5所示，在总共512个神经元中，有433个（84%，即6%）的方差有超过85% 的部分能够通过单一频率来解释。

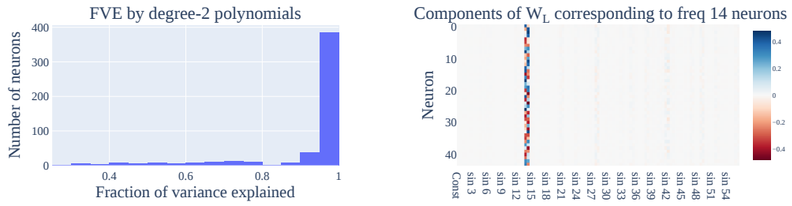


Figure 5: (Left) Most neurons are well-approximated by degree-2 polynomials of a single frequency. (Right) A heatmap showing weights in W_L corresponding to each of the 44 neurons of frequency 14. The non-trivial components correspond to $\sin(w_k)$ and $\cos(w_k)$ for $k = 14$.

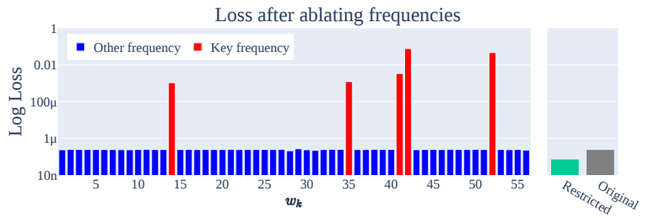


Figure 6: The loss of the transformer (lower=better) when ablating each frequency $k \in \{1, 2, ..., 56\}$ and *everything except for* the five key frequencies (restricted loss). We include the original unablated loss for reference. Ablating key frequencies causes a performance drop, while the other ablations do not harm performance.

Maps to the logits are localized by frequency. We partition these 433 neurons by the frequencies with the highest variance explained. For each resulting subset, the map W_L from neurons to logits has only two non-trivial components, corresponding to sine and cosine at that frequency. For example, in Figure 5 we plot the 44 columns of W_L corresponding to the 44 neurons in the $k = 14$ cluster and find that the only non-negligible components are $\sin\left(\frac{2k\pi}{P}\right)$ and $\cos\left(\frac{2k\pi}{P}\right)$ for $k = 14$.

4.4 Correctness checks: ablations

In previous sections, we showed that various components of the model were well-approximated by sparse combinations of sines and cosines. We verify that these approximations are faithful to the model's functionality, by replacing each component with its approximation. This generally does not hurt the performance of the model and in some cases *improves* it.

MLP neurons. In Section 4.3, we identified 433 neurons that were well-approximated by a degree-2 polynomial. We replace each of these neurons' activation value by the corresponding polynomial, leaving the other neurons untouched. This increases loss by only 3% in relative terms (from $2.41 \cdot 10^{-7}$ to $2.48 \cdot 10^{-7}$) and has no effect on accuracy.

We can instead apply a stricter ablation to the MLP layer and restrict each neuron's activation to just the components of the polynomial corresponding to terms of the form $\cos(w_k(a+b))$ and $\sin(w_k(a+b))$ in the key frequencies. This *improves* loss by 77% (to $5.54 \cdot 10^{-8}$), validating that the logits are calculated by trig identities of neurons as detailed in Section 4.2.

Logit frequencies. Next, we ablate various components of the final logits in the Fourier space. To do so, we take a 2D DFT on the $113 \cdot 113 \cdot 113$ logit matrix over all $113 \cdot 113$ pairs of inputs to get the logits in the Fourier basis, then set various frequencies in this basis to 0.

We begin by ablating the components corresponding to each of the key frequencies. As reported in Figure 6, ablating any key frequency causes a significant increase in loss. This confirms that the five frequencies identified in previous sections are indeed necessary components of the transformer. In contrast, ablating other frequencies does not hurt the model at all.

We then ablate *all* $113 \cdot 113 - 40$ of the Fourier components besides key frequencies; this ablation actually *improves* performance (loss drops 70% to $7.24 \cdot 10^{-8}$).

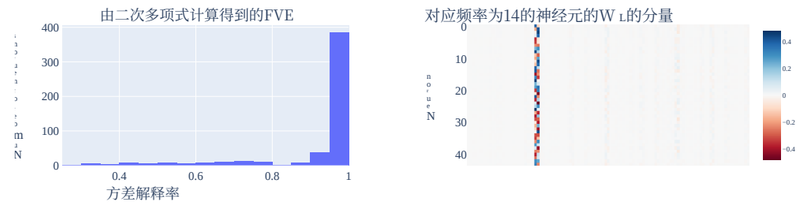


图5：（左）大多数神经元可被单一频率的二次多项式很好地近似。（右）热力图展示了频率为14的44个神经元所对应的 W_L 中的嵌入层权重矩阵权重值，其中非平凡的成分对应于 $k = 14$的$\sin(w_k)$和$\cos(w_k)$。

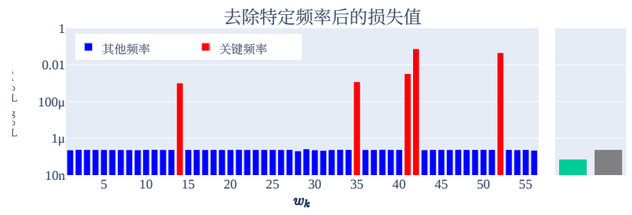


图6：通过进行消融实验，观察在剔除各个频率 $k \in \{1, 2, ..., 56\}$以及除五大关键频率之外的所有元素（即受限损失情况）时Transformer模型的损失值变化情况——损失值越低表示性能越好。我们同时给出了未进行消融实验时的原始损失值以供参考。剔除关键频率会导致模型性能下降，而其他消融实验则不会对性能产生负面影响。

映射到逻辑值的函数会根据频率进行区分。我们按照方差解释率最高的频率将这些433个神经元划分开。对于每一个这样的子集，从神经元到逻辑值的映射 W_L 仅包含两个非零分量，分别对应该频率下的正弦函数和余弦函数。例如，在图5中，我们展示了 W_L 与该 $k = 14$ 簇中的44个神经元相对应的44列数据，结果发现仅有$\sin\left(\frac{2k\pi}{P}\right)$和$\cos\left(\frac{2k\pi}{P}\right)$这两个分量具有不可忽略的数值。

4.4 正确性检验：消融实验

在前面的章节中，我们已经证明模型的各种组成部分都可以通过正弦函数和余弦函数的稀疏组合来很好地近似。为了验证这些近似值确实能够保持模型原有的功能，我们会用相应的近似值替换每个组件。通常这种做法不会损害模型的性能，而在某些情况下甚至还能提升 其性能。

多层感知机神经元。在4.3节中，我们找到了433个能够被二次多项式良好拟合的神经元。我们将这些神经元各自的激活值替换为对应的多项式表达式，而其余神经元则保持不变。这样仅会使损失值相对上升3% （从 $2.41\ 10^{-7}$ <style 'id='19'>到 $2.48 \cdot 10^{-7}$<style 'id='22'>），且不会影响准确率。

相反，我们可以对多层感知机层实施更严格的消融实验，将每个神经元的激活值限制为仅包含那些对应于关键频率中形如 $\cos(w_k(a+b))$ 以及 $\sin(w_k(a+b))$ 的多项式项的组成部分。这样的做法能够将损失值降低77%（降至 $5.54 \cdot 10^{-8}$），从而验证了如第4.2节所述，逻辑值确实是通过神经元的三角恒等式计算得出的。

逻辑值频率。接下来，我们会对傅里叶空间中最终逻辑值的各种组成元素进行消融实验。为此，我们会对所有 $113 \cdot 113$ 组输入对应的 $113 \cdot 113 \cdot 113$ 逻辑值矩阵进行二维离散傅里叶变换，从而得到以傅里叶基表示的逻辑值，随后将该基中的各种频率设为0。
我们首先对与各关键频率对应的组件进行删除实验。如图6所示，删除任意一个关键频率都会导致损失值显著上升。这一结果证实了前文所确定的五个频率确实是Transformer模型中不可或缺的组成部分；而删除其他频率则完全不会对模型性能造成影响。

接着我们进行了消融实验，移除了除关键频率之外的所有傅里叶 $113 \cdot 113 - 40$ 分量；实际上这种操作提升了模型性能（损失值下降了70%，降至 $7.24 \cdot 10^{-8}$）。

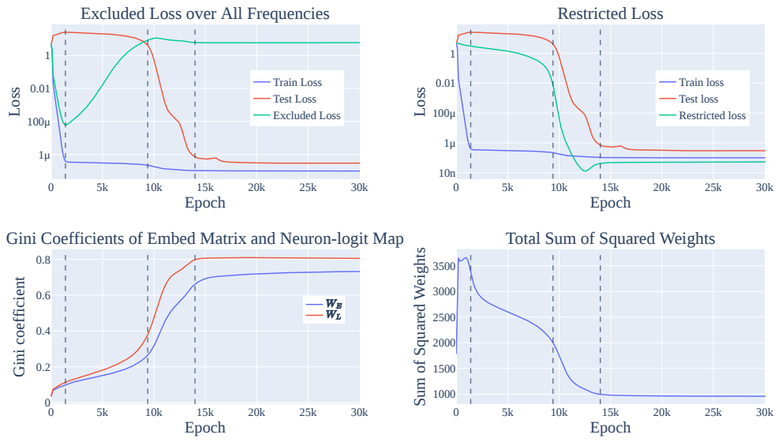


Figure 7: How each of the progress measures in Section 5.1 changes over the course of training. The lines delineate the 3 phases of training: memorization, circuit formation, and cleanup (and a final stable phase). *(Top Left)* Excluded loss increases during circuit formation, while train and test loss remain flat. *(Top Right)* The restricted loss begins declining before test loss declines, but has an inflection point when grokking begins to occur. *(Bottom Left)* The Gini coefficient of the norms of the Fourier components of W_E and W_L increase sharply during cleanup. *(Bottom Right)* The sums of squared weights decreases smoothly during circuit formation and more sharply during cleanup, indicating that both phases are linked to weight decay.

Directions in W_L. In Section 4.2, we found that W_L is well approximated by the 10 directions corresponding to the cosine and sine of key frequencies. If we project the MLP activations to these 10 directions, loss decreases 50% to $1.19 \cdot 10^{-7}$. If we instead projected the MLP activations onto the nullspace of these 10 directions, loss increases to 5.27—worse than uniform. This suggests that the network achieves low loss using these and *only* these 10 directions.

5 Understanding grokking behavior using progress measures

We now use our mechanistic understanding of the network to define two *progress measures*: metrics that can be computed during training that track the progress of the model over the course of training, including during phase transitions. This allows us to study *how* the network reaches its final solution.

5.1 Progress measures

We translate the ablations in Section 4.4 into two progress measures: restricted and excluded loss.

Restricted loss. Since the final network uses a sparse set of frequencies w_k, it makes sense to check how well intermediate versions of the model can do using only those frequencies. To measure this, we perform a 2D DFT on the logits to write them as a linear combination of waves in a and b, and set all terms besides the constant term and the 20 terms corresponding to $\cos(w_k(a+b))$ and $\sin(w_k(a+b))$ for the five key frequencies to 0. We then measure the loss of the ablated network.

Excluded loss. Instead of keeping the important frequencies w_k, we next remove *only* those key frequencies from the logits but keep the rest. We measure this on the *training* data to track how much of the performance comes from Fourier multiplication versus memorization. The idea is that the memorizing solution should be spread out in the Fourier domain, so that ablating a few directions will leave it mostly unaffected, while the generalizing solution will be hurt significantly.

Beyond these, we will also measure (1) the Gini coefficient (Hurley & Rickard, 2009) of the norms of the Fourier components of W_E and W_L, which measures the sparsity of W_E and W_L in the Fourier basis, and (2) the ℓ_2-norm of the weights during training, since weight decay should push these down once the train loss is near zero.

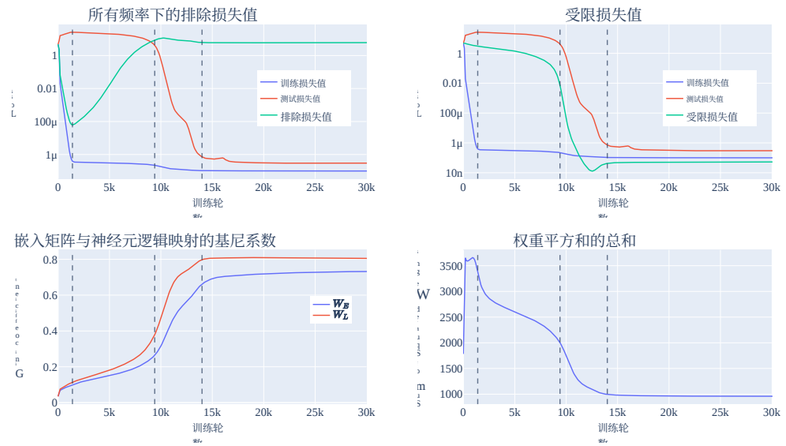


图7：第5.1 节中列出的各项进展度量指标在训练过程中的变化情况。这些曲线反映了训练的三个阶段：记忆阶段、电路形成阶段、清理阶段，以及最后的稳定阶段。（左上角）在电路形成阶段，排除损失会持续上升，而训练损失与测试损失则保持平稳。（右上角）受限损失在测试损失开始下降之前就已出现下降趋势，但在理解能力提升开始显现时会出现一个拐点。（左下角） W_E 和 W_L 的傅里叶分量的范数的基尼系数在清理阶段会急剧上升。（右下角）权重平方和在电路形成阶段呈平滑下降趋势，在清理阶段则下降得更明显，这表明这两个阶段都与权重衰减现象密切相关。

在 W_L中定义的若干方向。在第4.2节中，我们发现 W_L 可以通过与关键频率的余弦函数及正弦函数对应的10个方向来很好地近似。如果将多层感知机的激活值投影到这10个方向上，损失值会降低50%，达到 $1.19 \cdot 10^{-7}$。而如果将这些激活值投影到这10个方向的零空间上，损失值则会上升至5，甚至比均匀分布时的情况还要糟糕。这一结果表明，网络正是借助这些且仅这10个方向才得以实现较低的损失值。

5 使用进展度量指标理解理解能力

借助我们对网络结构的内在机制理解，我们现在定义了两种进展度量指标：即在训练过程中可以计算出来的、用于追踪模型在整个训练周期内进展情况的指标，包括在相变现象发生时的表现。通过这些指标，我们能够研究网络是如何最终找到最优解的。

5.1 进展度量指标

我们会将第4.4节中提到的消融实验转化为两项进展度量指标：受限损失与排除损失。

受限损失。由于最终模型仅使用一组稀疏的频率 w_k，因此有必要检验模型中间版本仅依靠这些频率时的表现。为此，我们对逻辑值进行二维离散傅里叶变换，将其表示为 a 和 b中波形的线性组合；同时将除常数项以及对应于五个关键频率的$\cos(w_k(a+b))$和$\sin(w_k(a+b))$这20项之外的所有项均设为0。随后即可计算经过消融处理后的模型的损失值。

排除损失。我们不会保留重要的频率 w_k，而是从逻辑值中仅移除那些关键频率，其余频率则保持不变。我们通过训练数据来衡量这一指标，以此判断模型的性能在多大程度上源自傅里叶乘法作用，又在多大程度上源于记忆机制。其核心原理在于：依赖记忆的解在傅里叶域中应当是较为分散的，因此即便对部分频率方向进行消融，该解也不会受到太大影响；而具备泛化能力的解则会受到显著损害。

除此之外，我们还会衡量以下两项指标：其一为Hurley与Rickard（2009年）提出的 W_E 以及 W_L 对应的傅里叶分量的范数的基尼系数，该系数可用于衡量 W_E 以及 W_L 在傅里叶基中的稀疏性；其二为训练过程中权重的 ℓ_2-范数，因为一旦训练损失接近零，权重衰减机制就会促使这些数值下降。

5.2 PHASES OF GROKKING: MEMORIZATION, CIRCUIT FORMATION, AND CLEANUP

Using the mainline model from Section 4, we plot the excluded loss, restricted loss, Gini coefficient of the matrices W_U and W_L, and sum of squared weights in Figure 7. We find that training splits into three phases, which we call the memorization, circuit formation, and cleanup phases. (We show similar results for other models in Appendix C.2.)

Memorization (Epochs 0k–1.4k). We first observe a decline of both excluded and train loss, with test and restricted loss both remaining high and the Gini coefficient staying relatively flat. In other words, the model memorizes the data, and the frequencies w_k used by the final model are unused.

Circuit formation (Epochs 1.4k–9.4k). In this phase, excluded loss rises, sum of squared weights falls, restricted loss starts to fall, and test and train loss stay flat. This suggests that the model's behavior on the train set transitions smoothly from the memorizing solution to the Fourier multiplication algorithm. The fall in the sum of squared weights suggests that circuit formation likely happens due to weight decay. Notably, the circuit is formed *well before* grokking occurs.

Cleanup (Epochs 9.4k–14k). In this phase, excluded loss plateaus, restricted loss continues to drop, test loss suddenly drops, and sum of squared weights sharply drops. As the completed Fourier multiplication circuit both solves the task well and has lower weight than the memorization circuit, weight decay encourages the network to shed the memorized solution in favor of focusing on the Fourier multiplication circuit. This is most cleanly shown in the sharp increase in the Gini coefficient for the matices W_E and W_L, which shows that the network is becoming sparser in the Fourier basis.

5.3 GROKKING AND WEIGHT DECAY

In the previous section, we saw that each phase of grokking corresponded to an inflection point in the ℓ_2-norm of the weights. This suggests that weight decay is an important component of grokking and drives progress towards the generalizing solution. In Appendix D.1, we provide additional evidence that weight decay is necessary for grokking: smaller amounts of weight decay causes the network to take significantly longer to grok (echoing the results on toy models from Liu et al. (2022)), and our networks do not grok on the modular arithmetic task without weight decay or some other form of regularization. In Appendix C.2, we also find that the amount of data affects grokking: when networks are provided with enough data, there is no longer a gap between the train and test losses (instead, both decline sharply some number of epochs into training). Finally, in Appendix D.3 we replicate these results on several additional algorithmic tasks.

6 CONCLUSION AND DISCUSSION

In this work, we use mechanistic interpretability to define progress measures for small transformers trained on a modular addition task. We find that the transformers embed the input onto rotations in $\mathbb{R}^2$ and compose the rotations using trigonometric identities to compute $a + b \mod 113$. Using our reverse-engineered algorithm, we define two progress measures, along which the network makes continuous progress toward the final algorithm prior to the grokking phase change. We see this work as a proof of concept for using mechanistic interpretability to understand emergent behavior.

Larger models and realistic tasks. In this work, we studied the behavior of small transformers on a simple algorithmic task, solved with a single circuit. On the other hand, larger models use larger, more numerous circuits to solve significantly harder tasks (Cammarata et al., 2020; Wang et al., 2022). The analysis reported in this work required significant amounts of manual effort, and our progress metrics are specific to small networks on one particular algorithmic task. Methods for automating the analysis and finding task-independent progress measures seem necessary to scale to other, larger models. We discuss possible scenarios for more realistic applications in Appendix F.

Discovering phase change thresholds. While the progress measures we defined in Section 5.1 increase relatively smoothly before the phase transition (and suffice to allow us to understand grokking for this task) we lack a general notion of criticality that would allow us to predict *when* the phase transition will happen ex ante. Future work should develop theory and practice in order to apply progress measures to predict the timing of emergent behavior.

5.2 理解能力的阶段划分：记忆阶段、电路形成阶段与清理阶段

借助第4节中介绍的主线模型，我们在7图中绘制了排除损失、受限损失、矩阵 W_U 以及 W_L的基尼系数，还有权重平方和。研究结果表明，训练过程可划分为三个阶段，即记忆阶段、电路形成阶段和清理阶段。（其他模型的类似结果可见附录C.2节。）

记忆阶段（训练轮数：0千–1.4千）。在此阶段，排除损失与训练损失均出现下降，而测试损失与受限损失依旧处于较高水平，基尼系数也基本保持稳定。换言之，模型正处于数据记忆阶段，最终模型所使用的频率实际上并未被使用。
电路形成阶段（训练轮数：1.4千–9.4千）。在这一阶段，排除损失上升，权重平方和下降，受限损失开始下降，同时测试损失与训练损失则保持平稳。这表明模型在训练集上的表现正从单纯的记忆模式平滑过渡到基于傅里叶乘法算法的运作模式。权重平方和的下降则说明电路形成很可能是由权重衰减作用导致的。值得注意的是，这种电路结构早在理解能力提升<span style id='3'>之前就已经形成了。

清理阶段（训练轮数：1.4万–2万）。在此阶段，排除损失会趋于平稳，受限损失持续下降，测试损失会突然降低，权重平方和也会急剧减少。由于已完成构建的傅里叶乘法电路不仅能更高效地完成任务，其权重也低于记忆阶段的电路结构，权重衰减机制便会促使网络放弃已记忆的解，转而专注于傅里叶乘法电路。这一点从和对应的基尼系数出现大幅上升中可以最清晰地体现出来，这表明网络在傅里叶基上的结构正变得越来越稀疏。

5.3 理解能力提升与权重衰减

在上一节中我们已经看到，理解能力提升的每个阶段都对应着权重{-范数}的一个拐点ℓ_2。这一现象说明权重衰减是理解能力提升过程中的重要因素，它推动模型不断向泛化解发展。在附录D.1中，我们提供了更多证据表明权重衰减对于实现理解能力提升是不可或缺的：较小的权重衰减幅度会导致模型需要更长的时间才能完成理解能力提升过程（这一结果与Liu等人(2022年)在玩具模型上得到的研究结果一致），而且如果没有权重衰减或其他形式的正则化处理，我们的模型在模运算任务上也无法实现理解能力提升。在附录C.2中，我们还发现数据量也会影响理解能力提升的效果：当模型拥有足够的数据时，训练损失与测试损失之间就不再存在差距（相反，在训练开始后的若干个训练轮数后，两者都会出现急剧下降）。最后，在附录D.3中，我们在多种其他算法任务上重新验证了这些研究结果。

6 结论与讨论

在本研究中，我们借助机制可解释性为在模块加法任务上训练的小型变换器制定了进展度量指标。研究发现，这些变换器会将输入映射到 $\mathbb{R}^2$ 中的特定旋转角度，再通过三角恒等式组合这些旋转角度，从而计算出 $a + b$ 模113的结果。基于我们逆向工程得到的算法，我们定义了两个进展度量指标，网络会在顿悟阶段转变之前沿着这两个指标持续向最终算法的目标靠近。我们认为这项工作为利用机制可解释性理解涌现行为提供了概念验证。

更大规模模型与现实任务。在本研究中，我们分析了小型变换器在仅通过单个电路结构即可解决的简单算法任务上的表现；而另一方面，更大规模模型则会借助更多、规模更大的电路结构来处理难度高得多的任务（Cammarata等人，2020年；Wang等人，2022年）。本研究中所进行的分析需要大量的人工操作，而且我们所设计的进展度量指标也仅适用于特定算法任务下的小型网络。若要将其应用到其他更大规模的模型上，似乎有必要开发自动化分析的方法，并找到与任务无关的进展度量指标。关于在更现实场景中应用该技术的可行方案，我们已在附录F中进行了探讨。

探究相变阈值。虽然我们在第5.1 节中定义的进展度量指标在相变发生之前会相对平稳地上升（足以让我们借此理解理解能力提升这一现象），但我们目前还缺乏一种能够用于预测何时相变将会发生的临界性概念。未来的研究应当进一步发展相关的理论与方法，以便运用这些进展度量指标来预测涌现行为的出现时间。

REPRODUCIBILITY STATEMENT

An annotated Colab notebook containing the code to replicate our results, including download instructions for model checkpoints, is available at https://neelnanda.io/grokking-paper.

AUTHOR CONTRIBUTIONS

Neel Nanda was the primary research contributor. He reverse engineered the weights of the mainline model to discover the Fourier multiplication algorithm and found the lines of evidence in Section 4. He also discovered the restricted and excluded loss progress measures and that grokking in mainline model could be divided into three discrete phases. Finally, he found the link between grokking, limited data, and phase transitions by exhibiting grokking in other settings with phase transitions.

Lawrence Chan was invaluable to the framing and technical writing of this work. In addition, he created the Gini coefficient progress measure and performed the analysis in the appendices exploring to what extent the results on the mainline model applied to the other small transformer models, including with other random seeds, architectures, prime moduli, and regularization methods.

Tom Lieberum contributed to the early stages of this work by creating a minimal setup of grokking with a 1L Transformer on the modular addition task with no LayerNorm and finding the surprising periodicity within the model's internals.

Jess Smith performed experiments exploring grokking with different random seeds, architectures, and other hyper-parameters.

Jacob Steinhardt helped clarify and distill the results, provided significant amounts of editing and writing feedback, and suggested the progress measure frame.

ACKNOWLEDGMENTS

In writing this paper, our thinking and exposition was greatly clarified by correspondence with and feedback from Oliver Balfour, David Bau, Sid Black, Nick Cammarata, Stephen Casper, Bilal Chughtai, Arthur Conmy, Xander Davies, Ben Edelman, Nelson Elhage, Ryan Greenblatt, Jacob Hilton, Evan Hubinger, Zac Kenton, Janos Kramar, Lauro Langosco, Tao Lin, David Lindner, Eric Michaud, Vlad Mikulik, Noa Nabeshima, Chris Olah, Michela Paganini, Michela Paganini, Alex Ray, Rohin Shah, Buck Shlegeris, Alex Silverstein, Ben Toner, Johannes Treutlein, Nicholas Turner, Vikrant Varma, Vikrant Varma, Kevin Wang, Martin Wattenberg, John Wentworth, and Jeff Wu.

We'd also like to thank Adam Gleave and Chengcheng Tan for providing substantial editing help, and Noa Nabeshima and Vlad Mikulik for pair programming with Neel.

This work draws heavily on the interpretability techniques and framework developed by Elhage et al. (2021) and Olsson et al. (2022).

We trained our models using `PyTorch` (Paszke et al., 2019) and performed our data analysis using `NumPy` (Harris et al., 2020), `Pandas` (Wes McKinney, 2010), and `einops` (Rogozhnikov, 2022). Our figures were made using `Plotly` (Plotly Technologies Inc., 2015).

Neel would like to thank Jemima Jones for providing practical and emotional support as he navigated personal challenges while contributing to this paper, and to the Schelling Residency for providing an excellent research environment during the distillation stage. He would also like to thank the Anthropic interpretability team, most notably Chris Olah, for an incredibly generous amount of mentorship during his time there, without which this investigation would never have happened.

REFERENCES

Boaz Barak, Benjamin L Edelman, Surbhi Goel, Sham Kakade, Eran Malach, and Cyril Zhang. Hidden progress in deep learning: Sgd learns parities near the computational limit. *arXiv preprint arXiv:2207.08799*, 2022.

可复现性声明

一份附有注释的Colab笔记本提供了复现我们实验结果的代码，其中还包含了模型检查点的下载说明，可访问地址为https://neelnanda.io/grokking-paper。

作者贡献

尼尔·南达是这项研究的主要贡献者。他通过反推主线模型的嵌入层权重矩阵，发现了傅里叶乘法算法，并在第4节中找到了相关证据。此外，他还发现了用于衡量理解能力提升的受限损失与排除损失等进展度量指标，同时指出主线模型中的理解能力提升可划分为三个独立的阶段。最后，他通过在具有相变现象的其他场景中展示理解能力提升的表现，揭示了理解能力提升、数据量有限以及相变现象之间的关联。

劳伦斯·陈在本文的框架构建与技术写作方面发挥了重要作用。他还提出了基尼系数这一进展度量指标，并在附录中进行了分析，探讨了主线模型上的实验结果在多大程度上适用于其他小型Transformer模型，包括使用不同随机种子、架构、素数模数以及正则化方法的情况。

汤姆·利伯鲁姆在该研究的初期阶段做出了贡献，他利用不含LayerNorm结构的1层Transformer构建了一个用于模块加法任务的简化模型，并在该模型内部发现了惊人的周期性现象。

Jess 史密斯通过使用不同的随机种子与各类架构，开展了探索理解能力提升的实验。 s, and 此外还测试了其他超参数。

雅各布·斯坦哈特协助对研究结果进行了梳理与提炼，提供了大量的编辑与写作方面的反馈，并提出了相应的进展衡量框架。

致谢

在撰写本文的过程中，奥利弗·巴尔弗、大卫·鲍、西德·布莱克、尼克·卡马拉塔、斯蒂芬·卡斯珀、比拉尔·楚赫泰、亚瑟·康米、赞德·戴维斯、本·埃德尔曼、纳尔逊·埃尔哈格、瑞安·格林布拉特、雅各布·希尔顿、埃文·胡宾格、扎克·肯顿、雅诺什·克拉马尔、劳罗·兰戈斯科、林涛、大卫·林德纳、埃里克·米肖、弗拉德·米库利克、诺亚·纳贝希马、克里斯·奥拉、米凯拉·帕加尼尼、亚历克斯·雷、罗欣·沙阿、巴克·施莱格里斯、亚历克斯·西尔弗斯坦、本·托纳、约翰内斯·特罗伊特莱因、尼古拉斯·特纳、维克兰特·瓦尔马、王凯文、马丁·瓦滕伯格、约翰·温特沃思以及吴杰夫等人的来信与反馈，极大地帮助我们明确了思路并提升了文章的表述质量。

我们还要感谢亚当·格利夫与陈诚诚在论文编辑方面提供的宝贵帮助，同时也要感谢诺亚·纳贝希马与弗拉德·米库利克与尼尔一起进行的结对编程协助。

本研究在很大程度上借鉴了 埃尔哈格等人(2021年)以及Olsson等人(2022年)所开发的可解释性技术及相关框架。

我们使用 PyTorch框架(帕斯克等人，2019年)来训练模型，而数据分析则借助NumPy库(哈里斯等人，2020年)、Pandas数据分析库(韦斯·麦金尼，2010年)以及einops运算库(罗戈日尼科夫，2022年)完成。此外，我们的图表也是利用Plotly可视化库(Plotly科技公司，2015年)制作的。

尼尔想要感谢杰米玛·琼斯，在他为撰写这篇论文而面临个人挑战时给予的实际支持与情感鼓励；同时也要感谢谢林驻留项目在模型精简阶段所提供的优质研究环境。此外，他还要特别感谢Anthropic公司的可解释性团队，尤其是克里斯·奥拉，他在那里期间给予的慷慨指导对本次研究而言至关重要，若非这些帮助，这项研究根本无从开展。

参考文献

博阿兹·巴拉克、本杰明·L·埃德尔曼、苏尔比·戈埃尔、沙姆·卡卡德、埃兰·马拉赫以及张西里尔共同撰文指出：深度学习中存在隐藏的进展——随机梯度下降算法能够在接近计算极限的情况下学习到奇偶性。*arXiv*预印本 *arXiv:2207.08799*，2022年。

Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. Language models are few-shot learners. *Advances in neural information processing systems*, 33:1877–1901, 2020.

Nick Cammarata, Shan Carter, Gabriel Goh, Chris Olah, Michael Petrov, Ludwig Schubert, Chelsea Voss, Ben Egan, and Swee Kiat Lim. Thread: Circuits. *Distill*, 2020. doi: 10.23915/distill.00024. https://distill.pub/2020/circuits.

Nelson Elhage, Neel Nanda, Catherine Olsson, Tom Henighan, Nicholas Joseph, Ben Mann, Amanda Askell, Yuntao Bai, Anna Chen, Tom Conerly, Nova DasSarma, Dawn Drain, Deep Ganguli, Zac Hatfield-Dodds, Danny Hernandez, Andy Jones, Jackson Kernion, Liane Lovitt, Kamal Ndousse, Dario Amodei, Tom Brown, Jack Clark, Jared Kaplan, Sam McCandlish, and Chris Olah. A mathematical framework for transformer circuits. *Transformer Circuits Thread*, 2021. https://transformer-circuits.pub/2021/framework/index.html.

Jonathan Frankle and Michael Carbin. The lottery ticket hypothesis: Finding sparse, trainable neural networks. *arXiv preprint arXiv:1803.03635*, 2018.

Deep Ganguli, Danny Hernandez, Liane Lovitt, Amanda Askell, Yuntao Bai, Anna Chen, Tom Conerly, Nova Dassarma, Dawn Drain, Nelson Elhage, et al. Predictability and surprise in large generative models. In *2022 ACM Conference on Fairness, Accountability, and Transparency*, pp. 1747–1764, 2022.

Charles R. Harris, K. Jarrod Millman, Stéfan J. van der Walt, Ralf Gommers, Pauli Virtanen, David Cournapeau, Eric Wieser, Julian Taylor, Sebastian Berg, Nathaniel J. Smith, Robert Kern, Matti Picus, Stephan Hoyer, Marten H. van Kerkwijk, Matthew Brett, Allan Haldane, Jaime Fernández del Río, Mark Wiebe, Pearu Peterson, Pierre Gérard-Marchant, Kevin Sheppard, Tyler Reddy, Warren Weckesser, Hameer Abbasi, Christoph Gohlke, and Travis E. Oliphant. Array programming with NumPy. *Nature*, 585(7825):357–362, September 2020. doi: 10.1038/s41586-020-2649-2. URL https://doi.org/10.1038/s41586-020-2649-2.

Niall Hurley and Scott Rickard. Comparing measures of sparsity. *IEEE Transactions on Information Theory*, 55(10):4723–4741, 2009.

Ziming Liu, Ouail Kitouni, Niklas Nolte, Eric J Michaud, Max Tegmark, and Mike Williams. Towards understanding grokking: An effective theory of representation learning. *arXiv preprint arXiv:2205.10343*, 2022.

Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. *arXiv preprint arXiv:1711.05101*, 2017.

Thomas McGrath, Andrei Kapishnikov, Nenad Tomašev, Adam Pearce, Demis Hassabis, Been Kim, Ulrich Paquet, and Vladimir Kramnik. Acquisition of chess knowledge in alphazero. *arXiv preprint arXiv:2111.09259*, 2021.

Beren Millidge. Grokking 'grokking', 2022. URL https://www.beren.io/2022-01-11-Grokking-Grokking/.

Catherine Olsson, Nelson Elhage, Neel Nanda, Nicholas Joseph, Nova DasSarma, Tom Henighan, Ben Mann, Amanda Askell, Yuntao Bai, Anna Chen, Tom Conerly, Dawn Drain, Deep Ganguli, Zac Hatfield-Dodds, Danny Hernandez, Scott Johnston, Andy Jones, Jackson Kernion, Liane Lovitt, Kamal Ndousse, Dario Amodei, Tom Brown, Jack Clark, Jared Kaplan, Sam McCandlish, and Chris Olah. In-context learning and induction heads. *Transformer Circuits Thread*, 2022. https://transformer-circuits.pub/2022/in-context-learning-and-induction-heads/index.html.

Alexander Pan, Kush Bhatia, and Jacob Steinhardt. The effects of reward misspecification: Mapping and mitigating misaligned models. *arXiv preprint arXiv:2201.03544*, 2022.

Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, et al. Pytorch: An imperative style, high-performance deep learning library. *Advances in neural information processing systems*, 32, 2019.

Plotly Technologies Inc. Collaborative data science, 2015. URL https://plot.ly.

汤姆·布朗、本杰明·曼恩、尼克·莱德、梅兰妮·苏比亚、贾里德·D·卡普兰、普拉富拉·达里瓦尔、阿尔文德·尼拉卡南坦、普拉纳夫·夏姆、吉里什·萨斯特里、阿曼达·阿斯克尔等人研究认为，语言模型属于少样本学习器。神经信息处理系统进展期刊，第33卷，第1877–1901页，2020年。

尼克·卡马拉塔、尚·卡特、加布里埃尔·戈、克里斯·奥拉、迈克尔·彼得罗夫、路德维希·舒伯特、切尔西·沃斯、本·伊根以及斯威基亚特·林共同撰写了《Thread系列论文：电路》一文。*Distill*期刊于2020年发表了该文，doi编号为10.23915/distill.00024，相关链接为https://distill.pub/2020/circuits。

纳尔逊·埃尔哈格、尼尔·南达、凯瑟琳·奥尔森、汤姆·亨尼汉、尼古拉斯·约瑟夫、本·曼恩、阿曼达·阿斯克尔、白云涛、陈安娜、汤姆·康纳利、诺娃·达斯萨玛、道恩·德雷恩、迪普·甘古利、扎克·哈特菲尔德·多兹、丹尼·埃尔南德斯、安迪·琼斯、杰克逊·科尔尼恩、莉安·洛维特、卡马尔·恩杜塞、达里奥·阿莫代、汤姆·布朗、杰克·克拉克、贾里德·卡普兰、萨姆·麦坎德利什与克里斯·奥拉共同提出了用于Transformer电路的数学框架。*Transformer Circuits Thread*系列论文于2021年发布了该框架的相关内容，访问地址为https://transformer-circuits.pub/2021/framework/index.html。

乔纳森·弗兰克尔与迈克尔·卡宾所提出的‘彩票票假说’旨在寻找那些结构稀疏且可训练的神经网络。*arXiv*预印本 *arXiv:1803.03635*，2018年。

迪普·甘古利、丹尼·埃尔南德斯、莉安·洛维特、阿曼达·阿斯克尔、白云涛、陈安娜、汤姆·康纳利、诺娃·达斯萨玛、道恩·德雷恩、纳尔逊·埃尔哈格等人共同撰写了《大型生成模型中的可预测性与意外性》一文，该文发表在*2022*年*ACM*公平性、可解释性与透明度会议上，页码为1747–1764，出版时间为2022年。

查尔斯·R·哈里斯、K·贾罗德·米尔曼、斯特·凡·德沃尔特、拉尔夫·戈默斯、保里·维尔塔宁、大卫·库尔纳波、埃里克·维瑟、朱利安·泰勒、塞巴斯蒂安·伯格、纳撒尼尔·J·史密斯、罗伯特·科尔恩、马蒂·皮库斯、斯蒂芬·霍耶、马滕·H·范克克维克、马修·布雷特、艾伦·霍尔丹、海梅·费尔南·安德斯·德尔·R·奥，马克·维贝、佩鲁·彼得森、皮埃尔·G·埃拉尔·马尔尚、凯文·谢帕德、泰勒·雷迪、沃伦·韦克瑟、哈米尔·阿巴西、克里斯托夫·戈尔克以及特拉维斯·E·奥利芬特所著的《使用NumPy进行数组编程》。《自然》杂志，585(7825):357–362，2020年9月。doi: 10.1038/ s41586-020-2649-2。网址：https://doi.org/10.1038/s41586-020-2649-2。

尼阿尔·赫利与斯科特·里卡德。关于稀疏性度量的比较。《IEEE信息理论汇刊》，55(10):4723–4741，2009年。

刘子明、奥威尔·基图尼、尼克拉斯·诺尔特、埃里克·J·米肖、马克斯·特格马克以及迈克·威廉姆斯。迈向对理解能力提升的认知：一种有效的表示学习理论。arXiv预印本平台，编号arXiv:2205.10343，2022年。

伊利亚·洛希洛夫与弗兰克·哈特。解耦式的权重衰减正则化方法。arXiv预印本平台，编号arXiv:1711.05101，2017年。

托马斯·麦格拉思、安德烈·卡皮什尼科夫、涅纳德·托马塞夫、亚当·皮尔斯、德米斯·哈萨比斯、宾·金、乌尔里希·帕凯以及弗拉基米尔·克拉姆尼克。AlphaZero中的国际象棋知识获取机制。arXiv预印本平台，编号arXiv:2111.09259，2021年。

贝伦·米利奇。《理解“理解能力”》，2022年。网址：https://www.beren.io/2022-01-11-Grokking-Grokking/。

凯瑟琳·奥尔森、纳尔逊·埃尔哈格、尼尔·南达、尼古拉斯·约瑟夫、诺娃·达斯萨玛、汤姆·亨尼汉、本·曼恩、阿曼达·阿斯克尔、白云涛、陈安娜、汤姆·康纳利、道恩·德雷恩、迪普·甘古利、扎克·哈特菲尔德·多兹、丹尼·埃尔南德斯、斯科特·约翰斯顿、安迪·琼斯、杰克逊·科尔尼恩、莉安·洛维特、KamalNdousse、DarioAmodei、TomBrown、JackClark、JaredKaplan、SamMcCandlish以及克里斯·奥拉共同撰写了《上下文学习与归纳头》一文。*Transformer CircuitsThread*，2022年，相关内容可见链接：https://transformer-circuits.pub/2022/in-context-learning-and-induction-heads/index.html。

亚历山大·潘、库什·巴蒂亚与雅各布·斯坦哈特共同研究了《奖励规格错误的影响：目标不一致的模型的映射与缓解方法》。该论文发表于*arXiv*预印本平台，编号为*arXiv:2201.03544*，发布时间为2022年。

Adam Paszke、Sam Gross、Francisco Massa、Adam Lerer、JamesBradbury、Gregory Chanan、Trevor Killeen、Zeming Lin、Natalia Gimelshein、Luca Antiga等人所著的《Pytorch：一种命令式风格的高性能深度学习库》。该文章发表于《神经信息处理系统进展期刊》2019年第32卷。

Plotly科技公司所提出的协作式数据科学概念，最初可追溯至2015年。相关链接如下：https://plot.ly。

Alethea Power, Yuri Burda, Harri Edwards, Igor Babuschkin, and Vedant Misra. Grokking: Generalization beyond overfitting on small algorithmic datasets. *arXiv preprint arXiv:2201.02177*, 2022.

Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, Ilya Sutskever, et al. Language models are unsupervised multitask learners. *OpenAI blog*, 1(8):9, 2019.

Alex Rogozhnikov. Einops: Clear and reliable tensor manipulations with einstein-like notation. In *International Conference on Learning Representations*, 2022. URL https://openreview.net/forum?id=oapKSVM2bcj.

Nitish Srivastava, Geoffrey Hinton, Alex Krizhevsky, Ilya Sutskever, and Ruslan Salakhutdinov. Dropout: a simple way to prevent neural networks from overfitting. *The journal of machine learning research*, 15(1):1929–1958, 2014.

Jacob Steinhardt. More is different for ai, Feb 2022. URL https://bounded-regret.ghost.io/more-is-different-for-ai/.

Vimal Thilak, Etai Littwin, Shuangfei Zhai, Omid Saremi, Roni Paiss, and Joshua Susskind. The slingshot mechanism: An empirical study of adaptive optimizers and the grokking phenomenon. *arXiv preprint arXiv:2206.04817*, 2022.

Kevin Wang, Alexandre Variengien, Arthur Conmy, Buck Shlegeris, and Jacob Steinhardt. Interpretability in the wild: a circuit for indirect object identification in gpt-2 small. *arXiv preprint arXiv:2211.00593*, 2022.

Jason Wei, Yi Tay, Rishi Bommasani, Colin Raffel, Barret Zoph, Sebastian Borgeaud, Dani Yogatama, Maarten Bosma, Denny Zhou, Donald Metzler, et al. Emergent abilities of large language models. *arXiv preprint arXiv:2206.07682*, 2022a.

Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Ed Chi, Quoc Le, and Denny Zhou. Chain of thought prompting elicits reasoning in large language models. *arXiv preprint arXiv:2201.11903*, 2022b.

Wes McKinney. Data Structures for Statistical Computing in Python. In Stéfan van der Walt and Jarrod Millman (eds.), *Proceedings of the 9th Python in Science Conference*, pp. 56 – 61, 2010. doi: 10.25080/Majora-92bf1922-00a.

Alethea Power、Yuri Burda、Harri Edwards、Igor Babuschkin以及Vedant Misra共同发表了题为《Grokking：在规模较小的算法数据集上克服过拟合以实现泛化》的研究论文。该论文为*arXiv*预印本，编号为*arXiv:2201.02177*，发表时间为*2022*年。同年，*Alec Radford*、*Jeffrey Wu*、*Rewon Child*、*David Luan*、*Dario Amodei*、*Ilya Sutskever*等人发表了另一篇论文，题为《语言模型是无需监督的多任务学习器》。该内容来自*OpenAI*博客。此外，Alex Rogozhnikov在2019年的《国际学习表征会议》上发表的文章《Einops：借助类爱因斯坦式的符号实现清晰可靠的张量操作》也极具参考价值，文章编号为1(8):9，相关链接为国际学习表征会议，链接为https://openreview.net/forum?idoapKSVM2bcj。Nitish Srivastava、Geoffrey Hinton、Alex Krizhevsky、Ilya Sutskever以及Ruslan Salakhutdinov则提出了丢弃机制，这是一种简单有效的防止神经网络过拟合的方法，相关论文发表于《机器学习研究期刊》。该文章发表于*2014*年，编号为*15(1):1929–1958*，作者为*Jacob Steinhardt*，其*2022*年*2*月发表的文章《对于人工智能而言，“更多往往更好”》也提供了相关见解，链接为https://bounded-regret.ghost.io/more-is-different-for-ai/。Vimal Thilak、Etai Littwin、Shuangfei Zhai、Omid Saremi、Roni Paiss以及Joshua Susskind则对弹弓机制进行了实证研究，该机制与自适应优化器以及理解能力提升现象密切相关，相关论文为arXiv预印本，编号为arXiv:2206.04817，发表时间为2022年。*Kevin Wang*、*Alexandre Variengien*、亚瑟·康米、巴克·施莱格里斯以及雅各布·斯坦哈特共同发表了题为《野外的可解释性：用于*GPT-2 Small*模型中间接宾语识别的电路结构》的研究论文，该论文为*arXiv*预印本，编号为*arXiv:2211.00593*，同样发表于*2022*年。Jason Wei、Yi Tay、Rishi Bommasani、Colin Raffel、Barret Zoph、Sebastian Borgeaud、Dani Yogatama、Maarten Bosma、Denny Zhou、Donald Metzler等人研究了大型语言模型的涌现能力，相关论文为arXiv预印本，编号为arXiv:2206.07682，发布于2022年。同年，*Jason Wei*、*Xuezhi Wang*、*Dale Schuurmans*、*Maarten Bosma*、*Ed Chi*、*Quoc Le*以及*Denny Zhou*发表了题为《通过思维链提示引导大型语言模型进行推理》的论文，该论文为*arXiv*预印本，编号为*arXiv:2201.11903*，发布于*2022*年。*Wes McKinney*在*2022*年撰写了《*Python*统计计算中的数据结构》一文，该文收录于*Stefan van der Walt*和*Jarrod Millman*主编的《第*9*届*Python*在科学中的应用会议论文集》中，发表在*2010*年，页码为*56 – 61*，对应的*DOI*编号为*10.25080/Majora-92bf1922-00a*。

A MATHEMATICAL STRUCTURE OF THE TRANSFORMER

We follow the conventions and notation of Elhage et al. (2021) in describing our model. Here, we briefly recap their notation and examine it in our specific case.

We denote our hyperparameters as follows: $d_{vocab} = 113$ is the size of the input and output spaces (treating '=' separately), $d_{model} = 128$ is the width of the residual stream (i.e. embedding size), $d_{head} = 32$ is the size of query, key and value vectors for a single attention head, and $d_{mlp} = 512$ is the number of neurons.

We denote the parameters as follows: W_E (embedding layer); W_{pos} (positional embedding); W_Q^j (queries), W_K^j (keys), W_V^j (values), W_O^j (attention output) (the 4 weight matrices of head j in the attention layer); W_{in} and b_{in} for the input linear map of the MLP layer; W_{out} and b_{out} for the output linear map of the MLP layer; and W_U (unembedding layer). Note that we do not have biases in our embedding, attention layer or unembedding, and we do not tie the matrices for the embedding/unembedding layers.

We now describe the mathematical structure of our network. Note that loss is only calculated from the logits on the final token, and information only moves between tokens during the attention layer, so our variables from the end of the attention layer onwards only refer to the final token. We use t_i to denote the token in position i (as a one-hot encoded vector), p_i to denote the ith positional embedding, $x_i^{(0)}$ to denote the initial residual stream on token with index i, $A^{(i)}$ to denote the attention scores from $=$ to all previous tokens from head i, $x^{(1)}$ to denote the residual stream after the attention layer on the final token, MLP to denote the neuron activations in the MLP layer on the final token, $x^{(2)}$ the final residual stream on the final token, Logits the logits on the final token.

The logits are calculated via the following equations:

$$
\begin{aligned}
x_i^{(0)} &= W_E t_i + p_i \\
A^j &= \text{softmax}({x^{(0)}}^T {W_K^j}^T W_Q^j x_2^{(0)}) \\
x^{(1)} &= [\sum_j W_O^j W_V^j (x^{(0)} \cdot A^j)] + x_2^{(0)} \\
\text{MLP} &= \text{ReLU}(W_{in} x^{(1)}) \\
x^{(2)} &= W_{out} N + x^{(1)} = W_{out} \text{ReLU}(W_{in} x^{(1)}) + x^{(1)} \\
\text{Logits} &= W_U x^{(2)}
\end{aligned}
$$

As in Elhage et al. (2021), we refer to the term $W_O^j W_V^j(x^{(0)})$ as the OV circuit for head j.

A.1 EMPIRICAL MODEL SIMPLIFICATIONS

We make two empirical observations:

- The attention paid from '=' to itself is trivial. In practice, the average attention paid is 0.1% to 0.4% for each head, and ablating this does not affect model performance at all.
- The skip connection around the MLP layer is not important for the model's computation and can be ignored. Concretely, if we set it to zero or to its average (zero or mean ablation) then model accuracy is unchanged, and loss goes from $2.4 \cdot 10^{-7}$ to $9.12 \cdot 10^{-7}$ and $7.25 \cdot 10^{-7}$ respectively. This is a significant increase in loss, but from such a small baseline that we can still ignore it and reverse engineer the model's computation. (That being said, both the attention heads and the skip connection around them are crucial to the functioning of the model: zero ablating attention heads increases loss to 24.3, while zero ablating the skip connection around the attention heads increases loss to 19.1, both *significantly* worse than chance.)

A consequence of the first observation is that the attention is now a softmax over 2 elements, i.e. a sigmoid over the difference. And $x_2^{(0)}$ is constant, as it is independent of x and y, and the embedding

Transformer模型的数学结构

我们在描述所构建的模型时，采用了Elhage等人（2021）所制定的规范与符号表示法。在此，我们会简要回顾他们的符号体系，并结合具体案例对其进行分析。

我们采用以下方式来标记各超参数： $d_{vocab} = 113$ 表示输入空间与输出空间的维度大小（其中'='会被单独处理）， $d_{model} = 128$ 代表残差流的宽度，即位置嵌入的维度；$d_{head} = 32$ 则是单个注意力头中查询向量、键向量和值向量的维度大小；而 $d_{mlp} = 512$则表示神经元的数量。

各参数的标识方式如下： W_E 对应嵌入层； W_{pos} 对应位置嵌入层； W_Q^j 对应查询向量， W_K^j 对应键向量， W_V^j 对应值向量， W_O^j 对应注意力层的输出，也就是注意力层中属于头部 j 的4个权重矩阵； W_{in} 和 b_{in} 用于标记多层感知机层中的输入线性映射； W_{out} 和 b_{out} 用于标记多层感知机层中的输出线性映射；最后 W_U 对应解嵌入层。需要说明的是，我们的嵌入层、注意力层以及解嵌入层均不设置偏置项，同时这些层的权重矩阵之间也没有任何关联。

接下来我们将介绍该网络的数学结构。需要说明的是，损失值仅根据最后一个标记的逻辑值来计算，且信息仅在注意力层中在各个标记之间传递，因此从注意力层末端开始的变量均仅指代最后一个标记。我们使用 t_i 来表示位置为 i 的标记（以one-hot编码向量形式）， p_i 表示第 i 个位置嵌入， $x_i^{(0)}$ 表示索引为 i 的标记上的初始残差流， $A^{(i)}$ 表示来自注意力头 i 的、从 $=$ 到所有先前标记的注意力分数， $x^{(1)}$ 表示最后一个标记在经过注意力层处理后的残差流，MLP表示最后一个标记所在MLP层中的神经元激活值， $x^{(2)}$ 表示最后一个标记上的最终残差流，Logits则表示最后一个标记的逻辑值。

这些逻辑值是通过以下公式计算得出的：

$$
\begin{aligned}
x_i^{(0)} &= W_E t_i + p_i \\
A^j &= \text{softmax}({x^{(0)}}^T {W_K^j}^T W_Q^j x_2^{(0)}) \\
x^{(1)} &= [\sum_j W_O^j W_V^j (x^{(0)} \cdot A^j)] + x_2^{(0)} \\
\text{MLP} &= \text{ReLU}(W_{in} x^{(1)}) \\
x^{(2)} &= W_{out} N + x^{(1)} = W_{out} \text{ReLU}(W_{in} x^{(1)}) + x^{(1)} \\
\text{Logits} &= W_U x^{(2)}
\end{aligned}
$$

如Elhage等人（2021）所述，我们将 $W_O^j W_V^j(x^{(0)})$ 称为注意力头 j的OV电路结构。

A.1 实证模型简化

我们得到了两项实证观察结果：

- '=' 对其自身所分配的注意力值极为微小。实际上，每个注意力头的平均注意力值仅在0.1%到0.4%之间，进行此类消融实验完全不会影响模型性能。
- 多层感知机层周围的跳接连接对模型的计算过程并无重要影响，因此可以忽略不计。具体而言，若将该连接的值设为零或取其平均值（即零消融或均值消融），模型的准确率不会发生变化，而损失值则分别从 $2.4 \cdot 10^{-7}$ 上升至 $9.12 \cdot 10^{-7}$ 和 $7.25 \cdot 10^{-7}$。虽然损失值有所上升，但由于初始基线本身就很低，我们依然可以忽略这一变化，继续逆向分析模型的计算逻辑。（需要说明的是，注意力头以及其周围的跳接连接对模型的正常运行至关重要：若将注意力头设为零，损失值会升至24.3；而若将注意力头周围的跳接连接设为零，损失值则会升至19.1，这两种情况的损失程度都远远高于随机水平。）

第一个观察结果带来的一个后果是，现在的注意力机制实际上是对2个元素应用softmax函数，即对两者之差再通过sigmoid函数处理。同时， $x_2^{(0)}$ 是一个常数，因为它与 x 、 y以及位置嵌入无关。

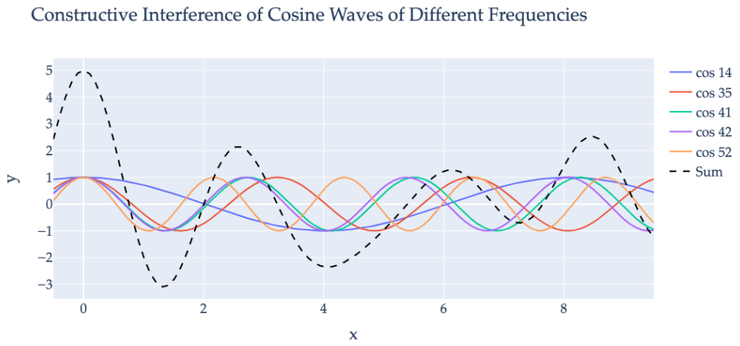


Figure 8: As discussed in Appendix B, while for every $k \in [0, ...P-1]$, $\cos\left(\frac{2k\pi}{P}x\right)$ achieves its maximum value (1) at $x = 0 \mod 113$, it still has additional peaks at different values that are close to the maximum value. However, by adding together cosine waves of the 5 keyfrequencies, the model constructs a periodic function where the value at $x = 0 \mod 113$ is significantly larger than its value anywhere else.

and positional embedding of '=' are fixed. So $A_0^j = \sigma\left({x_2^{(0)}}^T W_Q^{j^T} W_K (x_0^{(0)} - x_1^{(0)})\right)$ (and $A_1^j = 1 - A_0^j$)

A consequence of the second observation is that Logits $\approx W_U W_{out}$MLP, which we denote as $W_L = W_U W_{out}$. From the perspective of the network, W_L is the meaningful matrix, not either of its constituents, since they compose linearly.

B WHY USE CONSTRUCTIVE INTEREFERENCE?

As demonstrated in Section 4 and Appendix C.2.1, small transformers trained on this task use several different frequencies which they add together. The reason for this is to end up with a function whose value at $x = 0 \mod 113$ is significantly larger than any other x.

For example, consider the function $f_{14}(x) = \cos\left(\frac{2\pi \cdot 14}{113}x\right)$. This function has period 113 and is maximized at $x = 0 \mod 113$. However, other values of x cause this function to be close to 1: $f_{14}(8) = f_{14}(105) = 0.998$, $f_{14}(16) = f_{14}(89) = 0.994$, etc.

Now consider $f_{35}(x) = \cos\left(\frac{2\pi \cdot 35}{113}x\right)$. While this function also has period 113 and is maximized at $x = 0 \mod 113$, it turns out that $f_{35}(8) = f_{35}(105) = -0.990$. This means that by adding together f_{14} and f_{35}, we end up with a function that is not close to 1 at $x = 8 \mod 113$. Similarly, while $f_{35}(16) = 0.961$, $f_{52}(16) = -0.56$, and so adding a third frequency reduces the peak at $x = 16 \mod 113$.

We show the constructive interference resulting from the cosine waves for the five frequencies used by the mainline model in Figure 8.

C SUPPORTING EVIDENCE FOR MECHANISTIC ANALYSIS OF MODULAR ARITHMETIC NETWORKS

C.1 FURTHER ANALYSIS OF THE SPECIFIC TRAINING RUN DISCUSSED IN THE PAPER

In this section, we provide additional evidence relating to the mainline model.

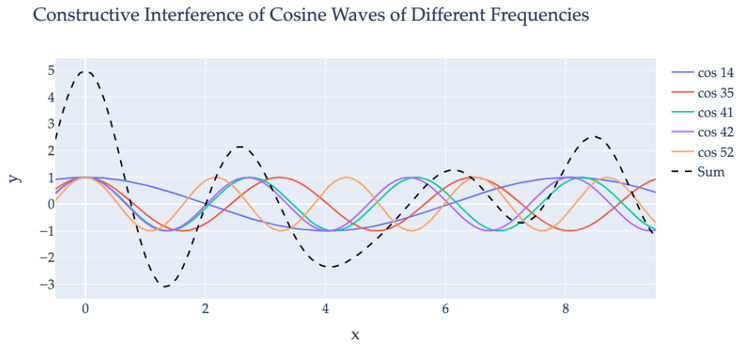


图8：如附录中所论述的B，对于每一个 $k \in [0, ...P-1]$，余弦函数 $\left(\frac{2k\pi}{P}x\right)$ 在 $x = 0 \mod 113$处会取得其最大值（1），不过它仍在接近该最大值的其他数值处出现额外的峰值。然而，通过将5个不同频率的余弦波相加，模型能够构建出一个周期函数，使得在 $x = 0 \mod 113$ 处的函数值远远高于其他任何位置的函数值。

此外，'='的位置嵌入也是固定不变的。因此，$A_0^j = \sigma\left({x_2^{(0)}}^T W_Q^{j^T} W_K (x_0^{(0)} - x_1^{(0)})\right)$（以及 $A_1^j =$ $1 - A_0^j$）的值也同样保持不变。

第二个观察结果所带来的一个结论是，我们记为$W_L = W_U W_{out}$的Logits $\approx W_U W_{out}$MLP实际上是一个有意义的矩阵，而非由其各组成部分构成的简单线性组合，因为这些组成部分只是以线性的方式相互叠加。

B 为何要使用构造性干扰?

如第4 节以及附录C.2.1中所展示的，针对该任务训练的小型变换器会使用多个不同的频率，并将这些频率的信号相加。这样做的目的就是得到一个函数，使其在 $x = 0 \mod 113$处的函数值显著大于其他任何位置上的函数值。

例如，考虑函数 $f_{14}(x) = \cos\left(\frac{2\pi \cdot 14}{113}x\right)$。该函数的周期为113，在 $x = 0 \mod 113$ 处取得最大值。不过，当 x 取其他数值时，该函数的值会接近1，比如 $f_{14}(8) = f_{14}(105) = 0.998$、$f_{14}(16) = f_{14}(89) = 0.994$ 等。

再来看函数 $f_{35}(x) = \cos\left(\frac{2\pi \cdot 35}{113}x\right)$。虽然它的周期同样是113，且在 $x = 0 \mod 113$ 处达到最大值，但实际情况下 $f_{35}(8) = f_{35}(105) = -0.990$。这意味着，如果将 f_{14} 与 f_{35}相加，得到的函数在 $x = 8 \mod 113$ 处的值就不会再接近1。类似地，$f_{35}(16) = 0.961$、$f_{52}(16) = -0.56$，因此加入第三个频率后，$x = 16 \mod 113$ 处的峰值也会随之降低。

我们在图8中展示了主线模型所使用的五种频率对应的余弦波所产生的建设性干涉现象

用于模块化算术网络机制分析的C语言支撑证据

C.1 对本文中讨论的特定训练轮次的进一步分析

在本节中，我们将提供与主线模型相关的更多证据。

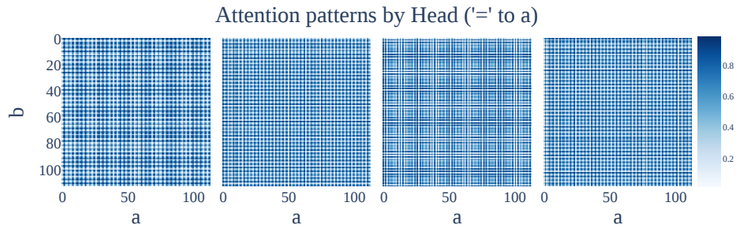


Figure 9: Attention patterns for each head, from the '=' token at the third sequence position to the a token at the first sequence position, as a heatmap over the inputs. All four attention heads exhibit striking periodicity.

Head	k	α^j	β^j	FVE
0	35	-0.26	-0.14	99.03%
1	42	0.27	-0.04	98.49%
2	52	0.29	-0.05	99.07%
3	42	-0.26	0.04	97.91%

Table 2: For each attention head, we show the pattern from '=' to a is well approximated by $0.5 + \alpha(\cos(w_k a) - \cos(w_k b)) + \beta(\sin(w_k a) - \sin(w_k b))$ and give the coefficients and fraction of variance explained for this approximation.

C.1.1 Periodicity in the activations of other attention heads

In Figure 9 we plot the attention patterns from the final token '=' to the first token a for all 4 attention heads, as a heatmap over the inputs a and b, as this is a scalar for each head. We observe a striking periodicity and further that heads 1 and 3 represent the same frequency while heads 0 and 2 are different.

As shown in Appendix A.1, the attention paid from '=' to itself is negligible, so $A_0^j = 1 - A_1^j$ and it suffices to plot attention to a.

C.1.2 Approximating attention heads with Sines and Cosines

Attention heads approximately compute degree-2 polynomials of a single frequency or are used to amplify W_E. In order to compute terms like $\cos(w_k(a+b))$, the model needs to compute the product of the sine and cosine embeddings output by W_E. As the attention heads are approximately bilinear (product of attention weights and OV circuit), they are a natural place to perform this computation. Indeed, for each head, the attention scores' Fourier transform is concentrated on a single frequency w_k. For two of the four heads, the corresponding OV circuit is concentrated on that same frequency. Moreover, the softmax mapping the attention scores to attention weights is in a regime where it behaves approximately linearly (and replacing it with a linear function actually improves performance). Thus the attention weights multiply with the OV output to create degree-2 polynomials of the frequency w_k, as would be needed for the cosine/sine addition formulas.

For the remaining two heads, their attention scores approximately sum to one and the OV circuits contain all five key frequencies, suggesting that they are used to increase the magnitude of key frequencies in the residual stream. We confirm all of these claims in Appendix C.1.3.

C.1.3 The attention pattern weights are well approximated by differences of sines and cosines of a single frequency.

The periodicity of the attention heads has a striking form—A_0^j is well approximated by $0.5 + \alpha^j(\cos(w_k a) - \cos(w_k b)) + \beta^j(\sin(w_k a) - \sin(w_k b))$, for some frequency w_k and constants α^j and β^j (which may differ for each head). Note further that this simplifies to $0.5 + \gamma(\cos(w_k(a + \theta)) - \cos(w_k(b + \theta)))$ for some constants γ and θ. We show the coefficients and fraction of variance explained in Table 1

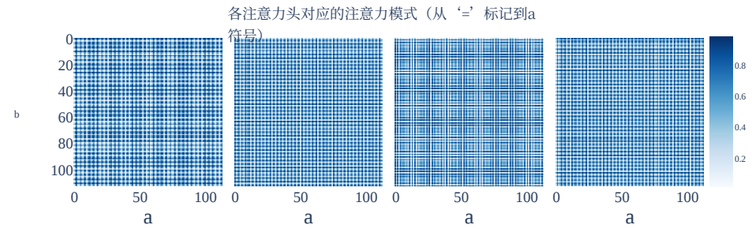


图9：展示了每个注意力头对应的注意力模式，范围涵盖从第三个序列位置处的‘=’标记到第一个序列位置处的a 标记，这些模式以热图形式呈现在输入值上。所有四个注意力头均呈现出显著的周期性。

Head	k	α^j	β^j	FVE
0	35	-0.26	-0.14	99.03%
1	42	0.27	-0.04	98.49%
2	52	0.29	-0.05	99.07%
3	42	-0.26	0.04	97<span style id='1'>。91%

表2：针对每一个注意力头，我们展示了从‘=’到 a 的分布能够通过 $0.5 + \alpha(\cos(w_k a) - \cos(w_k b)) + \beta(\sin(w_k a) - \sin(w_k b))$很好地近似表示，并给出了该近似方法的系数以及方差解释率。

C.1.1 其他注意力头的激活值的周期性

在图9 中，我们针对全部4个注意力头，将从最后一个标记‘=’到第一个标记 a 的注意力模式以热图形式展示在输入值 a 和 b上，因为每个注意力头对应的数值均为标量。我们观察到了显著的周期性，此外头部1和头部3呈现出相同的频率，而头部0和头部2的频率则有所不同。

如附录A.1中所示，从‘=’指向其自身的注意力几乎可以忽略，因此$A_0^j = 1 - A_1^j$，只需绘制其对 a 的注意力分布即可。

C.1.2 用正弦函数与余弦函数近似表示注意力头

注意力头大致会计算单一频率的二次多项式，或者被用来放大 W_E。为了计算诸如cos(())这样的项，模型需要计算由输出的正弦嵌入向量与余弦嵌入向量的乘积。由于注意力头本质上是近似的双线性结构（即注意力权重与OV电路结构的乘积），因此它们非常适合执行此类计算。实际上，对于每个注意力头，其注意力分数的傅里叶变换都集中在单一频率上。在四个注意力头中有两个，对应的OV电路也集中在同一频率上。此外，将注意力分数映射为注意力权重的softmax函数在特定范围内表现出近似线性的特性（用线性函数替代它实际上还能提升性能）。这样一来，注意力权重就会与OV电路的输出相乘，从而生成频率为的二次多项式，这正是应用余弦/正弦加法公式时所必需的。

对于剩下的两个注意力头而言，它们的注意力分数之和大致为1，且这些OV电路结构中包含了全部五个关键频率，这表明它们用于提升残差流中关键频率的幅度。我们在附录C.1.3中对这些结论进行了验证。

C.1.3 注意力模式权重可以通过单一频率的正弦函数与余弦函数的差值来很好地近似表示。

注意力头的周期性呈现出一种极为显著的规律——A_0^j 其值可以很好地通过 $0.5 + \alpha^j(\cos(w_k a) - \cos(w_k b)) + \beta^j(\sin(w_k a) - \sin(w_k b))$ 来近似表示，其中α^j (而这些参数对于不同的注意力头可能各不相同)。此外，该表达式还可进一步简化为 $0.5 + \gamma(\cos(w_k(a+\theta)) - \cos(w_k(b+\theta)))$，其中同样涉及某些常数 γ以及 θ。相关的系数与方差解释率详见表 1

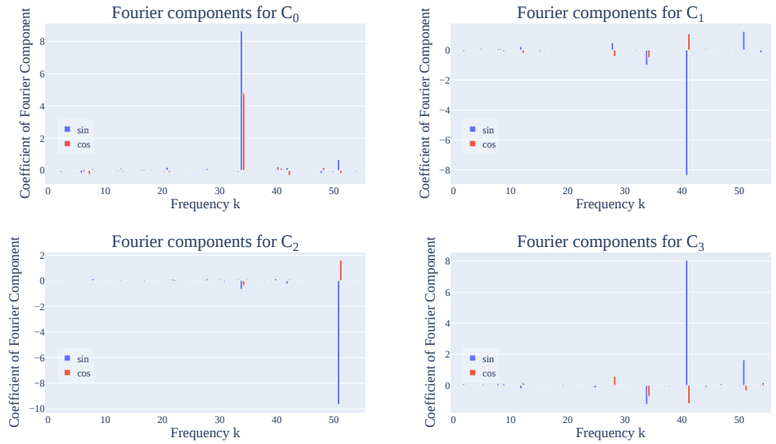


Figure 10: We plot the attention pattern weights C_j in the Fourier basis for each of the four heads $j \in \{0, 1, 2, 3\}$. We observe significant sparsity, with almost all of each term being associated with a single frequency.

Mechanistic Analysis of Attention Patterns. We can further mechanistically analyse *how* the model achieves this form. The following is a high-level sketch of what is going on:

First, note that the attention score on position 0 and head j is just a lookup table on the input token a (of size P). To see why, note that $A_0^j = m{x_0^{(0)}}^T W_K^{j^T} W_Q^j x_2^{(0)}$. $x_2^{(0)}$ is constant since the token is always '=' and $x_0^{(0)} = W_E t_0 + p_0$. So this reduces to $t_0 \cdot C_j + D$ for some constant vector $C_j = W_E^T W_K^{j^T} W_Q^j x_2^{(0)} \in \mathbb{R}^p$ and some scalar $D = p_0^T {W_K^j}^T W_Q^j x_2^{(0)}$. As t_0 is one-hot encoded, this is just a lookup table, which we may instead denote as $C_j[a]$

Next, note that the attention pattern from $=\!\!\rightarrow 0$ is $\sigma(C_j[a] - C_j[b])$. As argued in Appendix A.1, the attention paid $=\!\!\rightarrow\!=$ is negligible and can be ignored. So the softmax reduces to a softmax over two elements, which is a sigmoid on their difference. As form of C_j does not mention the token index or value, it is the same for position 0 and 1.

We now show that C_j is well-approximated by a wave of frequency w_{k_j} for some integer k_j. That is, $C_j[a] \approx F_j \cos(w_{k_j} a) + G_j \sin(w_{k_j} a)$. We do this by simply computing C_j and fitting the constants F_j and G_j to minimize ℓ_2 loss, and display the resulting coefficients for each head in Figure 10. This fit explain 99.02%, 95.21%, 99.10%, 92.42% of the variance of C_j respectively. Interestingly, the coefficients of heads 1 and 3 are almost exactly the opposite of each other.

For each head j, $\sigma(C_j[a] - C_j[b]) \approx 0.5 + E_j(C_j[a] - C_j[b])$ for some constant E_j—that is, the sigmoid has some linear approximation. (The intercept will be 0.5 by symmetry.) The striking thing is that, because the inputs to the sigmoid for the attention heads are over a fairly wide range ($[-5, 5]$ roughly), the linear approximation to the sigmoid is a fairly good fit, explaining 97.5% of the variance.

We validate that this is all that is going on, by replacing the sigmoid with the best linear fit. This *improves* performance, decreasing test loss from $2.41 \cdot 10^{-7}$ to $2.12 \cdot 10^{-7}$.

By properties of sinusoidal functions, the attention patterns of each head will be well approximated by $0.5 \pm C_j(\cos(w_{k_j}(a + \theta_j)) - \cos(w_{k_j}(b + \theta_j)))$ - the softmax is linear, with an intercept of 0.5, and the weights C_j map each token to a score that is a wave in a single frequency. This exactly gives us the periodic form shown in Figure 9.

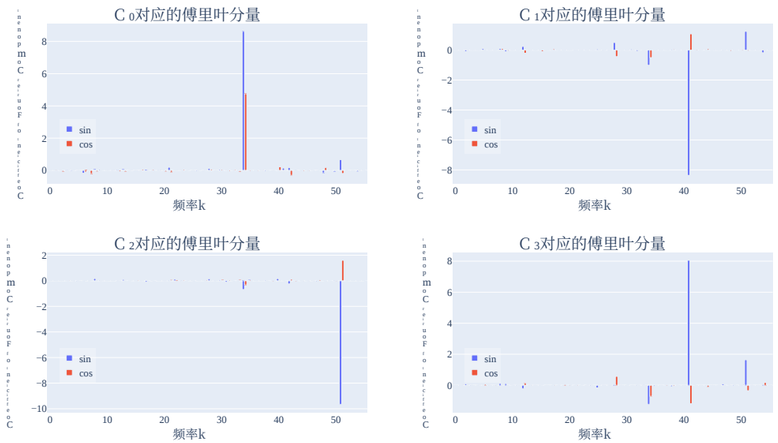


图10：我们绘制了四个注意力头在傅里叶基下的注意力模式权重 C_j 。可以观察到明显的稀疏性，几乎每个项都仅与一个频率相关联。

Me注意力模式的机制性分析。我们还可以进一步从机制层面分析 其形成方式 究竟如何 e model能够呈现出这样的形式。以下是对其中原理的高层概述：

首先需要注意的是，位置0和注意力头 j 的注意力分数实际上只是针对输入标记a（其大小为 P）构建的查找表。之所以如此，是因为 $A_0^j = m{x_0^{(0)}}^T {W_K^j}^T W_Q^j x_2^{(0)}$，且由于该标记始终为‘=’， $x_2^{(0)}$是常数，进而 $x_0^{(0)} = W_E t_0 + p_0$。因此这可以简化为某个常量向量$C_j = W_E^T {W_K^j}^T W_Q^j x_2^{(0)} \in \mathbb{R}^p$ 与某个标量 $D = p_0^T {W_K^j}^T W_Q^j x_2^{(0)}$ 构成的 $t_0 \cdot C_j + D$ 。由于 t_0 是经过独热编码的，它同样只是一种查找表，我们也可以将其表示为 $C_j[a]$。

接下来需要说明的是，$=\!\!\rightarrow 0$ 对应的注意力模式为 $\sigma(C_j[a] - C_j[b])$。如附录A.1中所论述的，所分配的 $=\!\!\rightarrow\!=$ 注意力值可以忽略不计，因此可直接舍去。这样一来，softmax运算就简化为对两个元素的softmax计算，进而等价于对其差值应用sigmoid函数。由于 C_j 并未提及标记的索引或具体数值，所以位置0 和1的情况是相同的。

现在我们将证明， C_j可以通过频率为w_{k_j}的正弦波来很好地近似表示，其中 k_j为某个整数。也就是说，$C_j[a] \approx F_j \cos(w_{k_j} a) + G_j \sin(w_{k_j} a)$。实现这一点的办法是直接计算 C_j，并调整常数F_j 和 G_j，以最小化 ℓ_2 损失值，随后在10图中展示每个注意力头对应的系数。这样的拟合分别能够解释 C_j 方差中的99.02%、95.21%、99.10%和92.42%。有趣的是，注意力头1 和3 的系数几乎完全互为相反数。

对于每一个注意力头 j, $\sigma(C_j[a] - C_j[b])$ ， 其对应的 $\approx 0.5 + E_j(C_j[a] - C_j[b])$ 均满足某个常数 E_j 的关系——也就是说，Sigmoid函数存在某种线性近似形式。（由于对称性，截距将为0。5。）值得关注的是，由于注意力头的输入值覆盖的范围相当宽（大致在$[-5, 5]$ 之间），这种对Sigmoid函数的线性近似效果相当不错，能够解释97.5%的方差。

我们通过用最佳线性拟合模型替代Sigmoid函数，来验证实际上仅存在这种机制。这样做提升了模型性能，使得测试损失从 $2.41 \cdot 10^{-7}$ 下降到了 $2.12 \cdot 10^{-7}$。

根据正弦函数的特性，每个注意力头的注意力模式可以通过 $0.5 \pm C_j(\cos(w_{k_j}(a + \theta_j)) - \cos(w_{k_j}(b + \theta_j)))$ ——由于softmax函数为线性函数且截距为0．5，而那些权重 C_j 会将每个标记映射为一个仅具有单一频率的波形形式的分数。这正好对应着图9中所展示的周期性特征。

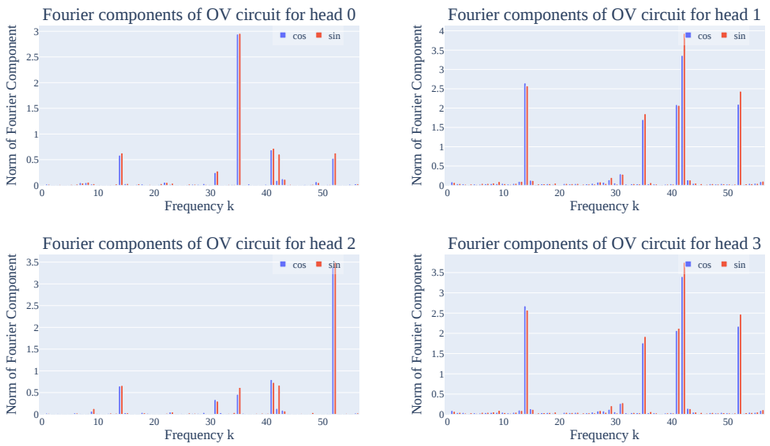


Figure 11: We plot the output of the OV circuit $W_O^j W_V^j x^{(0)}$ in the Fourier basis for each of the four heads $j \in \{0, 1, 2, 3\}$. As with the attention pattern weights C_j in Figure 10, we observe that the only components with significant norm are those corresponding to key frequencies, and that the largest component corresponds to the frequencies of the attention patterns of the attention heads. As attention pattern of heads 1 and 3 are sum to one, but their OV circuits are almost exactly the same and consist of all five key frequencies, this implies that heads 1 and 3 are used to increase the magnitude of key frequencies in the residual stream (Section C.1.3).

Finally, for each head j, we plot the output of the OV circuit $W_O^j W_V^j x^{(0)}$ in the Fourier basis and display the results in Figure 11). The largest component of each head corresponding to the frequency of the attention pattern C_j, with heads 0 and 2 being almost entirely composed of a sines and cosines of a single frequency. On the other hand, the norms for the components of heads 1 and 3 are almost exactly the same, and contain all five key frequencies. As the coefficients of the attention pattern weights have the opposite non-constant components (Table 2, Figure 10), their attention scores sum almost exactly to 1 across all inputs. This implies that heads 1 and 3 are used to output the first order terms $\sin(w_k), \cos(w_k)$ in the five key frequencies. We speculate that this is because of weight decay encouraging the embeddings W_E to be small, causing the network to allocate two of its attention heads to effectively increasing the size of W_E.

Bringing it all together, this implies that attention heads 0 and 2 are approximately computing a degree 2 polynomial of cosines and sines of a single frequency each, while heads 1 and 3 amplify the key frequencies in the residual stream.

C.1.4 Periodicity in the activations of additional neurons

In Figure 12, we display the activations of four more MLP neurons, as a function of the inputs. As with neuron 0, the activations of these neurons are also periodic in the inputs.

C.1.5 Additional grokking figures for mainline run

In Figure 13, we display the accuracy of the model when restricting the model to use only the five key frequencies. As with restricted loss, this *improves* model performance during training.

In Figure 14, we show the coefficients of the five key frequencies in the logits, calculated by regressing the logits against the five $\cos(w_k(a+b-c))$ terms.

In Figure 15, we plot the excluded loss if we exclude each of the five key frequencies (as opposed to all five key frequencies).

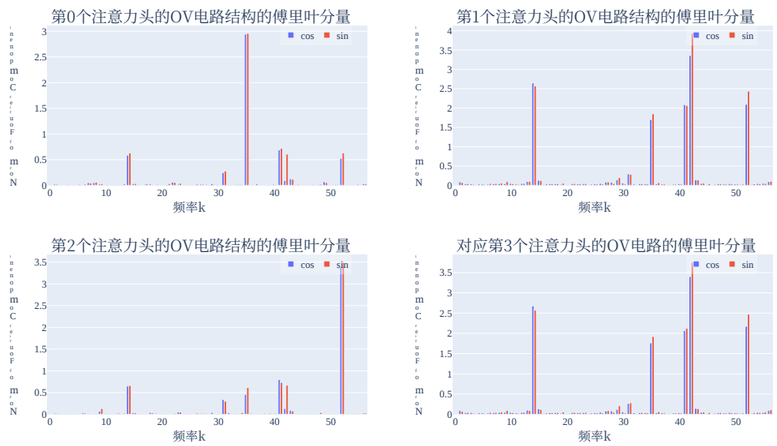


图11：我们展示了四个注意力头对应的OV电路在傅里叶基下的输出结果 $W_O^j W_V^j x^{(0)}$。 $j \in \{0, 1, 2, 3\}$。与图10中的注意力模式权重一样，我们发现只有那些对应于关键频率的分量才具有较大的范数，而且幅度最大的分量正是注意力头的注意力模式所对应的频率。由于注意力头1 和3 的注意力模式之和为1，且它们的OV电路几乎完全相同，都包含全部五个关键频率，这表明注意力头1 和3 的作用是提升残差流中关键频率的幅值（详见C.1.3节）。

最后，对于每一个注意力头 j，我们都会在傅里叶基下绘制OV电路 $W_O^j W_V^j x^{(0)}$ 的输出结果，并将相关内容展示在图11中。每个注意力头中与注意力模式频率对应的最大分量 C_j，其中第0号和第2号注意力头几乎完全由单一频率的正弦函数与余弦函数构成。另一方面，第1 号以及3 号注意力头各分量的范数几乎完全相同，且都包含了全部五个关键频率。由于注意力模式权重的系数具有相反的非恒定分量（见表2、图10），因此这些注意力头的注意力分数在所有输入上的求和几乎恰好为1。这说明第1 号和3 号注意力头用于输出五个关键频率范围内的第一阶项sin（w_k）、cos（w_k）。我们推测，这是因为权重衰减机制使得嵌入向量 W_E 的数值保持较小，从而导致网络分配两个注意力头来有效提升其规模 W_E。

综合以上分析可以得出，注意力头0和2大致是在计算单个频率的正弦函数与余弦函数的二次多项式，而注意力头1和3则负责放大残差流中的关键频率。

C.1.4 其他神经元的激活值中的周期性

在图12中，我们展示了另外四个多层感知机神经元的激活值随输入值的变化情况。与神经元0一样，这些神经元的激活值也呈现出随输入值变化的周期性。

C.1.5 线性运行过程中的额外理解能力提升相关图表

在图13中，我们展示了在限制模型仅使用五个关键频率时的模型准确率。与受限损失的情况类似，这一措施能够提升训练过程中的模型性能。

In F图14中，我们通过回归分析展示了逻辑值中对应这五个关键频率的系数。 -
ing 这些系数是针对由五个余弦函数（$\cos(w_k(a+b-c))$）项构成的逻辑值计算得出的。

在图 15中，我们展示了在排除五个关键频率中的每一个而非全部五个关键频率时的排除损失值。

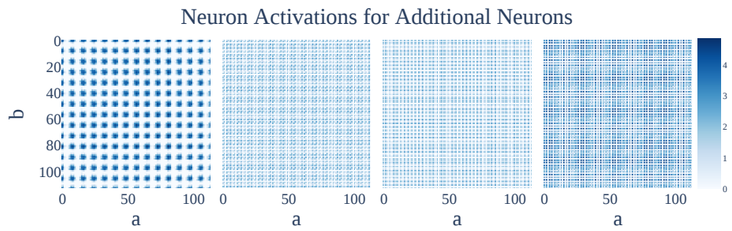


Figure 12: Plots of neuron activations for MLP neurons 1, 2, 3 and 4, for inputs $a, b \in \{0, 1, ..., 112\}$. As with Neuron 0, all of the activation patterns are periodic in both inputs.

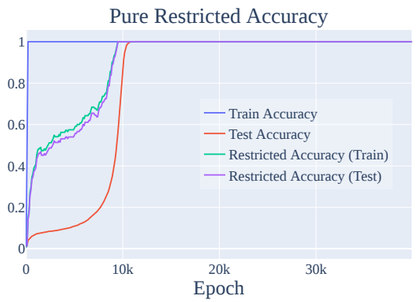


Figure 13: Accuracy when restricting Fourier Components to the five key frequencies. As with restricted loss, this shows that the model figures out how to generalize modulo deleting noise before it removes the noise.

All three of these figures have inflection points corresponding to the relevant phases of grokking, discussed in Section 5.1.

C.2 Additional results from different runs

In this section, we plot relevant figures from other runs, either with the same architecture (Appendix C.2.1) or with different architectures or experimental setups (Appendix C.2.2). Note that in general, while all models learn to use variants of the modular arithmetic algorithm, they use a varying number of *different* key frequencies. In order to find the key frequencies to calculate the excluded and restricted loss, we perform a DFT on the neuron-logit map W_L, then take the frequencies with nontrivial coefficients.[3]

C.2.1 Additional results for different runs with the same architecture

In this section, we provide evidence that all 4 other runs (i.e., random seeds) using the experimental setup of our mainline model also use the Fourier multiplication algorithm, and then confirm that the same phases of grokking also occur on these runs.

[3]One method for getting a general (model-independent) progress measure for this task is to compute the excluded loss for each of the 56 unique frequencies and then take the max. We omit the plots for this variant of the excluded loss as they are broadly similar.

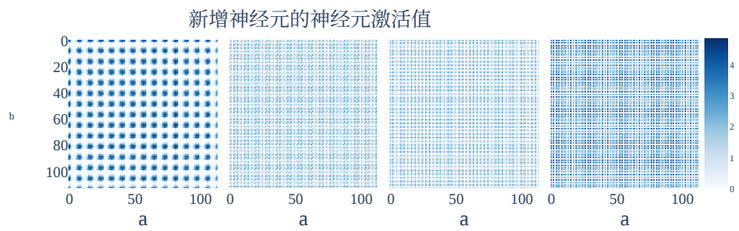


图12：展示了针对输入值 a， $b \in \{0, 1, ..., 112\}$时，MLP神经元1、2、3和4的神经元激活值曲线。与神经元0的情况类似，所有这些神经元的激活模式在两种输入条件下均为周期性变化。

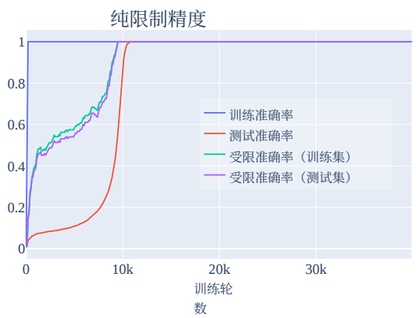


图13：将傅里叶分量限制在五个关键频率时的准确率。与受限损失的情况类似，该结果表明模型在执行模去噪声操作之前，就已经掌握了如何实现泛化。

这三幅图均存在拐点，这些拐点分别对应着在第 5.1节中所讨论的理解能力提升的不同阶段。

C.2 不同运行次数的额外结果

在本节中，我们展示了来自其他实验运行的相关图表，这些实验运行要么采用相同的架构（见附录C.2.1），要么采用不同的架构或实验设置（见附录C.2.2）。需要注意的是，虽然所有模型都被训练为使用模块化算术算法的多种变体，但它们所使用的不同关键频率的数量各不相同。为了找出用于计算排除损失和受限损失的关键频率，我们对神经元逻辑映射 W_L进行离散傅里叶变换，[3]>，然后提取出那些系数非零的频率。

C.2.1 使用相同架构进行不同实验运行后的额外结果

在本节中，我们提供了证据表明，使用我们主线模型实验设置的另外4次实验运行（即不同的随机种子）同样采用了傅里叶乘法算法，进而证实这些运行中也出现了相同的理解能力提升现象。

[3]为该任务获取一种通用的、与模型无关的进展度量指标，一种方法是分别计算56个不同频率对应的排除损失值，再取其中的最大值。由于这类排除损失值的图表形态大致相似，因此此处不再展示。

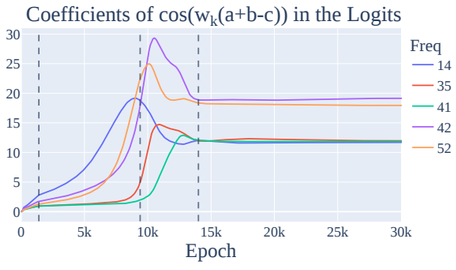


Figure 14: The coefficients of $\cos(w(a+b-c))$ in the logits over the model's training. As with the metrics in the paper, this shows a nice interpolation and growth of each cosine term.

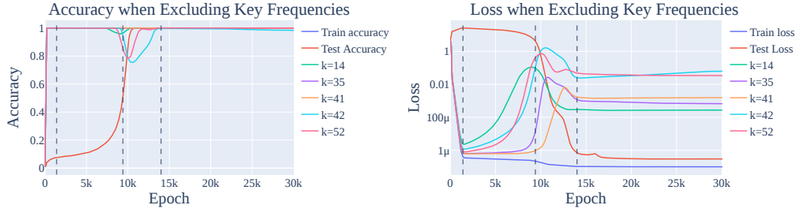


Figure 15: The excluded accuracy (left) and loss (right) if we exclude each of the five key frequencies for our mainline model. As with the excluded loss results in Section 5.1, this shows that the model interpolates between memorising and generalising.

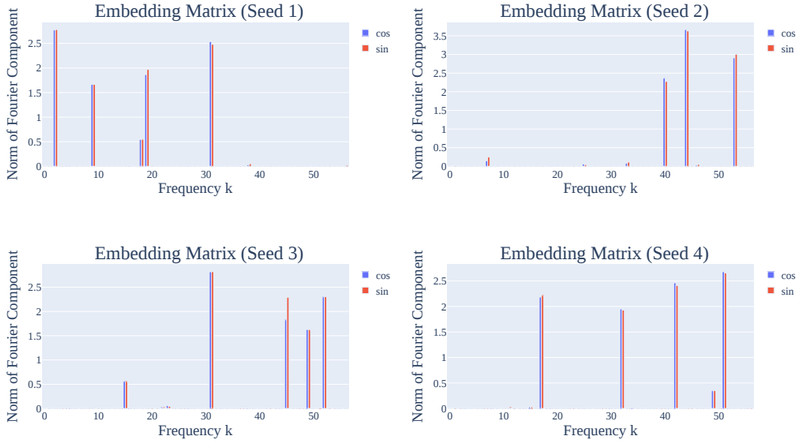


Figure 16: The norms of the Fourier components in the embedding matrix W_E for each of four other random seeds for the original (1 layer) architecture. As discussed in Section 4.1 and Appendix C.2.1, the sparsity of W_E in the Fourier basis is evidence that the network is operating in a Fourier basis.

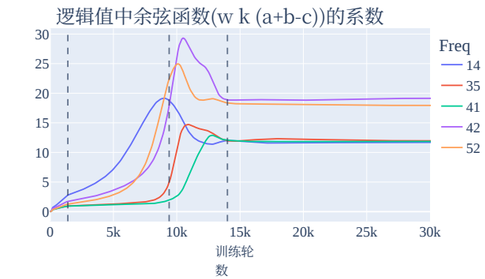


图15：展示在模型训练过程中，logits中$\cos(w(a+b-c))$各项系数的变化情况。与论文中的其他指标类似，该图清晰地显示出各个余弦项的平滑插值变化及持续增长趋势。

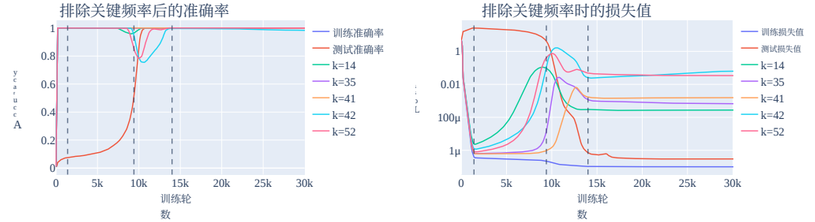


图15：若排除主线模型所对应的五个关键频率，对应的排除准确率（左侧）与损失值（右侧）变化情况。与第5.1节中的排除损失结果一致，该图表明该模型在记忆行为与泛化能力之间不断进行权衡。

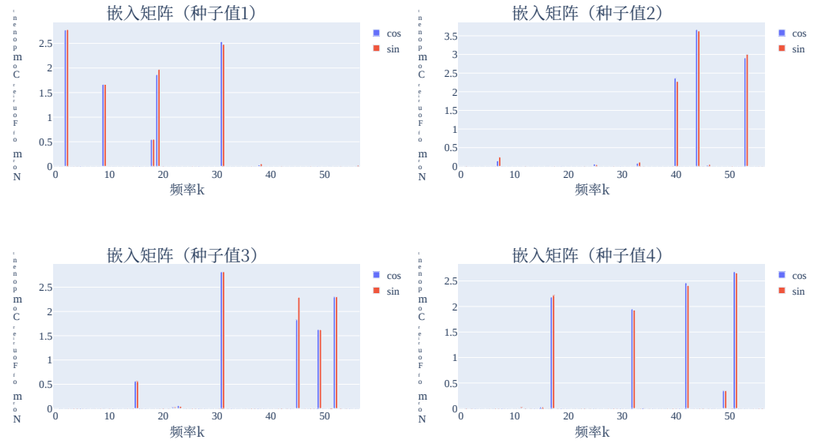


图16：针对原始（1层结构）架构的另外四种不同随机种子，其嵌入矩阵中傅里叶分量的范数变化情况。 W_E 如第4.1 节、附录以及C.2.1中所讨论的，傅里叶基中 W_E 的稀疏性正是该网络基于傅里叶基运行的重要证据。

W_L Component	Fourier components of u_k^TMLP(a,b) or v_k^TMLP(a,b)	FVE
$\cos(w_2c)$	$147.4\cos(w_2a)\cos(w_2b) - 145.8\sin(w_2a)\sin(w_2b) \approx 146.6\cos(w_2(a+b))$	99.2%
$\sin(w_2c)$	$145.5\cos(w_2a)\sin(w_2b) + 145.6\sin(w_2a)\cos(w_2b) \approx 145.5\sin(w_2(a+b))$	99.1%
$\cos(w_9c)$	$49.3\cos(w_9a)\cos(w_9b) - 48.0\sin(w_9a)\sin(w_9b) \approx 48.6\cos(w_9(a+b))$	96.4%
$\sin(w_9c)$	$48.6\cos(w_9a)\sin(w_9b) + 48.5\sin(w_9a)\cos(w_9b) \approx 48.5\sin(w_9(a+b))$	96.7%
$\cos(w_{19}c)$	$58.0\cos(w_{19}a)\cos(w_{19}b) - 58.3\sin(w_{19}a)\sin(w_{19}b) \approx 58.2\cos(w_{19}(a+b))$	95.4%
$\sin(w_{19}c)$	$59.3\cos(w_{19}a)\sin(w_{19}b) + 59.4\sin(w_{19}a)\cos(w_{19}b) \approx 59.4\sin(w_{19}(a+b))$	93.9%
$\cos(w_{31}c)$	$94.4\cos(w_{31}a)\cos(w_{31}b) - 96.4\sin(w_{31}a)\sin(w_{31}b) \approx 95.4\cos(w_{31}(a+b))$	98.4%
$\sin(w_{31}c)$	$97.2\cos(w_{31}a)\sin(w_{31}b) + 97.1\sin(w_{31}a)\cos(w_{31}b) \approx 97.2\sin(w_{31}(a+b))$	98.7%

(a) Seed 1

W_L Component	Fourier components of u_k^TMLP(a,b) or v_k^TMLP(a,b)	FVE
$\cos(w_{40}c)$	$97.0\cos(w_{40}a)\cos(w_{40}b) - 99.4\sin(w_{40}a)\sin(w_{40}b) \approx 98.2\cos(w_{40}(a+b))$	97.3%
$\sin(w_{40}c)$	$81.3\cos(w_{40}a)\sin(w_{40}b) + 81.3\sin(w_{40}a)\cos(w_{40}b) \approx 81.3\sin(w_{40}(a+b))$	92.7%
$\cos(w_{44}c)$	$309.1\cos(w_{44}a)\cos(w_{44}b) - 338.7\sin(w_{44}a)\sin(w_{44}b) \approx 323.9\cos(w_{44}(a+b))$	98.5%
$\sin(w_{44}c)$	$327.3\cos(w_{44}a)\sin(w_{44}b) + 327.2\sin(w_{44}a)\cos(w_{44}b) \approx 327.3\sin(w_{44}(a+b))$	98.9%
$\cos(w_{53}c)$	$192.1\cos(w_{53}a)\cos(w_{53}b) - 192.2\sin(w_{53}a)\sin(w_{53}b) \approx 192.1\cos(w_{53}(a+b))$	97.3%
$\sin(w_{53}c)$	$166.7\cos(w_{53}a)\sin(w_{53}b) + 166.8\sin(w_{53}a)\cos(w_{53}b) \approx 166.8\sin(w_{53}(a+b))$	95.7%

(b) Seed 2

W_L Component	Fourier components of u_k^TMLP(a,b) or v_k^TMLP(a,b)	FVE
$\cos(w_{31}c)$	$156.1\cos(w_{31}a)\cos(w_{31}b) - 156.5\sin(w_{31}a)\sin(w_{31}b) \approx 156.3\cos(w_{31}(a+b))$	99.3%
$\sin(w_{31}c)$	$150.7\cos(w_{31}a)\sin(w_{31}b) + 150.7\sin(w_{31}a)\cos(w_{31}b) \approx 150.7\sin(w_{31}(a+b))$	98.9%
$\cos(w_{45}c)$	$72.5\cos(w_{45}a)\cos(w_{45}b) - 76.8\sin(w_{45}a)\sin(w_{45}b) \approx 74.6\cos(w_{45}(a+b))$	95.9%
$\sin(w_{45}c)$	$74.7\cos(w_{45}a)\sin(w_{45}b) + 74.6\sin(w_{45}a)\cos(w_{45}b) \approx 74.6\sin(w_{45}(a+b))$	96.6%
$\cos(w_{49}c)$	$45.9\cos(w_{49}a)\cos(w_{49}b) - 45.5\sin(w_{49}a)\sin(w_{49}b) \approx 45.7\cos(w_{49}(a+b))$	97.0%
$\sin(w_{49}c)$	$45.8\cos(w_{49}a)\sin(w_{49}b) + 45.8\sin(w_{49}a)\cos(w_{49}b) \approx 45.8\sin(w_{49}(a+b))$	96.9%
$\cos(w_{52}c)$	$71.6\cos(w_{52}a)\cos(w_{52}b) - 72.1\sin(w_{52}a)\sin(w_{52}b) \approx 71.9\cos(w_{52}(a+b))$	98.5%
$\sin(w_{52}c)$	$68.7\cos(w_{52}a)\sin(w_{52}b) + 68.7\sin(w_{52}a)\cos(w_{52}b) \approx 68.7\sin(w_{52}(a+b))$	97.9%

(c) Seed 3

W_L Component	Fourier components of u_k^TMLP(a,b) or v_k^TMLP(a,b)	FVE
$\cos(w_{17}c)$	$66.0\cos(w_{17}a)\cos(w_{17}b) - 63.5\sin(w_{17}a)\sin(w_{17}b) \approx 64.8\cos(w_{17}(a+b))$	96.4%
$\sin(w_{17}c)$	$66.4\cos(w_{17}a)\sin(w_{17}b) + 66.4\sin(w_{17}a)\cos(w_{17}b) \approx 66.4\sin(w_{17}(a+b))$	94.9%
$\cos(w_{32}c)$	$68.7\cos(w_{32}a)\cos(w_{32}b) - 68.4\sin(w_{32}a)\sin(w_{32}b) \approx 68.5\cos(w_{32}(a+b))$	96.2%
$\sin(w_{32}c)$	$68.0\cos(w_{32}a)\sin(w_{32}b) + 68.0\sin(w_{32}a)\cos(w_{32}b) \approx 68.0\sin(w_{32}(a+b))$	96.3%
$\cos(w_{42}c)$	$100.4\cos(w_{42}a)\cos(w_{42}b) - 96.0\sin(w_{42}a)\sin(w_{42}b) \approx 98.2\cos(w_{42}(a+b))$	97.9%
$\sin(w_{42}c)$	$100.2\cos(w_{42}a)\sin(w_{42}b) + 100.1\sin(w_{42}a)\cos(w_{42}b) \approx 100.1\sin(w_{42}(a+b))$	98.6%
$\cos(w_{51}c)$	$118.0\cos(w_{51}a)\cos(w_{51}b) - 116.2\sin(w_{51}a)\sin(w_{51}b) \approx 117.1\cos(w_{51}(a+b))$	99.0%
$\sin(w_{51}c)$	$114.3\cos(w_{51}a)\sin(w_{51}b) + 114.2\sin(w_{51}a)\cos(w_{51}b) \approx 114.2\sin(w_{51}(a+b))$	98.5%

(d) Seed 4

Table 3: For each of the directions in the neuron-logit map W_L of the final models from 4 other random seeds (Appendix C.2.1), we project the MLP activations in that direction then perform a Fourier transform. For brevity, we omit terms with coefficients less than 15% of the largest coefficient. We then compute the fraction of variance explained (FVE) if we replace the projection with a multiple of a single term of the form $\cos(w_k(a+b))$ or $\sin(w_k(a+b))$, and find that this is consistently close to 1.

W_L 分量	u_k^T多层感知机$(a,\ b)$或 v_k^T多层感知机$(a,\ b)$的傅里叶分量	FVE
余弦函数(w_2c)	147。4 乘以余弦函数(w_2a)再乘以余弦函数(w_2b)，再加上 − 145.8正弦函数(w_2a)乘以正弦函数(w_2b)，最后乘以 ≈ 146.6余弦函数$(w_2(a+b))$	99。2%
$\sin(w_2c)$ 余弦函数(w_9c)	145<span style='id':'1'>.5 × 余弦函数(w_2a) × 正弦函数(w_2b) + + 145.6×正弦函数(w_2a)×余弦函数(w_2b) + ≈ 145.5×正弦函数$(w_2(a+b))$	99<span style='id':'1'>.1%
$\sin(w_9c)$ 余弦函数$(w_{19}c)$	49<span style='id':'1'>.3 × 余弦函数(w_9a) × 余弦函数(w_9b) + − 48.0 ×正弦函数(w_9a)×正弦函数(w_9b) + ≈ 48.6 ×余弦函数$(w_9(a+b))$	96<span style='id':'1'>.4%
$\sin(w_{19}c)$ 余弦函数$(w_{31}c)$	48<span style='id':'1'>.6 × 余弦函数(w_9a) × 正弦函数(w_9b) + + 48.5×正弦函数(w_9a)×余弦函数(w_9b) + ≈ 48.5×正弦函数$(w_9(a+b))$	96<span style='id':'1'>.7%
正弦函数$(w_{31}c)$	58<span style='id':'1'>.0 × 余弦函数$(w_{19}a)$ × 余弦函数$(w_{19}b)$ + − 58.3 ×正弦函数$(w_{19}a)$×正弦函数$(w_{19}b)$ + ≈ 58.2×余弦函数$(w_{19}(a+b))$	95<span style='id':'1'>.4%
	59<span style='id':'1'>.3 × 余弦函数$(w_{19}a)$ × 正弦函数$(w_{19}b)$ + + 59.4×正弦函数$(w_{19}a)$×余弦函数$(w_{19}b)$ + ≈ 59.4 ×正弦函数$(w_{19}(a+b))$	93<span style='id':'1'>.9%
	94<span style='id':'1'>.4 × 余弦函数$(w_{31}a)$ × 余弦函数$(w_{31}b)$ + − 96.4 ×正弦函数$(w_{31}a)$×正弦函数$(w_{31}b)$ + ≈ 95.4×余弦函数$(w_{31}(a+b))$	98<span style='id':'1'>.4%
	97<span style='id':'1'>。2 乘以余弦函数$(w_{31}a)$再乘以正弦函数$(w_{31}b)$，再加上 + 97.1乘以正弦函数$(w_{31}a)$再乘以余弦函数$(w_{31}b)$，最后加上 ≈ 97.2 乘以正弦函数$(w_{31}(a+b))$	98<span style='id':'1'>。7%

(a) 种子值1

W_L 组件	u_k^T多层感知机$(a<$<span style='id':'3'>，$b)$或 v_k^T多层感知机$(a<$<span style='id':'8'>，$b)$的傅里叶分量	FVE
余弦函数$(w_{40}c)$	97<span style='id':'1'>.0 乘以余弦函数$(w_{40}a)$再乘以余弦函数$(w_{40}b)$，再加上 − 99.4 乘以正弦函数$(w_{40}a)$再乘以正弦函数$(w_{40}b)$，最后加上 ≈ 98.2 乘以余弦函数$(w_{40}(a+b))$	97<span style='id':'1'>.3%
正弦函数$(w_{40}c)$	81<span style='id':'1'>.3 乘以余弦函数$(w_{40}a)$再乘以正弦函数$(w_{40}b)$，再加上 + 81.3乘以正弦函数$(w_{40}a)$再乘以余弦函数$(w_{40}b)$，最后加上 ≈ 81.3乘以正弦函数$(w_{40}(a+b))$	92<span style='id':'1'>.7%
余弦函数$(w_{44}c)$	309<span style='id':'1'>。1 乘以余弦函数$(w_{44}a)$再乘以余弦函数$(w_{44}b)$，再加上 − 338.7 乘以正弦函数$(w_{44}a)$再乘以正弦函数$(w_{44}b)$，最后加上 ≈ 323.9 乘以余弦函数$(w_{44}(a+b))$	98<span style='id':'1'>. 5%
正弦函数$(w_{44}c)$	327<span style='id':'1'>。3 乘以余弦函数$(w_{44}a)$再乘以正弦函数$(w_{44}b)$，再加上 + 327.2 乘以正弦函数$(w_{44}a)$再乘以余弦函数$(w_{44}b)$，最后加上 ≈ 327.3 乘以正弦函数$(w_{44}(a+b))$	98<span style='id':'1'>. 9%
余弦函数$(w_{53}c)$	192.1 乘以余弦函数$(w_{53}a)$再乘以余弦函数$(w_{53}b)$，再加上 − 192.2 乘以正弦函数$(w_{53}a)$再乘以正弦函数$(w_{53}b)$，最后加上 ≈ 192.1 乘以余弦函数$(w_{53}(a+b))$	97.3%
正弦函数$(w_{53}c)$	166.7 乘以余弦函数$(w_{53}a)$再乘以正弦函数$(w_{53}b)$，再加上 + 166.8 乘以正弦函数$(w_{53}a)$再乘以余弦函数$(w_{53}b)$，最后加上 ≈ 166.8 乘以正弦函数$(w_{53}(a+b))$	95.7%

(b) 种子2

W_L 组件	u_k^T多层感知机$(a,\ b)$或 v_k^T多层感知机$(a,\ b)$的傅里叶分量	FVE
余弦函数$(w_{31}c)$	156。1 乘以余弦函数$(w_{31}a)$再乘以余弦函数$(w_{31}b)$，再加上 − 156.5 正弦函数$(w_{31}a)$乘以正弦函数$(w_{31}b)$，最后乘以 ≈ 156.3 余弦函数$(w_{31}(a+b))$	99。3%
$\sin(w_{31}c)$ 余弦函数$(w_{45}c)$	150<span style='id':'1'>.7 × 余弦函数$(w_{31}a)$ × 正弦函数$(w_{31}b)$ + + 150.7 × 正弦函数$(w_{31}a)$ × 余弦函数$(w_{31}b)$ + ≈ 150.7 × 正弦函数$(w_{31}(a+b))$	98<span style='id':'1'>. 9%
$\sin(w_{45}c)$ 余弦函数$(w_{49}c)$	72<span style='id':'1'>。5 × 余弦函数$(w_{45}a)$ × 余弦函数$(w_{45}b)$ + − 76.8 × 正弦函数$(w_{45}a)$ × 正弦函数$(w_{45}b)$ + ≈ 74.6 × 余弦函数$(w_{45}(a+b))$	95<span style='id':'1'>. 9%
$\sin(w_{49}c)$ 余弦函数$(w_{52}c)$	74<span style='id':'1'>。7 × 余弦函数$(w_{45}a)$ × 正弦函数$(w_{45}b)$ + + 74.6×正弦函数$(w_{45}a)$×余弦函数$(w_{45}b)$ + ≈ 74.6×正弦函数$(w_{45}(a+b))$	98<span style='id':'1'>. 6%
正弦函数$(w_{52}c)$	45<span style='id':'1'>.9 × 余弦函数$(w_{49}a)$ × 余弦函数$(w_{49}b)$ + − 45.5 ×正弦函数$(w_{49}a)$×正弦函数$(w_{49}b)$ + ≈ 45.7 ×余弦函数$(w_{49}(a+b))$	97<span style='id':'1'>. 0%
	45<span style id='1'>.8 × 余弦函数$(w_{49}a)$ × 正弦函数$(w_{49}b)$ + + 45.8×正弦函数$(w_{49}a)$×余弦函数$(w_{49}b)$ + ≈ 45.8×正弦函数$(w_{49}(a+b))$	96<span style id='1'>.9%
	71<span style id='1'>.6 × 余弦函数$(w_{52}a)$ × 余弦函数$(w_{52}b)$ + − 72.1 ×正弦函数$(w_{52}a)$×正弦函数$(w_{52}b)$ + ≈ 71.9 ×余弦函数$(w_{52}(a+b))$	98<span style id='1'>.5%
	68<span style='id':'1'>。7 余弦函数$(w_{52}a)$ 乘以 正弦函数$(w_{52}b)$ 再乘以 + 68.7，再乘以 正弦函数$(w_{52}a)$ 乘以 余弦函数$(w_{52}b)$，最后乘以 ≈ 68.7，整体再乘以 正弦函数$(w_{52}(a+b))$	97<span st yle='id':'1'>。9%

(c) 种子值3

W_L 组件	u_k^T多层感知机$(a,\ b)$或 v_k^T多层感知机$(a,\ b)$的傅里叶分量	FVE
余弦函数$(w_{17}c)$	66<span style='id':'1'>.0 乘以余弦函数$(w_{17}a)$再乘以余弦函数$(w_{17}b)$，再加上 − 63.5 乘以正弦函数$(w_{17}a)$再乘以正弦函数$(w_{17}b)$，最后加上 ≈ 64.8 乘以余弦函数$(w_{17}(a+b))$	<span style='id':'1'>.4%
正弦函数$(w_{17}c)$	66<span style='id':'1'>.4 乘以余弦函数$(w_{17}a)$再乘以正弦函数$(w_{17}b)$，再加上 + 66.4乘以正弦函数$(w_{17}a)$再乘以余弦函数$(w_{17}b)$，最后加上 ≈ 66.4乘以正弦函数$(w_{17}(a+b))$	94<span style='id':'1'>.9%
余弦函数$(w_{32}c)$	68<span style='id':'1'>.7 乘以余弦函数$(w_{32}a)$再乘以余弦函数$(w_{32}b)$，再加上 − 68.4 乘以正弦函数$(w_{32}a)$再乘以正弦函数$(w_{32}b)$，最后加上 ≈ 68.5 乘以余弦函数$(w_{32}(a+b))$	96<span style='id':'1'>.2%
正弦函数$(w_{32}c)$	68<span style='id':'1'>.0 乘以余弦函数$(w_{32}a)$再乘以正弦函数$(w_{32}b)$，再加上 + 68.0乘以正弦函数$(w_{32}a)$再乘以余弦函数$(w_{32}b)$，最后加上 ≈ 68.0乘以正弦函数$(w_{32}(a+b))$	96<span style='id':'1'>.3%
余弦函数$(w_{42}c)$	100<span style='id':'1'>.4 × 余弦函数$(w_{42}a)$ × 余弦函数$(w_{42}b)$ + − 96.0 × 正弦函数$(w_{42}a)$ × 正弦函数$(w_{42}b)$ + ≈ 98.2 × 余弦函数$(w_{42}(a+b))$	97<span style='id':'1'>.9%
正弦函数$(w_{42}c)$	100<span style='id':'1'>.2 × 余弦函数$(w_{42}a)$ × 正弦函数$(w_{42}b)$ + + 100.1 × 正弦函数$(w_{42}a)$ × 余弦函数$(w_{42}b)$ + ≈ 100.1 × 正弦函数$(w_{42}(a+b))$	98<span style='id':'1'>.6%
余弦函数$(w_{51}c)$	118<span style='id':'1'>。0 余弦函数$(w_{51}a)$ 余弦函数$(w_{51}b)$ − 116.2 正弦函数$(w_{51}a)$ 正弦函数$(w_{51}b)$ ≈ 117.1 余弦函数$(w_{51}(a+b))$	99<span style='id':'1'>. 0%
正弦函数$(w_{51}c)$	114<span style='id':'1'>。3 余弦函数$(w_{51}a)$ 正弦函数$(w_{51}b)$ + 114.2 正弦函数$(w_{51}a)$ 余弦函数$(w_{51}b)$ ≈ 114.2 正弦函数$(w_{51}(a+b))$	98<span style='id':'1'>。5%

(d) 种子值4

表3：针对神经元逻辑映射中的各个方向 W_L，我们采用了另外4个随机种子生成的最终模型（详见附录C.2.1），先对这些模型在该方向上的多层感知机激活值进行投影，随后执行傅里叶变换。为简化表述，我们省略了系数小于最大系数15% 的项。接着，如果我们用形如$\cos(w_k(a+b))$ 或$\sin(w_k(a+b))$的单一项的倍数来替代上述投影，便会计算出方差解释率（FVE），结果始终接近1。

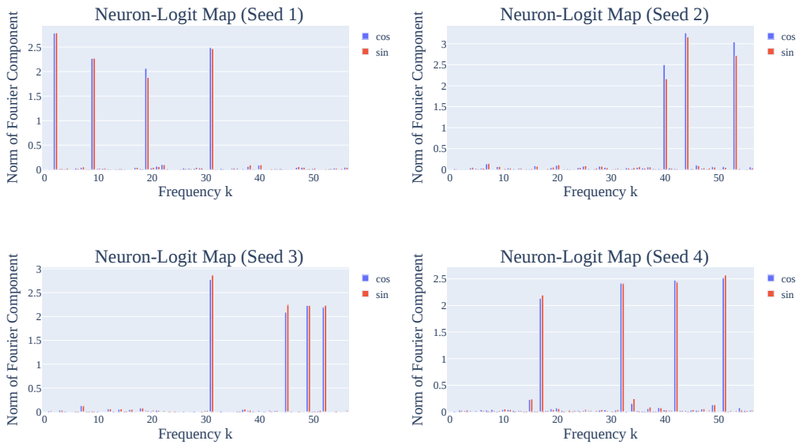


Figure 17: The norms of the direction corresponding to sine and cosine waves in the neuron-logit map weights W_L. As with the mainline model discussed in the main body and discussed in Appendix C.2.1, W_L is consistently sparse, providing is evidence that all four are operating in a Fourier basis.

Seed	Test Loss	Loss (Key frequencies removed)	Loss (All other frequencies removed)
1	$2.07 \cdot 10^{-7}$	$6.5 \cdot 10^{0}$	$5.7 \cdot 10^{-8}$
2	$2.1 \cdot 10^{-7}$	$1.1 \cdot 10^{1}$	$6.2 \cdot 10^{-8}$
3	$2.05 \cdot 10^{-7}$	$6.7 \cdot 10^{0}$	$5.5 \cdot 10^{-8}$
4	$2.33 \cdot 10^{-7}$	$6.8 \cdot 10^{0}$	$6.0 \cdot 10^{-8}$

Table 4: As discussed in Appendix C.2.1, ablating the key frequencies for each of the networks reduces performance to worse than chance, while ablating all other frequencies *improves* performance.

Confirming that the other seeds use the Fourier Multiplication Algorithm. In Figure 16, we show the norms of the Fourier components of the embedding matrix W_E for each of the 4 other random seeds. As with the mainline model, the matrices are sparse in the Fourier basis. In Figure 17, we show the norms of the Fourier components of the neuron-logit map W_L for the 4 other random seeds. The matrices are sparse in the Fourier basis, enabling us to identify 3 or 4 key frequencies for each of the seeds. Again, note that the specific frequencies differ by seed.

Using the key frequencies identified in the neuron-logit map, we repeat the experiment in Section 4.2, where we "read off" the MLP activations in the 6 or 8 directions corresponding to the key frequencies. As with our mainline model, this lets us identify the trigonometric identities for $\cos(w_k(a+b))$ and $\sin(w_k(a+b))$ being computed at the MLP layer. We confirm that the trigonometric identities are a good approximation by approximating the activations with a single term of the form $\cos(w_k(a+b))$ or $\sin(w_k(a+b))$—as with the mainline model, the fraction of variance explained is consistently close to 100%.

Next, we ablate the key frequencies from the logits as in Section 4.4 and report the results in Table 4. As with the mainline model, ablating all of the key frequencies reduces performance to worse than chance, while ablating everything *but* the key frequencies *improves* test performance.

Progress measures and grokking. Finally, we confirm the progress measure and grokking results from the mainline model on other runs with the same architecture. In Figure 18, we display the train, test, and restricted loss for each of the four other random seeds. In Figure 19, we display the Gini coefficients of the Fourier components of the embedding matrix W_E and the neuron-logit map W_L for each of the four other random seeds. The shape of the curves are very similar to those of the mainline model, allowing us to divide grokking on these models into the same three phases

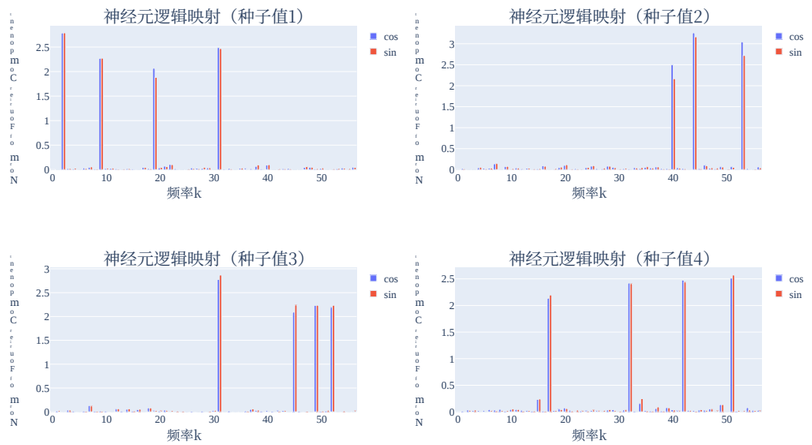


图17：神经元逻辑映射权重中对应正弦波与余弦波的方向范数 W_L。与正文及附录中讨论的主线模型情况类似C.2.1， W_L 这些数值始终呈现稀疏特征，这为所有四个参数均在傅里叶基上运行提供了证据。

Seed	测试损失值	移除关键频率后的损失值	移除所有其他频率后的损失值
1	$2.07 \cdot 10^{-7}$	$6.5 \cdot 10^{0}$	$5.7 \cdot 10^{-8}$
2	$2.1 \cdot 10^{-7}$	$1.1 \cdot 10^{1}$	$6.2 \cdot 10^{-8}$
3	$2.05 \cdot 10^{-7}$	$6.7 \cdot 10^{0}$	$5.5 \cdot 10^{-8}$
4	$2.33 \cdot 10^{-7}$	$6.8 \cdot 10^{0}$	$6.0 \cdot 10^{-8}$

表4：如附录中所述C.2.1，若移除各网络中的关键频率，其性能会降至略好于随机水平的程度；而移除所有其他频率则能提升性能。

这可以证实其他随机种子采用的是傅里叶乘法算法。在图16中，我们展示了另外4个随机种子的嵌入矩阵的傅里叶分量的范数 W_E 。与主线模型类似，这些矩阵在傅里叶基下也是稀疏的。在图17中，我们展示了另外4个随机种子的神经元逻辑映射的傅里叶分量的范数 W_L 。这些矩阵在傅里叶基下同样呈稀疏状态，因此我们能够为每个种子识别出3到4个关键频率。需要再次注意的是，不同种子对应的特定频率是有所差异的。

利用在神经元逻辑映射中识别出的关键频率，我们重新进行了第4.2节中的实验，即从与这些关键频率相对应的6个或8个方向上“读取”多层感知机层的激活值。与主线模型相同，这种方法让我们能够确定在多层感知机层中正在被计算的余弦函数（w_k（$a+b$）和正弦函数（w_k（$a+b$）对应的三角恒等式。我们通过用形如余弦函数（w_k（$a+b$）或正弦函数（w_k（$a+b$）的单一项来近似这些激活值，从而验证了这些三角恒等式确实是一种很好的近似方式——与主线模型一样，方差解释率始终接近100%。

接下来，我们按照第4.4节所述的方法从逻辑值中剔除关键频率，并在4表中呈现相关结果。与主线模型类似，若剔除所有关键频率，性能会下降到甚至低于随机水平；而仅剔除除关键频率之外的其他元素，则能提升测试性能。

进展度量指标与理解能力提升。最后，我们使用相同架构在其他多次运行中，针对主线模型验证了进展度量指标与理解能力提升的测试结果。如图18所示，我们展示了另外四个随机种子对应的训练损失、测试损失以及受限损失数值。而在图19中，则呈现了另外四个随机种子对应的嵌入矩阵的傅里叶分量的基尼系数 W_E 以及神经元逻辑映射的基尼系数 W_L 。这些曲线的形态与主线模型的曲线极为相似，因此我们可以将这些模型上的理解能力提升过程同样划分为三个阶段

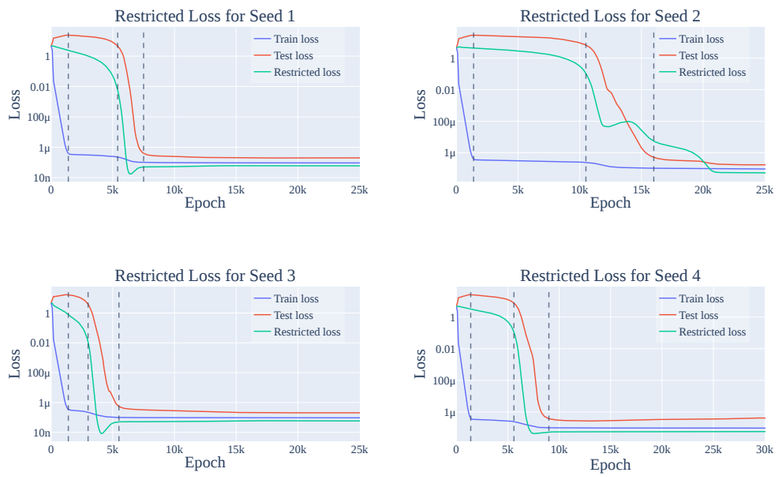


Figure 18: The train, test, and restricted loss for each of the four other random seeds described in Appendix C.2.1. The lines delineate the 3 phases of training: memorization, circuit formation, and cleanup (and a final stable phase). As with the mainline model, restricted loss consistently declines prior to train loss. Note that while the shapes of the loss curves are similar to each other and those of the mainline model, the exact time that grokking occurs (and thus the dividers between the phases of grokking) differ by random seed. Interestingly, memorization is complete by around 1400 steps for all five runs.

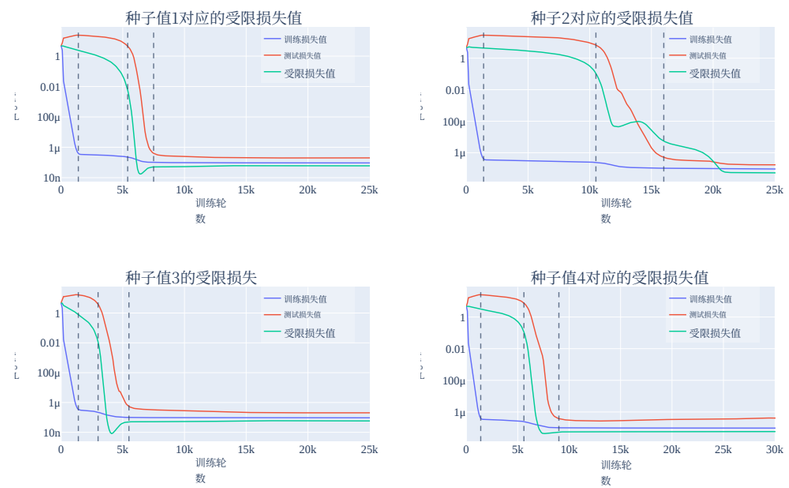


图18：附录中描述的另外四种随机种子对应的训练损失、测试损失以及受限损失值C.2.1。这些曲线划分出了训练的三个阶段：记忆阶段、电路形成阶段、清理阶段，以及最后的稳定阶段。与主线模型类似，受限损失值始终会在训练损失值之前开始下降。需要注意的是，尽管各损失曲线的形状彼此相似，也与主线模型的曲线形状相近，但理解能力提升实际发生的时间（进而也决定了不同理解能力提升阶段的划分点）会因随机种子的不同而有所差异。有趣的是，在所有五次实验中，记忆阶段大约在约1400步时就已经完成。

identified in the main text. Interestingly, while all of the models complete memorization by around 1400 epochs, circuit formation and cleanup occur at different times.

即主线文本中所述的三个阶段。有趣的是，尽管所有模型都在大约1400个训练轮数时完成记忆阶段，但电路结构的形成与清理阶段的具体时间却各不相同。

C.2.2 RESULTS FOR OTHER EXPERIMENTAL SETUPS

C.2.2 其他实验设置的结果

In this section, we provide further evidence that small transformers grok on the modular addition task, by varying the size of the network, the amount of training data, and the size of the prime P.

在本节中，我们通过改变网络规模、训练数据量以及素数 P的大小，进一步证明小型变换器能够在模块加法任务上实现理解能力提升。

1-Layer Transformers with Varying Fractions of Training Data. We find that grokking occurs for the modular addition task with $P = 113$ for many data fractions (that is, the fraction of the $113 \cdot 113$ pairs of inputs that the model sees during training), as shown in Figure 20. Smaller amount lead to slower grokking, but sufficiently large fractions of data ($\geq 60\%$) lead to immediate generalization, as shown in Figures 20 and 21.

采用不同训练数据比例的单层变换器。研究结果表明，在模块加法任务中，当模型在训练过程中所看到的输入对所占的$113 \cdot 113$ 数据比例（即 $P = 113$ 众多数据比例范围内$113 \cdot 113$ ），都会出现理解能力提升现象，如图20所示。数据比例过小会导致理解能力提升的速度变慢，而当数据比例足够大时（$\geq 60\%$），模型就能立即展现出良好的泛化能力，这一结论在图20 和21中也有体现。

As with the results in Appendix C.2.1, all of the 1-layer transformers in this section also converge to using the Fourier multiplication algorithm.

与附录C.2.1中的结果一致，本节中的所有单层变换器也是通过傅里叶乘法算法实现收敛的。

2-Layer Transformers. As shown in Figure 22, 2-layer transformers also exhibit some degree of grokking. However, this is complicated by the slingshot mechanism (Thilak et al., 2022). We display the excluded loss of a 2-layer transformer in Figure 23 and find it shows a similar pattern to the mainline 1-layer transformer, in that it improves relatively smoothly *before* grokking occurs.

两层变换器。如图22所示，两层变换器同样也表现出一定程度的理解能力提升。不过，由于存在弹弓机制（Thilak等人，2022年），这一现象变得较为复杂。我们在图23 中展示了两层变换器的排除损失曲线，发现其变化趋势与主流一层变换器类似，即在出现理解能力提升之前会相对平稳地上升。

Smaller and larger primes. We also examined smaller and larger prime moduli. For $P = 53$ (Figure 24), we explored a variety of weight decays to observe grokking in the small prime case. With the original weight decay setting of $\lambda = 1$, we found that the models never generalized. However, increasing the weight decay to $\lambda = 5$ does allow the model to grok. We speculate that

规模更小和更大的素数。我们还研究了规模更小和更大的素数模数。在 $P = 53$(图 24)中，我们尝试了多种权重衰减方式，旨在观察在小素数场景下的理解能力提升现象。采用 $\lambda = 1$原有的权重衰减设置时，我们发现模型完全无法实现泛化；而将权重衰减值提高至 $\lambda = 5$ 相应水平后，模型则能够展现出理解能力提升的效果。我们推测

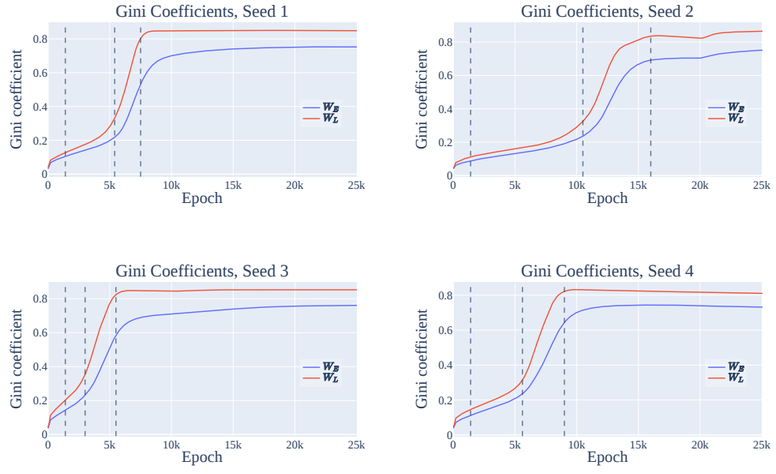


Figure 19: The Gini coefficients (a measure of sparsity) of the Fourier components of the embedding matrix W_E and the neuron-logit map W_L for each of the four other random seeds. The lines delineate the 3 phases of training: memorization, circuit formation, and cleanup (and a final stable phase). As with the mainline model, sparsity increases slowly during memorization and circuit formation, and then quickly during cleanup.

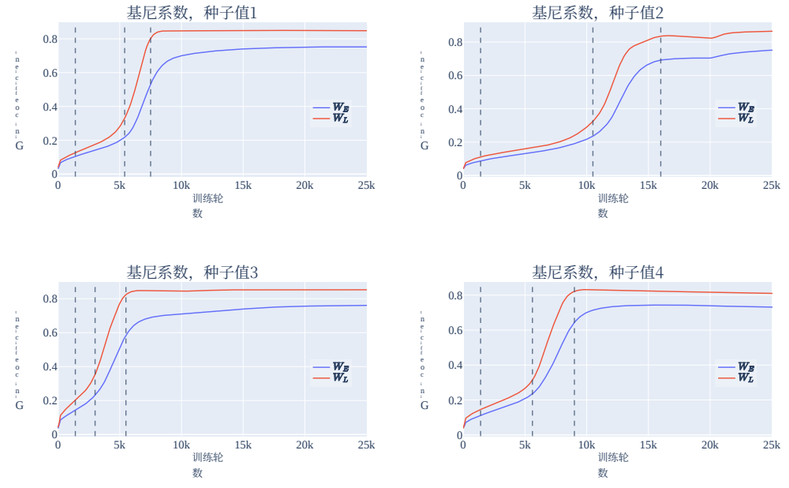


图19：展示了使用不同随机种子的情况下，嵌入矩阵 W_E 与神经元逻辑映射 W_L 的傅里叶分量的基尼系数（用于衡量稀疏性）变化情况。曲线划分了训练过程中的三个阶段：记忆阶段、电路形成阶段以及清理阶段（最后还有一个稳定阶段）。与主线模型类似，在记忆阶段和电路形成阶段稀疏性增长较为缓慢，而在清理阶段则上升迅速。

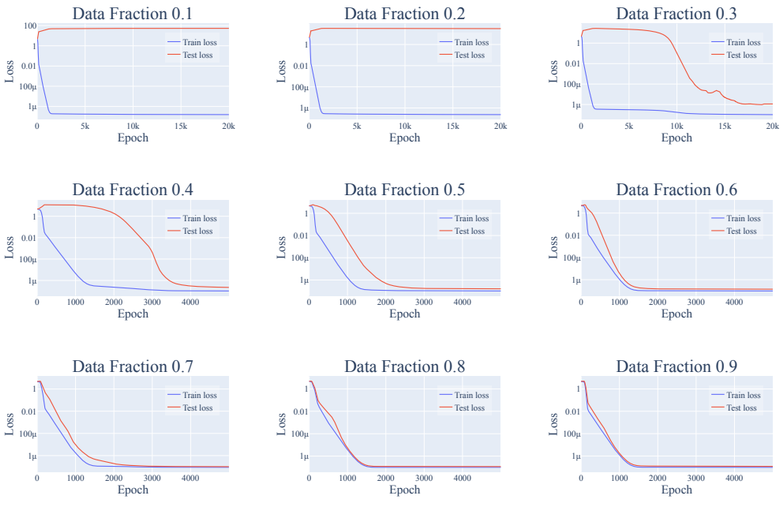


Figure 20: Training and test losses for a 1-layer transformer on the modular addition task with $P = 113$, with varying fractions of the $113 \cdot 113$ pairs of possible inputs used in training. Grokking occurs when between $30 - 50\%$ of the dataset is used during training and lower fractions of data lead to slower grokking. Using $\geq 60\%$ data leads to immediate generalization, while using 10% or 20% of the data doesn't lead to grokking even after 40k epochs. Note the different x-axes: we only show 5k epochs for the runs with data fraction $\geq 40\%$ for more detail.

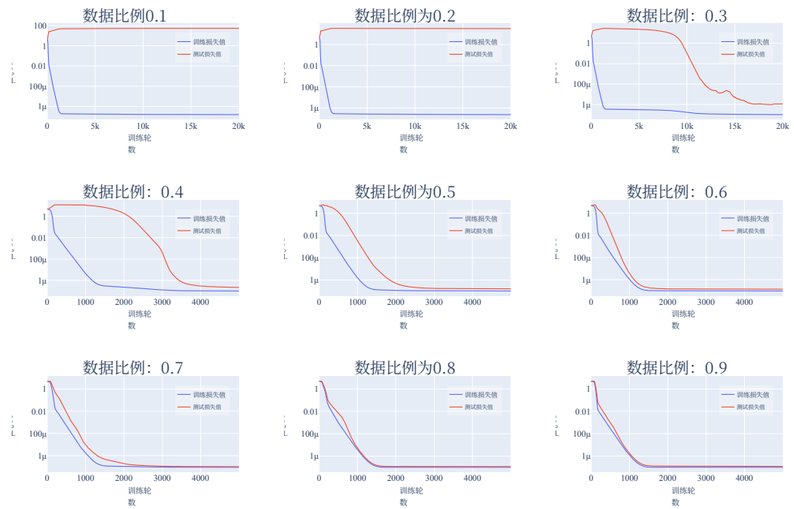


图20：展示了在模块加法任务上使用1层Transformer模型时，随着训练中使用的 $113 \cdot 113$ 类可能输入对的比例变化，其训练损失与测试损失的情况。当训练时使用的数据集比例达到 $30 - 50\%$时就会出现理解能力提升的现象，而较低的数据比例则会降低这种提升速度。若使用 $\geq 60\%$的数据，模型能够立即展现出良好的泛化能力；而即便使用10%或20%的数据，即便经过4万次训练轮数，也依然无法实现理解能力提升。需要注意的是，此处x轴的显示范围有所区别——对于数据比例为 $\geq 40\%$的实验，为呈现更详细的数据，仅展示了5千次训练轮数的结果。

Figure 21: Number of steps for train/test loss to be $< 10^{-6}$, as a function of the amount of training data. While train loss immediately converges to below 10^{-6} for all data fractions, generalization takes significantly longer with lower fractions of data. Note that the plots for other thresholds are also qualitatively similar.

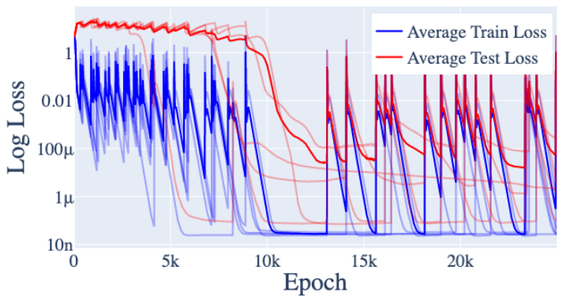


Figure 22: Training and test loss for a 2-layer version of the original architecture. Average across 5 random seeds is in bold.

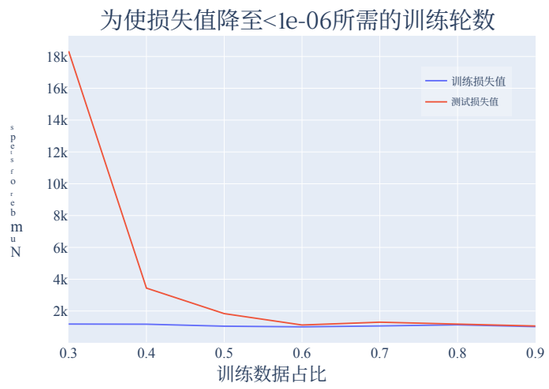


图21：随着训练数据量的变化，使训练/测试损失值达到 $< 10^{-6}$所需的步数。虽然对于所有数据比例，训练损失值都会立即降至 10^{-6} 以下，但随着数据比例的降低，模型的泛化能力提升所需的时间会显著延长。需注意的是，其他阈值对应的曲线在形态上也大致相似。

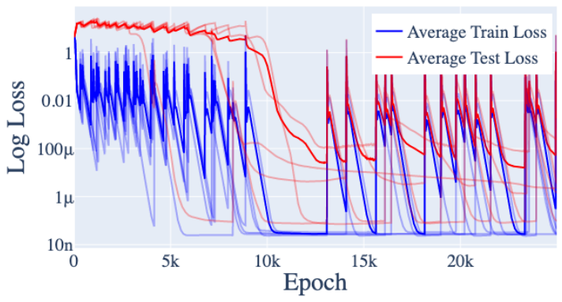


图22：原始架构的2层版本所对应的训练集与测试集损失值。以粗体显示的是在5个不同随机种子下计算得到的平均值。

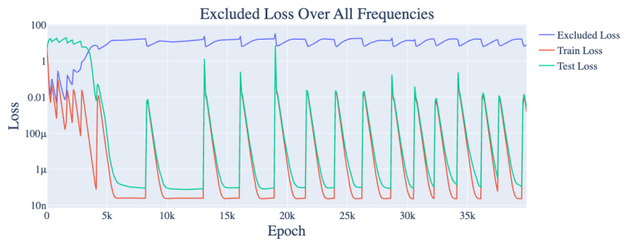


Figure 23: Training, test, and full excluded loss for a 2-layer version of the original architecture. One random seed chosen for readability.

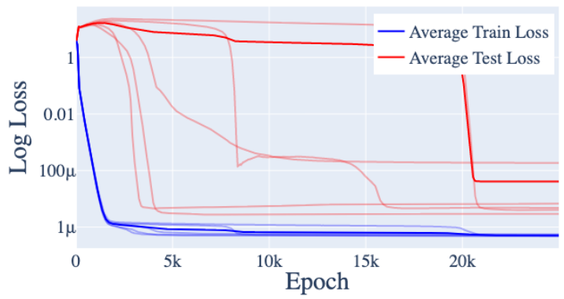


Figure 24: The training and test losses for $P = 53$ and all other hyperparameters except weight decay ($\gamma = 5$) the same as the main training run discussed in the paper. The averages are bold, and all contributing runs are partially transparent. Note that grokking occurs.

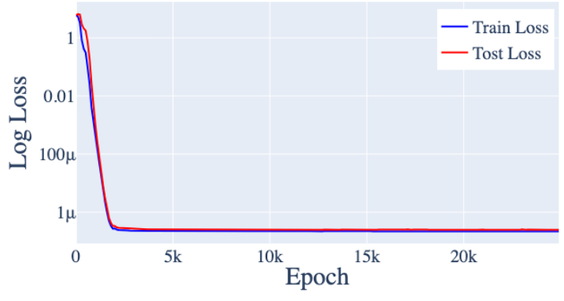


Figure 25: The training and test losses for $P = 401$ and all other hyperparameters the same as the main training run discussed in the paper. Grokking doesn't occur (the model generalizes immediately), even across a variety of weight decays.

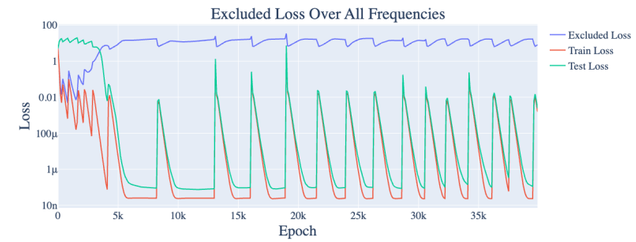


图23：原始架构的2层版本所对应的训练损失、测试损失以及完全排除损失值。为便于阅读，此处仅选用了一个随机种子。

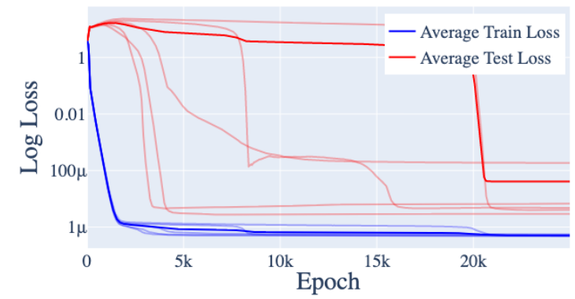


图24： $P = 53$ 以及其他所有超参数（权重衰减除外，其值与文中所述的主训练过程相同）对应的训练损失和测试损失值。平均值以粗体显示，而所有参与计算的训练过程则采用半透明处理。可见此时确实发生了理解能力提升现象。

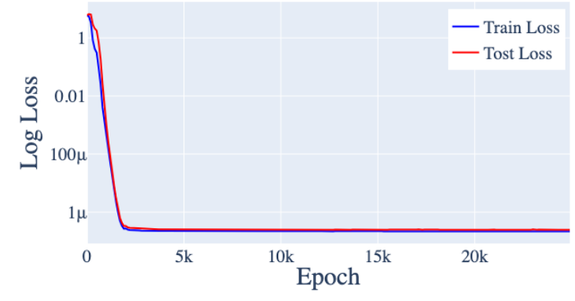


图25： $P = 401$ 以及其他所有超参数（与文中主训练过程相同的那些）对应的训练损失和测试损失值。即便在不同强度的权重衰减条件下，模型也并未出现理解能力提升现象（即能够立即实现泛化）。

this is because the memorization solution is significantly smaller (since there are only $53 \cdot 53$ total pairs), thereby requiring more aggressive weight decay for the generalizing solution to be favored.

For $P = 109$, we saw exactly the same behavior as with the mainline model.

For $P = 401$ (Figure 25), we could not get grokking, even by varying the weight decay parameter $\lambda \in \{0.3, 0.5, 1, 3, 5, 8\}$. Instead, the model immediately learns the generalizing solution. We believe this is because the amount of data seen by the model is greatly increased compared to the $P = 113$ case (from 30% of $113 \cdot 113$ pairs to 30% of $401 \cdot 401$ pairs), thereby favoring the generalizing solution from the start. We then trained 3 models each using 5%, 10%, 20% of the pairs of training data with $\lambda = 1$, and found that the models trained on 5% and 10% of the data immediately overfit and never generalized, while the models trained on 20% of the data also generalized immediately.

C.2.3 Generalizing models consistently use the Fourier Multiplication Algorithm

For each of the models in Appendix C.2.2 that achieve low test loss, we repeated the analysis performed in the mainline model, and summarize the results in Table 5. We list their key frequencies, Gini coefficients, and relevant FVEs. We find that every model trained with weight decay and that generalizes correctly implements some variation of the Fourier multiplication algorithm.

Interestingly, the embedding and unembedding matrices of the models trained with dropout are *not* sparse in the Fourier basis, and the logits for the $p = 0.2$ models are not as well explained by a sum of cosines as the other models (likely because the $p = 0.2$ models are simply worse at the task). We speculate that this is likely due to a combination of insufficient training epochs (as dropout models seem to take much longer to grok) and the inherent need for redundancy for networks trained via dropout.

As with the mainline model, we ignore the final skip connection (around the final MLP), as all of the generalizing models studied do not suffer significant performance penalties if the skip connection is zero or mean ablated (Table 6).

D Additional results on grokking

D.1 Both regularization and limited data are necessary for grokking

As discussed in Section 7 and Appendix C.2, the weight decay and the amount of data seem to have a strong effect on whether grokking occurs. To confirm this, we experiment with removing weight decay and varying the amount of data on 1-layer transformers. In Figure 26, we give the training, test, and full excluded loss for a typical training run with $\lambda = 0$ (no weight decay). As the figure shows, no grokking occurs, and excluded loss does not increase, suggesting that the model does not form the circuit for generalizing algorithm at all.

In Figure 20, we show the test loss curves for models trained with weight decay $\lambda = 1$ and on various fractions of the data. Though all the train losses are approximately the same—that is, they memorize at the same rate, models trained on smaller fractions of data take longer to grok.

In Figure 27, we display the test and train loss of models trained with $\lambda = 0.3$ and $\lambda = 3.0$. Smaller amounts of weight decay lead to slower grokking, while larger amounts of weight decay lead to faster grokking—on average, it takes around 3k epochs for models to grok with weight decay $\lambda = 0.3$, 5-10k epochs for the models to grok with weight decay $\lambda = 1.0$, and 20k epochs for the models to grok with weight decay $\lambda = 3.0$.

Finally, we test whether other forms of regularization can also induce grokking. We replaced weight decay with the following types of regularization while keeping all other hyperpameters the same:

1. **Dropout** We add dropout Srivastava et al. (2014) to the MLP neurons, with $p \in \{0.2, 0.5, 0.8\}$. That is, for each individual neuron, we set it to 0 with probability p during training, and also multiply the outputs of the other neurons by $\frac{1}{1-p}$.

这是因为记忆化解的规模要小得多（毕竟总共有 $53 \cdot 53$ 对数据），所以需要更强的权重衰减才能让泛化解占据优势。

对于 $P = 109$的情况，我们观察到的表现与主线模型完全一致。

针对 $P = 401$(图 25)，即便调整权重衰减参数 $\lambda \in \{0.3, 0.5, 1, 3, 5, 8\}$，也无法实现理解能力提升。相反，模型会立即学习到泛化解。我们认为这是由于模型所接触的数据量相比$P = 113$ 之前的情况有了显著增加（从 $113 \cdot 113$ 30%的对数据增加到 $401 \cdot 401$ 30%的对数据），因此从一开始就更有利于泛化解的形成。随后我们分别用5%、10%、20% 的训练数据对模型进行训练，并发现使用相应比例数据训练的模型会立即出现过拟合现象且无法实现泛化，而使用5% 和10% 比例数据训练的模型也能立刻实现泛化。

C.2.3 一致地使用傅里叶乘法算法对模型进行泛化处理

在附录C.2.2中那些测试损失较低的模型中，我们重复了主线模型中所进行的分析，并将结果汇总于5表中。表中列出了这些模型的关键频率、基尼系数以及相关的FVE值。研究结果表明，所有采用权重衰减训练且能够正确实现泛化的模型，都采用了某种形式的傅里叶乘法算法。

有趣的是，那些采用丢弃机制训练的模型的嵌入矩阵与解嵌入矩阵在傅里叶基下并非稀疏矩阵，而且这些模型的逻辑值无法像其他模型那样仅通过余弦函数来很好地解释（这很可能是因为这类模型在该任务上的表现本来就更差）。我们认为，造成这一现象的原因可能是训练轮数不足（因为带有丢弃机制的模型似乎需要更长的时间才能实现理解能力提升），再加上通过丢弃机制训练的网络本身就需要具备一定的冗余性。

与主线模型类似，我们忽略了最终跳接连接（即最终多层感知器周围的连接），因为所有经过研究的泛化算法模型在跳接连接被设为零或被完全移除时，都不会出现显著的性能下降（见表6）。

D 关于理解能力提升的额外结果

D.1 正则化与有限数据都是实现理解能力提升的必要条件

如第7节及附录C.2中所讨论的，权重衰减与数据量似乎会对是否出现理解能力提升产生显著影响。为验证这一点，我们在单层变换器上进行了实验，尝试去除权重衰减并改变数据量。在图26中，我们展示了采用 $\lambda = 0$ （无权重衰减）条件进行典型训练时的训练损失、测试损失以及完全排除损失的情况。如图所示，此时并未出现理解能力提升现象，且排除损失也没有上升，这说明该模型根本未形成用于实现泛化算法的电路结构。

在图20中，我们展示了应用了权重衰减 $\lambda = 1$ 且使用不同比例数据的模型对应的测试损失曲线。尽管所有模型的训练损失大致相同，也就是说它们的记忆速度一致，但那些使用较少数据量训练的模型需要更长的时间才能实现理解能力提升。

在图27中，我们展示了使用 $\lambda = 0.3$ 以及 $\lambda = 3.0$训练的模型所对应的测试损失与训练损失。较小的权重衰减量会导致理解能力提升的速度变慢，而较大的权重衰减量则会加快这一过程——通常情况下，当采用 $\lambda = 0.3$10千 $\lambda = 3.0$的权重衰减时，模型需要大约3千个训练轮数才能实现理解能力提升；采用 $\lambda = 1.0$2万步 $\lambda = 3.0$的权重衰减时则需要5到10千个训练轮数；而采用 $\lambda = 3.0$2万步 $\lambda = 3.0$的权重衰减时，则需要2万步的训练轮数。

最后，我们测试了其他形式的正则化是否也能带来理解能力提升。在保持所有其他超参数不变的情况下，我们将原有的权重衰减替换为了以下几种正则化方式。

1.**丢弃机制**我们会在多层感知机神经元中引入丢弃机制Srivastava等人（2014年）指出，具体做法是 $p \in \{0.2, 0.5, 0.8\}$。也就是说，在训练过程中，对于每一个单独的神经元，我们会以0的概率将其值设为该数值，同时还会将其他神经元的输出乘以$\frac{1}{1-p}$。

Model	Test Loss	Gini(W_E)	Gini(W_L)	Key Frequencies	Logit FVE	MLP FVE
40% Training Data	$1.98 \cdot 10^{-7}$	0.76	0.79	[17, 43, 49, 55]	94.9%	83.3% [26.1%]
50% Training Data	$1.68 \cdot 10^{-7}$	0.75	0.77	[2, 17, 31, 41, 44]	91.2%	85.2% [28.2%]
60% Training Data	$1.23 \cdot 10^{-7}$	0.79	0.84	[2, 23, 34, 51]	96.4%	95.7% [1.4%]
70% Training Data	$9.85 \cdot 10^{-8}$	0.80	0.91	[14, 15, 26]	99.0%	98.9% [0.4%]
80% Training Data	$5.83 \cdot 10^{-7}$	0.62	0.80	[38, 41]	63.9%	94.1% [2.5%]
90% Training Data	$1.11 \cdot 10^{-7}$	0.79	0.88	[3, 26, 34, 43]	98.6%	98.7% [0.3%]
2 Layer Transformer	$9.54 \cdot 10^{-7}$	0.59	0.80	[14, 18, 29]	91.8%	95.2% [1.9%]
2 Layer Transformer	$4.41 \cdot 10^{-5}$	0.55	0.73	[7, 12, 35, 49]	86.1%	86.2% [6.4%]
2 Layer Transformer	$6.50 \cdot 10^{-2}$	0.66	0.80	[4, 9, 28]	88.5%	85.4% [5.9%]
2 Layer Transformer	$4.18 \cdot 10^{-2}$	0.56	0.76	[4, 5, 15, 54]	91.4%	81.2% [17.8%]
2 Layer Transformer	$1.75 \cdot 10^{-2}$	0.68	0.71	[3, 4, 13, 30, 38]	84.0%	71.9% [19.5%]
$P = 53$	$3.00 \cdot 10^{-4}$	0.61	0.68	[6, 9, 16, 21]	91.2%	90.2% [5.8%]
$P = 53$	$1.03 \cdot 10^{-4}$	0.56	0.72	[4, 13, 16]	94.8%	93.1% [6.4%]
$P = 53$	$1.21 \cdot 10^{-5}$	0.66	0.79	[13, 22, 23]	98.2%	97.6% [0.9%]
$P = 53$	$3.95 \cdot 10^{-6}$	0.66	0.74	[3, 14, 15]	88.5%	91.8% [4.6%]
$P = 53$	$5.56 \cdot 10^{-6}$	0.67	0.80	[10, 14, 22]	98.1%	98.3% [0.6%]
$P = 109$	$2.02 \cdot 10^{-7}$	0.76	0.83	[6, 7, 22, 25]	98.0%	97.3% [1.9%]
$P = 109$	$2.95 \cdot 10^{-7}$	0.69	0.82	[8, 14, 29, 32, 41]	95.2%	94.7% [2.3%]
$P = 109$	$1.66 \cdot 10^{-7}$	0.78	0.86	[13, 23, 39, 45]	98.5%	97.6% [0.9%]
$P = 109$	$2.50 \cdot 10^{-7}$	0.68	0.82	[8, 13, 32, 41]	96.8%	95.5% [2.3%]
$P = 109$	$2.77 \cdot 10^{-7}$	0.76	0.85	[29, 37, 38, 49]	97.9%	98.1% [0.8%]
Dropout $p = 0.2$	$2.65 \cdot 10^{-1}$	0.19	0.46	[1, 4, 7, 17, 22, 33, 40, 49, 55]	71.3%	65.0% [17.5%]
Dropout $p = 0.2$	$4.52 \cdot 10^{-1}$	0.19	0.46	[3, 8, 19, 28, 32, 34, 40, 44]	73.3%	71.4% [10.7%]
Dropout $p = 0.2$	$2.03 \cdot 10^{-1}$	0.20	0.45	[4, 5, 32, 38, 41, 44, 49, 50]	74.2%	71.1% [10.6%]
Dropout $p = 0.5$	$< 10^{-8}$	0.26	0.56	[1, 4, 26, 46, 47, 55]	89.4%	88.9% [3.5%]
Dropout $p = 0.5$	$2.01 \cdot 10^{-2}$	0.20	0.49	[16, 21, 35, 47, 53]	88.4%	88.4% [3.0%]
Dropout $p = 0.5$	$< 10^{-8}$	0.25	0.54	[1, 4, 7, 19, 29, 31, 42]	86.1%	85.6% [4.0%]

Table 5: For each of the models in Appendices C.2.3 and D.1 that generalizes to test data, we report the test loss, the Gini coefficients of the norms of the Fourier components of W_E and W_L (Section 5.1), the key frequencies of the network, and the fraction of variance in logits explained by a weighted sum of $\cos(w_k(a+b-c))$s over the key frequencies (Section 4.2).

In addition, we find the components u_k, v_k of W_L that correspond to cosines and sines of the key frequencies, and then report the average fraction of variance of $u_k^T \text{MLP}(a,b)$ and $v_k^T \text{MLP}(a,b)$ explained by a single term of form $\cos(w_k(a+b))$ or $\sin(w_k(a+b))$ respectively (Section 4.2). Numbers in square brackets represent the standard deviation. For 2 Layer models, we use the final layer MLP activations for $\text{MLP}(a,b)$.

We omit test accuracy because every model on this list except for the dropout $p = 0.2$ models achieves $> 99.95\%$ test accuracy, while the dropout $p = 0.2$ models achieve around 99.6% test accuracy.

Model Type	Loss	Accuracy	Ablated Loss	Ablated Acuracy
Varying Data Fraction	$1.83 \cdot 10^{-7}$ $(1.65 \cdot 10^{-7})$	100%	$7.74 \cdot 10^{-7}$ $(6.74 \cdot 10^{-7})$	100%
2 Layer Transformer	$1.97 \cdot 10^{-2}$ $(2.41 \cdot 10^{-2}$	99.6%	$4.63 \cdot 10^{-2}$ $(6.72 \cdot 10^{-2}$	98.7%
$P = 53$	$5.96 \cdot 10^{-5}$ $(8.91 \cdot 10^{-5})$	100%	$1.5 \cdot 10^{-4}$ $(2.70 \cdot 10^{-4})$	100%
$P = 109$	$1.94 \cdot 10^{-7}$ $(3.74 \cdot 10^{-8})$	100%	$6.53 \cdot 10^{-7}$ $(1.41 \cdot 10^{-7})$	100%
Dropout $p = 0.2$	0.215 (0.091)	99.7%	0.205 (0.075)	99.7%
Dropout $p = 0.5$	$4.68 \cdot 10^{-3}$ $(8.11 \cdot 10^{-3})$	100%	$3.6 \cdot 10^{-3}$ $(5.82 \cdot 10^{-3})$	100%

Table 6: We confirm that the skip connection around the final MLP layer is not important for performance by mean ablating the skip connection and computing loss and accuracy over the entire dataset for each problem, averaged over all runs. (We report the standard deviation of loss over the runs in parentheses.) While loss does increase a small amount, accuracy remains consistently high and the loss of the ablated model remains low. Results with zero ablations are also similar.

模型	测试损失值	Gini(W_E)	Gini(W_L)	关键频率	Logit FVE指标	MLP FVE指标
40%的训练数据	$1.98 \cdot 10^{-7}$	0.76	0.79	[17, 43, 49, 55]	94.9%	83.3% [26.1%]
50%的训练数据	$1.68 \cdot 10^{-7}$	0.75	0.77	[2, 17, 31, 41, 44]	91.2%	85.2% [28.2%]
60% 训练数据集	$1.23 \cdot 10^{-7}$	0.79	0.84	[2, 23, 34, 51]	96.4%	95.7% [1.4%]
70% 训练数据集	$9.85 \cdot 10^{-8}$	0.80	0.91	[14, 15, 26]	99.0%	98.9% [0.4%]
80% 训练数据集	$5.83 \cdot 10^{-7}$	0.62	0.80	[38, 41]	63.9%	94.1% [2.5%]
90% 训练数据集	$1.11 \cdot 10^{-7}$	0.79	0.88	[3, 26, 34, 43]	98.6%	98.7% [0, 0.3%]
双层Transformer模型	$9.54 \cdot 10^{-7}$	0.59	0.80	[14, 18, 29]	91.8%	95.2% [1.9%]
双层Transformer模型	$4.41 \cdot 10^{-5}$	0.55	0.73	[7, 12, 35, 49]	86.1%	86.2% [6.4%]
双层Transformer模型	$6.50 \cdot 10^{-2}$	0.66	0.80	[4, 9, 28]	88.5%	85.4% [5.9%]
双层Transformer模型	$4.18 \cdot 10^{-2}$	0.56	0.76	[4, 5, 15, 54]	91.4%	81.2% [17, 再加上0.8%]
双层Transformer模型	$1.75 \cdot 10^{-2}$	0.68	0.71	[3, 4, 13, 30, 38]	84.0%	71.9% [19, 再减去0.5%]
$P = 53$	$3.00 \cdot 10^{-4}$	0.61	0.68	[6, 9, 16, 21]	91.2%	90.2% [5.8%]
$P = 53$	$1.03 \cdot 10^{-4}$	0.56	0.72	[4, 13, 16]	94.8%	93.1% [6.4%]
$P = 53$	$1.21 \cdot 10^{-5}$	0.66	0.79	[13, 22, 23]	98.2%	97.6% [0.9%]
$P = 53$	$3.95 \cdot 10^{-6}$	0.66	0.74	[3, 14, 15]	88.5%	91.8% [4.6%]
$P = 53$	$5.56 \cdot 10^{-6}$	0.67	0.80	[10, 14, 22]	98.1%	98.3% [0, 0.6%]
$P = 109$	$2.02 \cdot 10^{-7}$	0.76	0.83	[6, 7, 22, 25]	98.0%	97.3% [1.9%]
$P = 109$	$2.95 \cdot 10^{-7}$	0.69	0.82	[8, 14, 29, 32, 41]	95.2%	94.7% [2.3%]
$P = 109$	$1.66 \cdot 10^{-7}$	0.78	0.86	[13, 23, 39, 45]	98.5%	97.6% [0.9%]
$P = 109$	$2.50 \cdot 10^{-7}$	0.68	0.82	[8, 13, 32, 41]	96.8%	95.5% [2.3%]
$P = 109$	$2.77 \cdot 10^{-7}$	0.76	0.85	[29, 37, 38, 49]	97.9%	98.1% [0.8%]
丢弃机制 $p = 0.2$	$2.65 \cdot 10^{-1}$	0.19	0.46	[1, 4, 7, 17, 22, 33, 40, 49, 55]	71.3%	65.0% [17.5%]
丢弃机制 $p = 0.2$	$4.52 \cdot 10^{-1}$	0.19	0.46	[3, 8, 19, 28, 32, 34, 40, 44]	73.3%	71.4% [10.7%]
丢弃机制 $p = 0.2$	$2.03 \cdot 10^{-1}$	0.20	0.45	[4, 5, 32, 38, 41, 44, 49, 50]	74.2%	71.1% [10, 0.6%]
丢弃机制 $p = 0.5$	$< 10^{-8}$	0.26	0.56	[1, 4, 26, 46, 47, 55]	89.4%	88.9% [3.5%]
采用丢弃机制 $p = 0.5$	$2.01 \cdot 10^{-2}$	0.20	0.49	[16, 21, 35, 47, 53]	88.4%	88.4% [3.0%]
采用丢弃机制 $p = 0.5$	$< 10^{-8}$	0.25	0.54	[1, 4, 7, 19, 29, 31, 42]	86.1%	85.6% [4.0%]

表5：针对附录C.2.3 和D.1中那些能够泛化到测试数据的模型，我们列出了测试损失、 W_E 以及 W_L （见第5.1节）这些数据的傅里叶分量范数的基尼系数、网络中的关键频率，还有由$\cos(w_k(a+b-c))$s在关键频率上的加权求和所能解释的逻辑值方差的比例（见第4.2节）。

此外，我们找到了 W_L 中对应于关键频率的余弦函数与正弦函数的 u_k、 v_k 两个分量，随后分别报告了 $u_k^T \text{MLP}(a,b)$以及$v_k^T \text{MLP}(a,b)$的方差中有多少比例能够由形式为$\cos(w_k(a+b))$或$\sin(w_k(a+b))$的单一项来解释（详见第4.2节）。方括号内的数值代表标准差。对于双层模型，我们在计算$\text{MLP}(a,b)$时使用的是最末层的多层感知机激活值。

我们没有列出测试准确率，因为除了采用丢弃机制 $p = 0.2$ 的模型外，此列表中的所有模型都能达到 $> 99.95\%$的测试准确率，而那些使用了丢弃机制 $p = 0.2$ 的模型则能达到约99.6%的测试准确率。

模型类型	Loss	准确率	损失值削减情况	精度降低情况
不同数据比例下的表现	$1.83 \cdot 10^{-7}$ $(1.65 \cdot 10^{-7})$	100%	$7.74 \cdot 10^{-7}$ $(6.74 \cdot 10^{-7})$	100%
双层Transformer模型	$1.97 \cdot 10^{-2}$ $(2.41 \cdot 10^{-2})$	9999%	$4.63 \cdot 10^{-2}$ $(6.72 \cdot 10^{-2})$	9898%
$P = 53$	$5.96 \cdot 10^{-5}$ $(8.91 \cdot 10^{-5})$	100%	$1.5 \cdot 10^{-4}$ $(2.70 \cdot 10^{-4})$	100%
$P = 109$	$1.94 \cdot 10^{-7}$ $(3.74 \cdot 10^{-8})$	100%	$6.53 \cdot 10^{-7}$ $(1.41 \cdot 10^{-7})$	100%
采用丢弃机制 $p = 0.2$	0.215（0.091）	99.7%	0.205（0.075）	99.7%
丢弃机制 $p = 0.5$	$4.68 \cdot 10^{-3}$ $(8.11 \cdot 10^{-3})$	100%	$3.6 \cdot 10^{-3}$ $(5.82 \cdot 10^{-3})$	100%

表6：我们通过针对每个问题在全部数据集上执行消融实验，并计算损失值与准确率，再对所有实验结果取平均值，从而证实最终多层感知器层周围的跳接连接对模型性能并无重要影响。（损失值的标准差会在括号中给出。）虽然损失值会有轻微上升，但准确率依然保持较高水平，且经过消融处理的模型的损失值也处于较低状态。未进行任何消融实验的结果也与此类似。

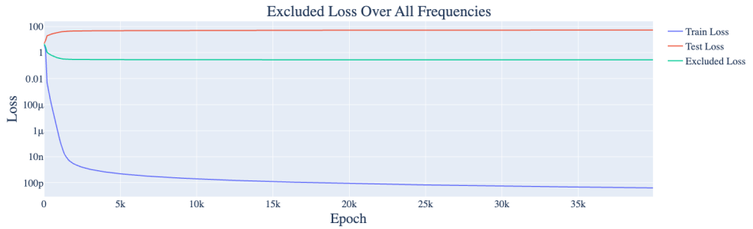


Figure 26: Training, test, and full excluded loss for a 1-layer version of the original architecture without weight decay. One random seed chosen for readability. Note that not having weight decay prevents grokking.

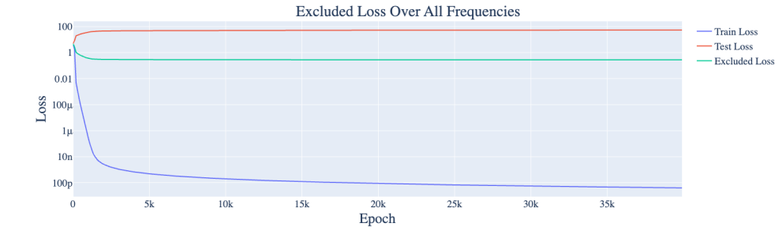


图26：未应用权重衰减的原始架构单层版本的训练损失、测试损失以及完全排除损失。为便于阅读，此处仅选用了一个随机种子。需注意，不使用权重衰减会阻碍理解能力提升。

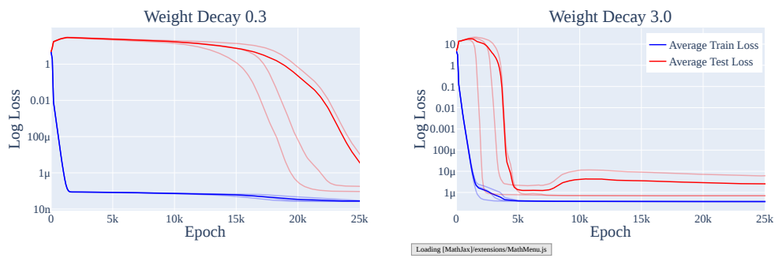
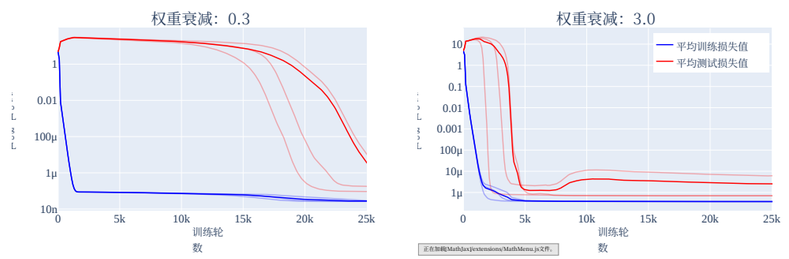


Figure 27: The train and test loss over the course of training with weight decay $\lambda = 0.3$ (left) and $\lambda = 3.0$ (right). Less aggressive weight decay leads to slower grokking.



Figu图27：应用权重衰减 $\lambda = 0.3$ （左侧）a时的训练过程及对应的训练损失与测试损失变化情况。ηdλ = 3.0（右侧）。较温和的权重衰减会导致理解能力提升的速度变慢。

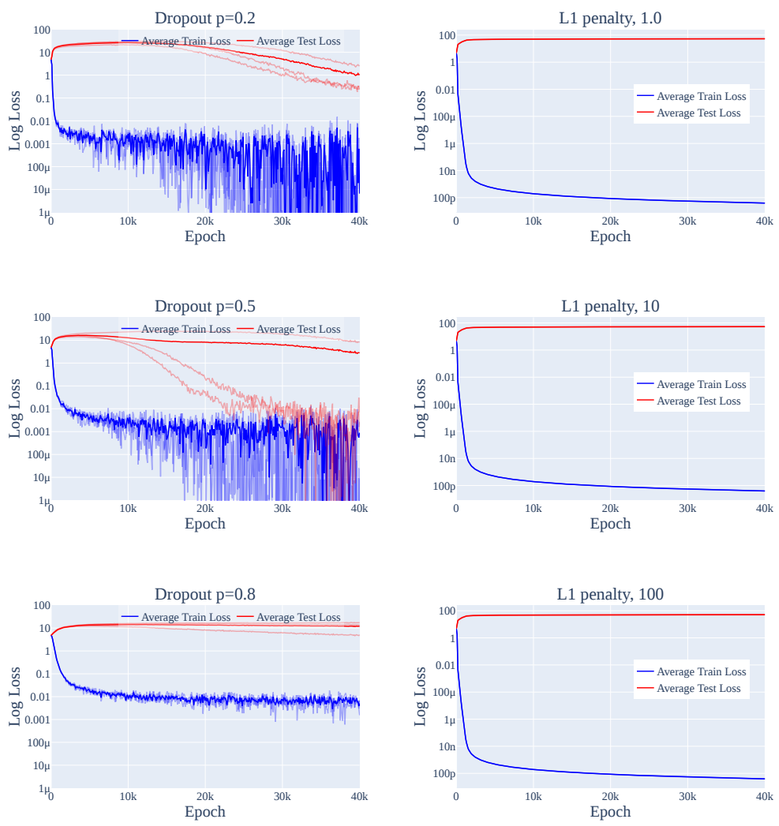


Figure 28: The train and test loss over the course of training with two types of regularization, dropout and ℓ_1 regularization. Grokking occurs with some runs for dropout but never for ℓ_1 regularization.

2. ℓ_1 **Regularization** We add an ℓ_1 penalty to the loss term. We use $\lambda \in \{1, 10, 100\}$. Note that we *do not decouple* the updates with respect to the ℓ_1 penalty from optimization steps done with respect to the log loss (as is done for ℓ_2 regularization via AdamW Loshchilov & Hutter (2017)).

In each case, we ran three random seeds. We show the results in Figure 28. While grokking did not occur with ℓ_1 regularization, we found that it does occur for all three seeds using dropout with $p = 0.2$ or $p = 0.5$. We speculate that this is because both dropout and weight decay encourage the network to spread out computation (which is required for the Fourier multiplication algorithm), while ℓ_1 regularization encourages the network to become more sparse in the neuron basis and thus *less* sparse in the Fourier basis, preventing the network from learning the Fourier Multiplication Algorithm.

D.2 The slingshot mechanism often occurs, but is unnecessary for grokking

As noted in Section C.2, our 2-layer transformers exhibit significant slingshots (Thilak et al., 2022) during training. We speculate that this is due to how gradients of different scale interact with adaptive

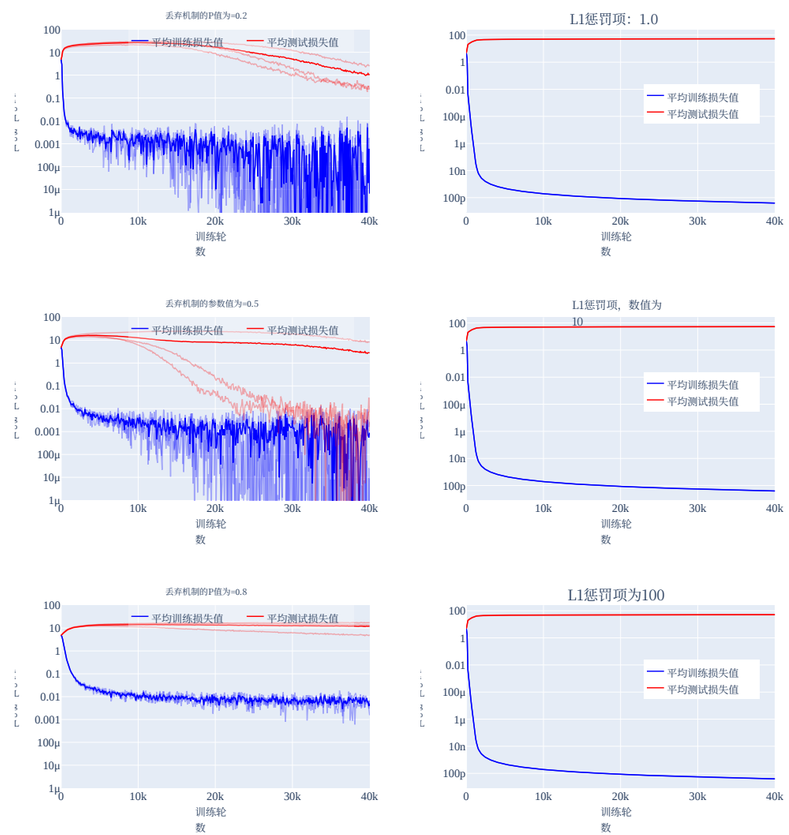


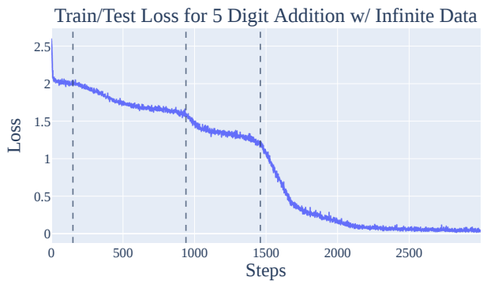
图28：展示了在应用两种正则化方式——丢弃机制以及 ℓ_1 正则化——的训练过程中，训练损失与测试损失的变化情况。使用丢弃机制时，部分训练轮次会出现理解能力提升的现象，而采用 ℓ_1 正则化时则从未出现此类现象。

2. ℓ_1 **正则化**我们会在损失项中加入 ℓ_1 惩罚项。此处采用的 $\lambda \in \{1, 10, 100\}$。需要说明的是，我们并未将针对该惩罚项的更新操作与针对对数损失的优化步骤分开处理（这与通过AdamW算法实现正则化的Loshchilov与Hutter（2017年）的做法不同）。

在每种实验设置下，我们都使用了三个不同的随机种子进行测试，相关结果展示在图28中。虽然使用 ℓ_1 正则化时并未出现理解能力提升的现象，但我们发现，当采用丢弃机制并结合$p = 0.2$ 或 $p = 0.5$ 时，三个随机种子对应的实验均出现了该现象。我们认为这是因为丢弃机制与权重衰减都会促使网络分散计算过程，而这正是傅里叶乘法算法所要求的；而 ℓ_1 正则化则会促使网络在神经元基中变得更加稀疏，进而在傅里叶基中的稀疏程度降低，从而阻碍网络学习傅里叶乘法算法。

D.2 弹弓机制虽经常出现，但对实现理解能力提升而言并非必需

如C.2节中所述，我们使用的两层变换器在训练过程中会出现明显的弹弓效应（Thilak等人，2022年）。我们认为，这是由于不同量级的梯度与自适应优化器之间相互作用所导致的。



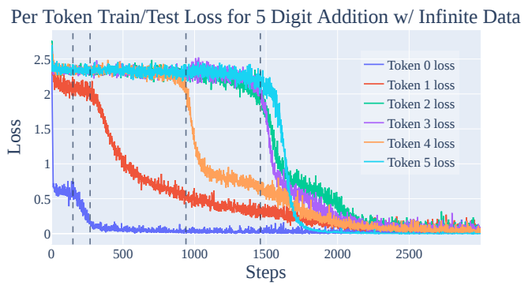


Figure 29: (*Top*) The training/test loss for 5 Digit Addition trained on randomly generated data. Note that training and test loss coincide, as the model does not see repeated pairs.(*Bottom*) The train/test loss *per token* for 5 Digit Addition, trained with randomly generated data at each step. Note that phase changes in the average loss correspond to phase changes in individual tokens, though one phase change (token 1, around step 270) is not visible on the averaged loss as it overlaps with the end of the first phase change (token 0, starting around step 150).

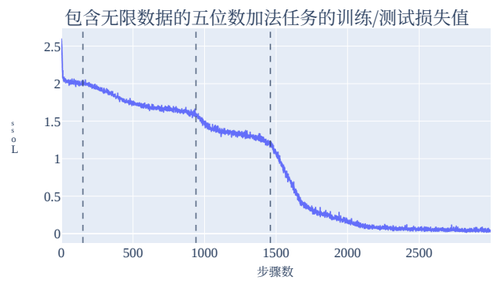
optimizers. We were even able to induce slingshots on a 1-layer by reducing the precision of the loss calculations (as this causes many gradients to round to 0 and thus greatly increases the differences in scale of gradients).

However, as many of our 1-layer models do not exhibit slingshots but nonetheless grok, the slingshot mechanism is unnecessary for grokking to occur, in the presence of weight decay or other regularization. We speculate that the slingshots of Thilak et al. (2022) (which co-occur with grokking for training runs *without* weight decay) serve as an implicit regularization mechanism that favors the simpler, generalizing solution over the more complicated

D.3 Additional evidence from other algorithmic tasks

We now provide addition analysis of grokking phenomena on 3 additional algorithmic tasks and confirm that limited data is an important part of grokking:

1. **5 digit addition.** We sample pairs of random 5 digit numbers and have the model predict their sum
2. **Predicting repeated subsequences.** We take a uniform random sequence of tokens, randomly choose a subsequence to repeat, and train the model to predict the repeated tokens.
3. **Skip trigram.** We feed in a sequence of tokens from 0 to 19, of which exactly one is greater than or equal to 10, and the model needs to output the token that is ≥ 10. This can be solved with learning 10 skip trigrams.



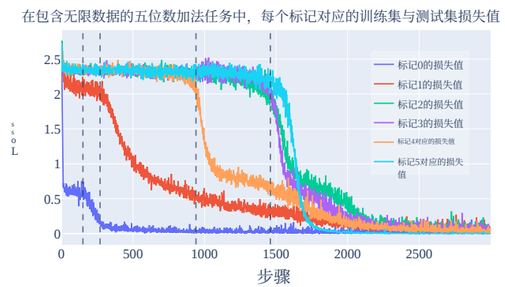


图29：（顶部）在随机生成的数据上训练的五位数加法任务的训练集与测试集损失值。由于模型不会遇到重复的输入对，因此这两者的数值是完全一致的。（底部）展示的是在每一步均使用随机生成数据训练五位数加法任务时，每个标记对应的训练集与测试集损失值。按标记计算的数值。需要注意的是，平均损失值的相位变化对应着各个单个标记的相位变化，不过其中一次相位变化（即标记1，在大约第270步时）并未在平均损失值上显现出来，因为它与第一次相位变化（标记0，大约从第150步开始）的时间段重叠了。

我们甚至通过降低损失值计算的精度，在仅包含1层结构的模型中也引发了弹弓效应——因为这种做法会导致许多梯度被四舍五入为0，进而大幅增大各梯度之间的量级差异。

不过，由于我们的许多单层模型虽然不存在弹弓效应却依然具备理解能力提升，因此在存在权重衰减或其他正则化手段的情况下，弹弓机制对于实现理解能力提升其实并非必需。我们推测，Thilak等人（2022年）在那些没有权重衰减的训练过程中与理解能力提升同时出现的弹弓效应，实际上是一种隐式的正则化机制，它更倾向于选择更为简单且具有泛化解性质的模型，而非更为复杂的模型

D.3 其他算法任务提供的补充证据

接下来，我们将针对另外3种算法任务对理解能力提升现象进行进一步分析，并证实数据量有限是影响理解能力提升的重要因素。

1. **五位数加法**。我们会抽取随机生成的五位数对，让模型预测这两者的求和结果。2. **预测重复子序列**。我们选取一个由标记构成的均匀随机序列，随机选定一个子序列进行重复，然后训练模型预测这些重复出现的标记。3. **跳过三词组任务**。我们会输入一个包含0到19的标记序列，其中恰好有一个标记大于或等于10，模型需要输出那个为 ≥ 10的标记，而这一问题可以通过学习10个跳过三词组模式来解决。

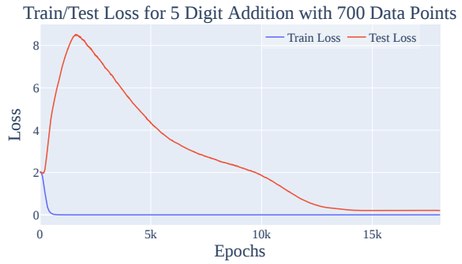


Figure 30: The train and test loss for 5 Digit Addition trained on 700 data points. Unlike the infinite, randomly generated data case, this shows both a sharp phase change and clear train test divergence.

We use a 1-layer full transformer for 5-digit addition, a 2-layer attention only transformer for predicting repeated subsequences, and a 1-layer attention only transformer for the skip trigram task. Otherwise, we use the same hyperparameters as in the mainline model.

5 Digit Addition We first consider the case where we train on the approximately infinite data regime. For each minibatch, we randomly new sample 5 digit numbers. We report the results in Figure 29. Train loss coincides with test loss, so grokking does not occur, as the model almost never sees the same pair of 5 digit numbers twice, with 10^{10} such pairs. Interestingly, the various small bumps in Figure 29 correspond to the model learning how to calculate each of the 6 tokens in the output. However, grokking does occur when we restrict the model to only see 700 data points, as shown in Figure 30.

Repeated subsequence As with the 5-digit addition task, we find that restricting the amount of data is necessary and sufficient for grokking on the repeated subsequence task. In Figure 31, the model sees new data at every step exhibits no grokking. In contrast, clear grokking occurs when we restrict the model to only see 512 data points in Figure 32.

Skip trigram As with the previous tasks, we find that restricting the amount of data is necessary and sufficient for grokking on the skip trigram task. The model that sees new data at every step exhibits no grokking in Figure 33. Meanwhile, the model restricted to only see 512 data points exhibits clear grokking in Figure 34.

Taken together, these results echo the importance of limited data for grokking.

E Further speculations on grokking

E.1 An intuitive explanation of grokking

In this section, we speculate on what might be happening "under the hood" when a model groks and explore why this phenomena happens. The evidence is only suggestive, so this a promising direction for future research.

Grokking occurs when models, trained on algorithmic tasks with certain hyperparameters, initially overfit the training data where train loss significantly improves while test loss worsens and the two diverge. But later in training, there is a sudden improvement in test loss, so test and train loss converge. In contrast to Power et al. (2022) but in line with Liu et al. (2022), grokking does *not* occur when both train and test loss improve together without the initial divergence, as shown in many of the figures in this paper, for example Figures 2 and 18.

The core issue is that the model has two possible solutions: memorization (with low train loss and high test loss) and a generalization (with low train loss *and* low test loss). In our case, the

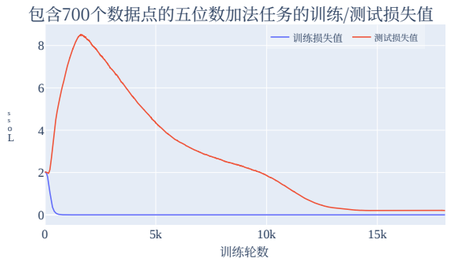


图30：在700个数据点上训练的五位数加法任务的训练损失与测试损失曲线。与无限随机生成数据场景不同，该场景出现了明显的相位变化以及显著的训练集与测试集损失差异。

在处理五位数加法时，我们使用单层的全Transformer模型；在预测重复子序列时，采用两层的仅注意力机制的Transformer模型；而在处理跳过三词组任务时，则使用单层的仅注意力机制的Transformer模型。除此之外，其他超参数均与主线模型保持一致。

五位数加法首先我们研究在近似无限数据的场景下进行训练的情况。对于每个小批量数据，我们会随机抽取五位数作为新样本。相关结果展示在图29中。由于模型几乎从未见过相同的五位数对出现两次，因此训练损失与测试损失完全一致，也就不会发生理解能力提升的现象， 10^{10} 即不会出现这类数据对。有趣的是，图29 中的那些细微波动，实际上对应着模型正在学习如何计算输出结果中的6个标记。不过，如图30所示，当将模型训练的数据量限制为700个数据点时，理解能力提升现象确实会出现。

重复子序列任务与五位数加法任务类似，我们发现，在重复子序列任务中，要实现理解能力提升，限制数据量既是必要条件也是充分条件。在图31中，模型在每一步都会接触到新数据，这种情况下并未出现理解能力提升的现象。而相反，如图32所示，当将模型训练的数据量限制为512个数据点时，便能明显观察到理解能力提升的出现。

跳过三词组任务与之前的任务类似，我们发现，在跳过三词组任务中，限制数据量既是实现理解能力提升的必要条件，也是充分条件。在图33中可以看出，那些在每一步都会接触新数据的模型并未展现出理解能力提升的现象；而仅被限制为处理512个数据点的模型则在图34中明显出现了理解能力提升的表现。

综合来看，这些结果进一步印证了在实现理解能力提升的过程中，限制数据量具有至关重要的意义。

E 关于理解能力提升的进一步探讨

E.1 对理解能力提升的直观解释

在本节中，我们将探讨模型在实现理解能力提升时其内部可能发生的过程，并分析这一现象出现的原因。目前的相关证据仅为推测性质，因此这为未来的研究指明了一个极具前景的方向。

当模型在带有特定超参数的算法任务上经过训练后，最初会因为过拟合而导致训练损失显著下降，而测试损失则上升，二者出现分化。但在训练后期，测试损失会突然开始下降，进而使得训练损失与测试损失趋于一致。这与Power等人（2022年）的研究结论不同，却与Liu等人（2022年）的发现相符。正如本文中的许多图表所示，例如图2 和18，如果训练损失和测试损失同时下降且没有最初的那种分化现象，那么理解能力提升就不会发生。

核心问题在于该模型存在两种可能的解决方案：记忆阶段（此时训练损失较低而测试损失较高），以及具备泛化能力的情况（此时训练损失和测试损失都较低）。在我们的案例中，

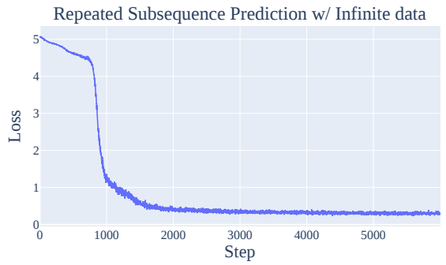


Figure 31: The training/test loss for repeated subsequences trained on randomly generated data. Note that training and test loss coincide, as the model does not see repeated pairs. There sharp phase change corresponds to the model forming induction heads. (Olsson et al., 2022)

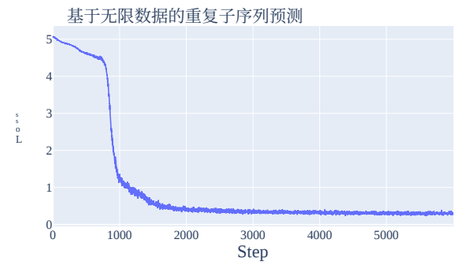


图31：在随机生成数据上训练的重复子序列对应的训练集与测试集损失值。由于模型无法识别重复的词对，因此训练集与测试集的损失值完全一致。而出现的剧烈相位变化则对应着模型正在形成归纳头。（Olsson等人，2022年）

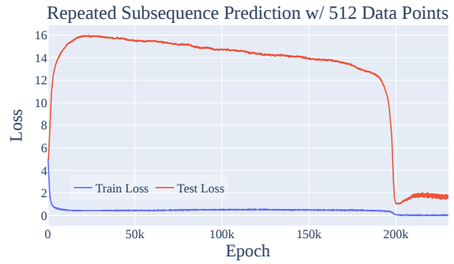


Figure 32: The train and test loss for the repeated subsequence task, trained on 512 data points. Unlike the infinite, randomly generated data case, this shows both a sharp phase change and clear train test divergence.

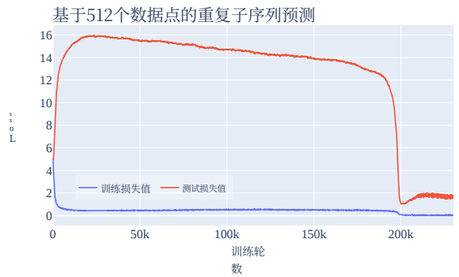


图32：在512个数据点上训练的重复子序列任务对应的训练集与测试集损失值。与无限随机生成数据场景不同，这种情况下不仅会出现剧烈的相位变化，还会出现明显的训练集与测试集损失差异。

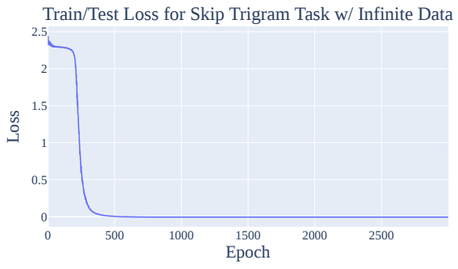


Figure 33: The training/test loss for the skip trigram task, trained on randomly generated data. Note that training and test loss coincide, as the model does not see repeated pairs. The sharp phase change corresponds to the network learning all of the skip trigrams.

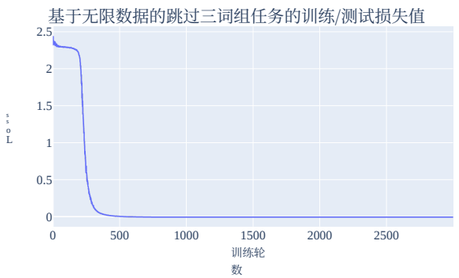


图33：在随机生成数据上训练的跳过三词组任务对应的训练集与测试集损失值。同样因为模型无法识别重复的词组，训练集与测试集的损失值也保持一致。此处的剧烈相位变化则意味着网络已经学会了所有的跳过三词组模式。

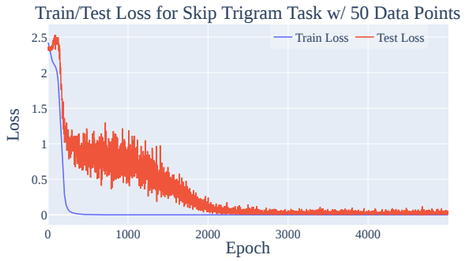


Figure 34: The train and test loss for the skip trigram task, trained on 512 data points. Unlike the infinite, randomly generated data case, this shows both a sharp phase change and clear train test divergence.

Fourier Multiplication Algorithm is the generalization solution. Intuitively, with very little training data, the model will overfit and memorize. With more training data, the model must generalize or suffer poor performance on both train and test loss. Since neural networks have an inductive bias favoring "simpler" solutions, memorization complexity scales with the size of the training set, whereas generalization complexity is constant. The two must cross at some point! Yet, the surprising aspect of grokking is the abrupt shift during training, when the model *switches* from memorization to generalization.

The other component of grokking is phase transitions - the phenomena where models trained on a certain task develop a specific capability fairly rapidly during a brief period of training, as shown for the case of induction heads forming in transformer language models in Olsson et al. (2022) and our results in Appendix D.3. That is, rather than slowly forming that capability over training, the model rapidly goes from being bad at it to being good at it. One interpretation of a phase transition is that there's some feature of the loss landscape that makes the generalising solution harder to reach - rather than a smooth gradient for the model to follow, it instead initially finds it difficult to make progress, but then crosses some threshold where it can rapidly make progress.

Therefore, grokking occurs with phase transitions, limited data, and regularization. Models exhibit phase transitions despite having enough training data to avoid overfitting. Regularization (weight decay in our case) favors simpler solutions over complex ones. The model has enough data to marginally prefer generalization over memorization. The phase transition indicates that generalization is "hard to reach" while the model has no problems with memorization. But as it memorizes, the network becomes more complex until the weight decay prevents further memorization then moves towards equilibrium. The gradient to memorize balances the gradient towards smaller weights. With generalization, the model is incentivized to both memorize and simplify. Strikingly, it is capable of both while maintaining a somewhat constant training performance in this circuit formation phase. Next, as the model approaches generalization, the memorization weights are removed in the cleanup phase. The cost from complexity outweighs the benefit from lower loss. Due to the phase transition during this training period, as model's progress towards generalization accelerates, the cleanup rate sharpens as well.

A model that learns a perfect solution and is trained with weight decay has competing incentives: larger weights (for more extreme logits and thus lower loss) and smaller weights (from weight decay). So for any solution and any level of weight decay, there will always be a level of train loss where these two forces equilibrate. Thus, memorization is not necessarily a "simpler" solution than generalization. The key is that generalization will have smaller weights *holding train loss fixed*. In fact, weight decay should be expected to equilibrate at a slightly lower train loss in generalization, since the base solution is simpler. This matches what we observe in practice. [4]

[4]One subtlety: the grokking phenomena is often incorrectly summarized as "the model learned to generalize even after achieving zero loss." Zero loss does not exist with cross-entropy loss. Although the model achieves

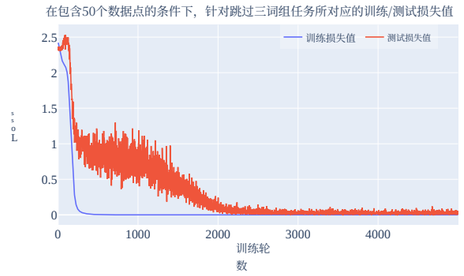


图34：在512个数据点上训练的跳过三词组任务的训练损失与测试损失情况。与无限随机生成数据场景不同，该场景展现了明显的相位变化以及显著的训练集与测试集损失差异。

傅里叶乘法算法正是解决这一问题的通用方案。从直观角度而言，当训练数据量极少时，模型容易过拟合并仅停留在记忆阶段；而当训练数据增多后，模型必须具备泛化能力，否则在训练集和测试集上的表现都会十分糟糕。由于神经网络存在倾向于"更简单"解法的归纳偏置，记忆阶段的复杂度会随训练数据集规模的增大而上升，而泛化能力的复杂度则保持不变。因此这两者必然会在某个点相交！不过，理解能力提升最令人惊讶的地方在于训练过程中的突然转变——此时模型会从记忆模式切换到泛化模式。

理解能力提升的另一个关键因素是相变现象——即模型在针对特定任务进行训练时，能够在短暂的训练期内迅速获得某种特定能力，正如Olsson等人（2022年）的研究以及我们附录D.3节中的结果所展示的那样。也就是说，模型并非在训练过程中逐步形成这种能力，而是会迅速从表现不佳变为表现优异。对于相变现象的一种解释是，损失函数的特性使得达到具有泛化能力的解法变得更加困难——模型最初难以找到合适的优化路径，但一旦越过某个临界点，就能快速取得进展。

因此，理解能力的提升伴随着相变现象、有限的数据量以及正则化处理。即便拥有足够的训练数据以避免过拟合，模型依然会出现相变现象。正则化机制（在本研究中表现为权重衰减）会促使模型倾向于选择更简单的解决方案而非复杂的方案。由于数据量足够，模型在记忆阶段与泛化能力之间能够实现微小的平衡。相变现象表明泛化能力难以达成，而模型在记忆阶段则不存在问题。但随着模型不断进行记忆，其结构会逐渐变得复杂，直至权重衰减阻止进一步的记忆行为，进而使模型趋向稳定状态。推动模型进行记忆的梯度与促使权重减小的梯度相互制衡。在追求泛化能力的过程中，模型既会被激励去记忆信息，也会被激励去简化结构。令人惊讶的是，在这个电路形成阶段，模型即便同时具备这两种能力，其训练性能仍能保持相对稳定。随后，当模型逐渐接近具备泛化能力时，会在清理阶段移除那些用于记忆的权重。此时，结构复杂带来的负面影响已经超过了较低损失值所带来的好处。由于在此训练期间存在相变现象，随着模型向泛化能力迈进的步伐加快，清理的速率也会随之加快。

一个学习了完美解且经过权重衰减训练的模型会面临两种相互矛盾的驱动因素：较大的权重（能够产生更极端的逻辑值，进而降低损失值）与较小的权重（由权重衰减机制所施加）。因此，对于任何特定的解以及任何程度的权重衰减，总存在一个训练损失值，使得这两种力量达到平衡。由此可见，记忆阶段并不一定比泛化能力代表着"更简单"的解决方案。关键在于，在保持训练损失不变的情况下，泛化能力对应的权重会更小。实际上，由于基础解本身更为简单，权重衰减通常会让泛化能力在略低的训练损失水平上实现平衡，这一结论也与我们在实际观察到的情况相符。 [4]

[4]有一个细微之处需要注意：理解能力提升现象常常被错误地概括为'模型在损失值降为零后仍学会了泛化'。实际上，在采用交叉熵损失的情况下根本不存在损失值为零的情况。尽管模型确实能够达成某种优化结果，

E.2 HYPOTHESIS: PHASE TRANSITIONS ARE INHERENT TO COMPOSITION

A promising line of work in the growing field of mechanistic interpretability suggests that models form *circuits* (Cammarata et al., 2020) – clean interpretable algorithms formed by subnetworks of the model, such as curve detectors (Cammarata et al., 2020) in image classification networks and induction heads (Elhage et al., 2021; Olsson et al., 2022) in LLMs. This is surprisingly true! A circuit represents the model learning an algorithm, a fundamentally discrete thing; each step in the algorithm only makes sense if the other steps are present. But neural networks are fundamentally continuous, trained to follow gradients towards lower loss and struggle to jump to new optima without following a smooth gradient. So how can a model learn a discrete algorithm?

As a concrete example, let's consider the case of induction heads in LLMs. There is a subnetwork of a next-token prediction autoregressive language model that learns to continue repeated subsequences. It detects whether the current token occurred earlier in the context. If so, it predicts the same token after that previous occurrence will also come next. The circuit consists of a previous token head, which attends to each previous token and copies the context of the previous token to the current token, and an induction head which attends to the token *after* a previous occurrence of the current token. The induction head composes with the previous token head by forming a query vector representing the current token and a key vector representing the previous token head's output using K-Composition, the context of the previous token. It attends to a token where this query and key match.

This circuit significantly improves loss but only in the context of the other heads present. Before either head is present, no gradient encourages the formation of either head. At initialization, we have neither head, so gradient descent should never discover this circuit. Naively, we might predict that neural networks will only produce circuits analogous to linear regression, where each weight will marginally improve performance as it continuously trains. And yet in practice, neural networks indeed form such sophisticated circuits, involving several parts interacting in non-trivial, algorithmic ways. So how can this be?

A few possible explanations:

- **Lottery tickets (Frankle & Carbin, 2018):** Initially, each layer of the network is the superposition of many partial circuit components, and the output of each layer is the average of the output of each component. The full output of the network is the average of many different circuits, with significant interference from non-linear interaction. Some of these circuits are systematically useful to reducing loss, but most aren't. Gradients for useless circuits will have zero mean, while gradients for useful circuits will have non-zero mean, with a lot of noise. SGD reinforces relevant circuits and suppresses useless ones, so circuits will gradually form.
- **Random walk:** The network wanders randomly around the loss landscape until it encounters a half-formed previous token head and induction head that somewhat compose. This half-formed circuit becomes useful for reducing loss, so gradient descent completes the circuit.
- **Evolution:** A similar mystery arises from how organisms develop sophisticated machinery, like the human eye. Each part is only useful in the context of other parts. A compelling explanation is a component first developed that was somewhat useful in its own right, like a light-detecting membrane. It was reinforced as a useful component. Then, later components developed depending on the first, like the lens of the eye.

Evolution is a natural explanation, However, based on our toy tasks, it cannot be the whole story. In the repeated subsequence task, we have a sequence of uniform randomly generated tokens, apart from a repeated subsequence at an arbitrary location, e.g. 7 2 8 3 1 9 3 8 3 1 9 9 2 5 END. This means all pairs of tokens are independent, apart from pairs of equal tokens in the repeated subsequence. In particular, this means that a previous token head can never reduce loss for the current token. The previous token will always be independent of the next token. So a previous token head is only useful in the context of an induction-like head that completes the circuit. Likewise, an induction head relies

perfect *accuracy*, it is trained to optimize loss not accuracy. This means the model is *always* incentivized to further improve. In particular, the easiest way to improve performance with perfect accuracy is by scaling up the logits. This lowers the temperature and pushes the softmax closer to an argmax.

E.2 假设：相变现象是组合运算所固有的。

在不断发展的机制可解释性研究领域中，有一项极具前景的研究方向指出，模型会形成电路结构（Cammarata等人，2020年）——即由模型的子网络构成的、具备良好可解释性的算法，例如图像分类网络中的曲线检测器（Cammarata等人，2020年），以及大语言模型中的归纳头（Elhage等人，2021年；此外在LLMs中还有Olsson等人的相关研究（Olsson等人，2022年）。这一发现着实令人惊讶！电路结构意味着模型正在学习一种本质上属于离散类型的算法，因为该算法的每一步只有在其他步骤存在的前提下才有意义。而神经网络本质上是连续的，它们通过遵循梯度来降低损失值，若没有平滑的梯度引导，就很难跃迁到新的最优解。那么，模型究竟是如何学习这种离散算法的呢?

作为一个具体的例子，我们以大语言模型中的归纳头为例进行说明。这类自回归语言模型的下一个标记预测子网络能够学习如何延续重复出现的子序列，它会判断当前标记是否曾在上下文中出现过；如果出现过，它就会预测在之前的出现之后同一个标记会再次出现。该电路结构包含一个先前标记头，它负责关注每一个之前的标记，并将之前标记的上下文信息传递给当前标记；同时还包含一个归纳头，它负责关注在当前标记之前 出现的标记。归纳头会利用K组合运算，结合先前标记头的输出以及之前标记的上下文，生成表示当前标记的查询向量和表示先前标记头输出的键向量，进而关注那些查询向量与键向量相匹配的标记。

这种电路结构确实能够显著降低损失值，但前提是必须存在其他注意力头。在没有任何注意力头存在的情形下，不会有任何梯度促使这些注意力头形成。在模型初始化阶段，两种注意力头都不存在，因此随机梯度下降法根本不可能发现这种电路结构。从直觉上来看，人们可能会认为神经网络只会生成类似于线性回归模型的电路结构——在这种结构中，每个权重在持续训练的过程中都会略微提升模型性能。然而实际上，神经网络确实能够构建出如此复杂的电路结构，其中多个组成部分以非平凡的、基于算法的方式相互协作。那么这种现象究竟是如何产生的呢?

以下是几种可能的解释：

- **彩票假说（Frankle& Carbin，2018）认为：**最初，网络中的每一层都是由许多部分电路组件叠加而成的，每一层的输出则是这些组件输出值的平均值。整个网络的最终输出则是众多不同电路结构输出值的平均值，而这种非线性相互作用会带来显著的干扰效应。在这些电路结构中，有些确实有助于降低损失值，但大多数并无此效果。无用电路结构的梯度均值将为零，而有用电路结构的梯度均值则不为零，且其中包含大量噪声。随机梯度下降法会强化那些有用的电路结构，同时抑制无用的结构，如此一来，电路结构便会逐渐形成。
- **随机游走：**网络会在损失函数所构成的景观中随机探索，直到遇到尚未完全形成的标记头与归纳头，二者稍加组合便能形成初步的电路结构。这样的半成品电路有助于降低损失值，随后通过梯度下降法完成整个电路结构的构建。
- **进化过程：**生物体如何演化出诸如人类眼睛这样复杂的器官，同样存在类似的谜题。每个组成部分只有与其他部分相互配合时才能发挥作用。一个很有说服力的解释是：最初演化出的某些组件本身就具备一定的实用价值，比如能够感知光线的膜结构，它因具有实用性而得到进一步强化。之后，其他组件则基于最初的这些组件逐渐发展起来，例如眼睛中的晶状体。

进化论是一种自然的解释方式，然而根据我们设计的简化任务来看，它并不能解释全部现象。在重复子序列任务中，我们会使用一串随机生成的均匀标记，其中仅在任意位置出现重复子序列，例如 7 2 8 3 1 9 3 8 3 1 9 9 2 5 END。这意味着除了重复子序列中的相同标记对之外，所有标记对都是相互独立的。特别地，这表明前面的标记头永远无法降低当前标记的损失值，因为前面的标记始终与后面的标记保持独立关系。因此，单独的标记头只有在与类似归纳头的结构配合、从而构成完整的电路结构时才有用。同样地，归纳头也依赖于

即便准确率达到完美状态，模型被训练的目标依然是优化损失值而非准确率。这意味着模型会始终被激励去进一步提升性能。尤其需要注意的是，要想在准确率已完美的情况下提升性能，最简单的方法就是调整逻辑值的大小——这会降低温度参数，从而使softmax函数的输出更接近argmax值。

on K-composition with a previous token head and so cannot be useful on its own. Yet the model eventually forms an induction circuit!

A priori, the random walk seems insufficient on its own. An induction circuit is relatively complicated, representing a small region in model space. So a random walk is unlikely to stumble upon it. Concretely, in our modular addition case, progress measures show significant hidden progress pre-grokking, indicating the model did not stumble upon the solution by chance.

Thus, the lottery ticket hypothesis seems the most explanatory. An induction head is useless without a previous token head but might be slightly useful when composing with a head that uniformly attends to prior tokens, since part of its output will include the previous token! Nevertheless, we suspect that all explanations contribute to the entire picture. This seems most plausible if the uniform head just so happens to attend a bit more to the previous token via a random walk.

Returning to phase transitions, the lottery ticket-style explanation suggests that we might expect phase transitions as circuits form. Early in circuit formation, each part of the circuit is rough, so the effect on the loss of improving any individual component is weak, meaning gradients will be small. As each component develops, other components will become more useful, meaning all gradients will increase together non-linearly. As the circuit nears completion, we should expect an acceleration in the loss curve for this circuit, resulting in a phase transition.

F Further discussion on using mechanistic interpretability and progress measures for studying emergent phenomena

While we find approach of using mechanistic interpretability to define progress measures relatively promising, there remains significant uncertainty as to how scalable existing mechanistic interpretability approaches really are. Broadly speaking, depending on the success of future mechanistic interpretability work, we think there are three methods through which mechanistic interpretability and progress measures can help with understanding and predicting emergent phenomena:

1. If mechanistic interpretability can be scaled to large models to the level where we can understand the mechanisms behind significant portions of their behavior, we could perform the same style of analysis as was done in this work. We believe it's currently unclear as to whether or not mechanistic interpretability will successfully scale to large models to this extent (or even if there exist human-understandable explanations for all of their sophisticated behavior). That being said, in cases where mechanistic interpretability does recover human-understandable mechanisms, we could simply use the parts of the mechanism as progress measures.
2. If future mechanistic interpretability can only recover parts of the mechanism of larger models (as in Wang et al. (2022)) and can only generate comprehensive understanding of the mechanisms of smaller models, we might still be able to use our understanding from smaller models to guide the development measures that track parts of the behavior of the larger model. We find this scenario relatively plausible, as existing mechanistic interpretability work already allows us to recover fragments of large model behavior and understand these fragments by analogy to smaller models. For example, Olsson et al. (2022) use this approach to understand the emergence of in-context learning in medium-sized language transformers.
3. Even if mechanistic interpretability fails to recover understandable mechanisms at all on large models, we might still be able to derive progress measures that don't require human understanding. For example, if we end up with automated mechanistic interpretability (that nonetheless still fails to recover human-understandable mechanisms), we might be able to use the outputs of those opaque processes.

   Another approach is task-independent progress measures: if we can discover progress measures that don't depend on the task, perhaps using many small, interpretable models as testbeds, we might be able to apply these progress measures to large models.

That being said, we think the future work outlined in Section 6 is necessary to successfully apply our approach to predict and understand emergent behavior in existing large language models, and so remain cautiously optimistic.

结合包含前面标记头的K组合运算，因此仅靠自身是无法发挥作用的。尽管如此，模型最终依然能够构建出归纳电路结构！
从初步分析来看，仅依靠随机游走似乎并不足够。归纳电路的结构相对复杂，仅占据模型空间中的较小区域，因此随机游走很难偶然发现它。以我们的模加法案例为例，进展度量指标显示在预理解阶段存在显著的隐性进展，这表明模型并非偶然找到解决方案。

因此，彩票假说似乎是最能解释这一现象的理论。如果没有先前的标记头，归纳头便毫无用处；但当它与能够均匀关注先前标记的头结合使用时，或许会稍有作用，因为其输出中会包含之前的标记内容！尽管如此，我们认为所有这些解释共同构成了完整的图景。如果这种均匀关注的头恰好通过随机游走而更多地关注到先前标记，这一解释就显得最为合理。

再回到相变现象上，彩票票假说认为，随着电路结构的逐步形成，我们或许会观察到相变现象。在电路形成初期，电路的各个部分都较为粗糙，因此单个组件的改进对损失值的影响较弱，对应的梯度也会很小。随着各组件逐渐发展，其他组件的作用会日益凸显，此时所有组的梯度都会呈非线性方式同步上升。当电路接近完成时，该电路的损失曲线会出现加速下降的趋势，进而引发相变现象。

F值：关于运用机制可解释性及进展度量指标研究涌现现象的进一步探讨

尽管我们认为利用机制可解释性来定义进展度量指标是一种颇具前景的方法，但目前仍存在较大不确定性，难以确定现有的机制可解释性方法在实际应用中的扩展能力究竟如何。总体而言，视未来在机制可解释性研究方面取得的成果而定，我们认为该技术可通过三种途径帮助我们理解并预测涌现现象。

1. 如果能够将机制可解释性扩展到大型模型上，使其达到让我们能够理解这些模型大部分行为背后机制的水平，那么我们就可以采用与本研究所采用的类似分析方法。目前尚不清楚机制可解释性是否真的能扩展到这种程度的大型模型上（甚至也不清楚这些模型所有复杂行为是否都能找到人类可以理解的解释）。不过，即便在机制可解释性确实能够揭示出人类可理解的机制的情况下，我们也可以直接将这些机制的相关部分作为进展度量指标使用。

2. 如果未来的机制可解释性技术仅能还原更大规模模型的部分机制（如Wang等人（2022年），且只能生成对较小规模模型机制的全面理解，我们或许依然可以利用从较小模型中获得的认知来指导那些用于追踪大型模型部分行为的进展度量指标。我们认为这种情景具备一定的合理性，因为现有的机制可解释性研究已经能够让我们还原大型模型行为的片段，并通过类比小型模型来理解这些片段。例如，Olsson等人（2022年）就采用了这种方法来探究中等规模语言Transformer中上下文学习现象的产生机制。

3. 即便机制可解释性技术完全无法在大型模型上还原出可被人类理解的机制，我们仍有可能推导出无需依赖人类理解的进展度量指标。例如，即便我们只有自动化机制可解释性技术（而该技术依然无法还原出人类可理解的信息），我们或许依然可以利用那些不透明处理过程的输出结果。

   另一种方法是采用与任务无关的进展度量指标：如果我们能够找到不依赖于具体任务的进展度量指标，或许可以通过将许多小型、可解释的模型作为测试平台，进而将这些指标应用于大型模型上。

话虽如此，我们认为第6 节中提出的后续工作对于成功应用我们的方法来预测和理解现有大型语言模型中的涌现行为至关重要，因此我们仍保持谨慎的乐观态度。